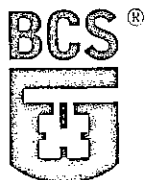


VECTOR

All that's new in APL:

- San Quirico & Minnowbrook 19,30
- APLX v4 in experimental physics 39
- Neville Holmes on Tacit J 52,57
- User-defined Classes in APLX 67
- Working with the .Net DataSet 89
- Smillie plays with Meccano 108
- Reiter in the Adirondacks with J 115



*The Journal of the
British APL Association*

ISSN 0955-1433

www.vector.org.uk

Vol.23 No.3 June 2008

A specialist Group of the British Computer Society

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file or an HTML document. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2 or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from the website), and MS Word (any version, ideally using the Vector template which is also available from the website).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 2007-2008

Category	Fee	Vectors	Passes
UK Private	£20	1	1
Overseas Private	£22	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	5	5
Overseas Corporate	£110	5	5
Sustaining	£500	10	5
Non-voting Member (Student, OAP, unemployed)	£10	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or Amex, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 5 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to Gill Smith, Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385.

Email: gill@apl385.com.

Changes at the British APL Association

At the Extraordinary General Meeting held on 19 June 2008 at the BCS headquarters in London the result of the vote on the resolution to end our affiliation to the British Computer Society was announced. The votes were cast 184 in favour, and 1 against. The resolution was therefore passed.

The Annual General Meeting that followed was held under the amended regulations which will be posted in *Vector* 23.4.

In accordance with the resolution:

1. I shall write to BCS notifying them of the change and thanking them for their administrative help over the years.
2. Our Treasurer will open a new bank account for the Association and request repayment of our funds from BCS.

You might be aware BCS claims ownership of our funds. I am pleased to say Richard Nabavi is assisting me in negotiating their return. Your committee has fair hopes of success; it also has plans for carrying on without them.

While we would be glad to recover the money, our future does not depend on it. Please be assured that the Association will continue to function as before, dedicated to furthering APL. Production of *Vector* will continue. In addition it was decided that our fee structure will remain unchanged for the year 2008-9

We expect to distribute *Vector* 23.4 before the end of September. When we do, the Treasurer will ask for your subscription for the year beginning 1 May 2008. If you are happy to pay your subscription now, please simply send your cheque to the Treasurer.

We expect to recommend further changes to our structure, roles, responsibilities and regulations but think it best to change, right now, only what we must. We thank you for your support during this change, and hope to continue to receive it.

Your committee is confident this move will provide new opportunities, and will invigorate and free us to inject a new dynamism into the APL community.

Further announcements will be made as we progress.

Paul Grosvenor
Chairman British APL Association
25th June 2008



Contents

		Page
Editorial: Amateur Hour	Stephen Taylor	3
NEWS		
News from Sustaining Members		6
Product Guide & Address Book	Gill Smith	11
RECENT MEETINGS		
The Moot at San Quirico	Adrian Smith	19
Minnowbrook Revisited	Adrian Smith	30
DISCOVER		
Alive and Well	Morten Kromberg	34
Review: APLX version 4 – from the viewpoint of an experimental physicist	Beau Webber	39
Tacit J and I	Neville Holmes	52
LEARN		
Functional Calculation 0 : Numerical Apéritif	Neville Holmes	57
Tutorial: User-Defined Classes in APLX Version 4	Simon Marsden	67
J-ottings 49: Are you thinking what I'm thinking?	Norman Thomson	84
Using the .Net DataSet with Dyalog 12	Adrian Smith	89
SALT: a Simple APL Library Toolkit	Dan Baronet	103
In Session "Under the Table"	Stephen Taylor	106
PROFIT		
A J Simulator for a Meccano Differential Analyzer	Keith Smillie	108
GPS and J Propel an Adirondack Hiking Guide	Cliff Reiter	115
Index to Advertisers		127



Girls in the Rain in Tuscany

Back from lunch at the San Quirico Moot

Highlights of the APL Year in 2007

**The June Moot at
San Quirico d'Orcia
and
Minnowbrook
in the Fall**



Moonset over Blue Mountain Lake

From the deck outside the classroom

Editorial: Amateur Hour

Stephen Taylor (editor@vector.org.uk)

Anyone reading *comp.lang.apl* these days could be forgiven for thinking APL is enjoying a revival. The boards are more active than they have been for a decade.

The vendors are doing nothing to discourage this view. The last year has seen a major release from MicroAPL and two from Dyalog. Both vendors now support native class-instance relations and Dyalog has made the jump to Unicode as its default encoding, breaking down a long-standing barrier between APL and the mainstream.

At the same time, Kx Systems' and Dyalog's new licensing regimes have seen hundreds of student and personal licences issued this winter and new users are appearing on the boards.

The J community has produced copious learning material over the years: labs and documentation installed with J, textbooks, and recently the superb J Wiki. This spring has seen publication of Jeffrey Borror's *Q for Mortals*, and he is rumoured to have started on its sequel, *Q for Minor Deities*. New users of the older APLs can only await the new textbook Dyalog has commissioned from Bernard Legrand.

We face a need once more to explain what we do: what is special about it and what is it good for? The answers to these questions matter a great deal, because they determine how we should invest our scarce marketing and development resources.

Two schools emerge. Either or both answers could be right, but they indicate very different investments.

One school points to the larger opportunities available today for programs written in languages outside the main stream. APL's special skills can find employment in this world, encapsulated in Web Services, ActiveX controls, web applications and DLLs. What matters in this scenario is to be a team player and participate in a world of professional programming dominated by Visual Studio. This is the vision that drives Visual APL.

The other school points to APL's established record of enabling non-professional programmers to exploit their domain expertise without the trouble of acquiring or hiring advanced programming skills. What matters in this scenario is providing tools that are both light enough for non-professionals and strong enough for

commercial use. Dyalog's native GUI classes, for example, aim at this: to provide enough tools for substantial applications, without requiring mastery of the deep arts of writing GUI. On this view, most APL programmers will always be amateur programmers. This is not to deny a role for professional APL programmers. Software that makes it out of the garage and into commercial use usually needs professional help to remain strong.

While we are waiting for time to tell which of these two courses proves profitable, we do well to respect both, and refrain from familiar grievances about how the technology never got its due. In this vein, we reprint Morten Kromberg's post from `comp.lang.apl` and invite more discussion there, or in articles for *Vector*, on what the APL 'proposition' is.

Also in this issue: Adrian Smith's reports of Paul Mansour's productive and moot *de luxe* in San Quirico d'Orcia, and of the revived Minnowbrook conference series. Neville Holmes' essay kicks off his series of articles about functional programming in J. We're pleased to carry a review of APLX version 4 from Beau Webber, a 'gentlemen amateur' in the best sense, and Simon Marsden's tutorial on using APLX's new classes. Norman Thomson thinks about how J helps him think. Adrian Smith uses .Net's "look, no database" dataset. Dan Baronet introduces script management tools for Dyalog. Keith Smillie finds a differential analyser made of Meccano and re-builds it in J. Lastly Cliff Reiter takes us hiking in the Adirondacks.

Next issue will feature Boss on partitions of numbers, Quario on polynomials, Adrian Smith on moving to Unicode, Richard Smith on the Rowan graphical session, Camacho on infinity, Thomson on complex arithmetic, and news of changes in our affiliation to the British Computer Society.

Quick Reference Diary 2008

Date	Venue	Event
10-12 Jun	New York City	SIFMA 2008 (Securities and Financial Markets Assoc.)
12-13 Jun	IBM Hannover	APL Germany Spring Meeting
19 Jun 2-5pm	BCS, London	BAA Annual General Meeting, preceeded by a short EGM to agree future BCS relationship
12-15 Oct	Denmark	Dyalog 2008 Conference at LO-skolen, Helsingør
10-11 Nov	Bethesda, MD	APL2000 User Conference

Dates for Future Issues of VECTOR

	Vol.23 No.4	Vol.24 No.1
Copy date	21st June	21st Sept
Distribution	August 2008	December 2008

Vector Back Numbers

Back numbers of Vector are available from:

British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Please note that Vol.1 No.2 is now out of stock.

NEWS

News from Sustaining Members

APL 2000

APL2000 User Conference 2008

The APL2000 User Conference 2008 will be held 10-11 November at the Hyatt Regency Bethesda, in Bethesda, Maryland. A special pre-conference weekend package, including breakfast, will be available for anyone wishing to spend a few extra days visiting Washington, D.C.

The hotel is conveniently located on the Metro Red Line, just a short ride from downtown Washington, D.C., with easy access to all the monuments, museums and cultural centres. Near by the hotel is the Capital Crescent Trail for cycling, running and walking. Bethesda's vibrant downtown, and its numerous restaurants and shops are within easy walking distance.

There are three airports in the Washington D.C. area – Ronald Reagan Washington National Airport, Washington-Dulles International Airport and Baltimore-Washington International Airport. Metro service is available from Ronald Reagan Washington National Airport to the hotel and shuttle service is available from all three airports to the hotel.

Plan to join us for an exciting conference filled with stimulating presentations, interesting demonstrations, shared ideas and camaraderie with the larger APL community.

Dyalog Ltd

Dyalog 2008

The User Group Conference this year will take place 12-15 October 2008. Many of you will be pleased to learn that we will be returning to LO-Skolen in Elsinore in Denmark for this year's Conference. For those of you who have not visited LO-Skolen before we can highlight the interesting architecture which has made this conference centre truly unique. The atmosphere in the main building is very inspiring and caters for intense networking as there are lots of areas throughout

the venue for sitting down and having an informal talk. The kitchen has never failed to impress us deeply either.

From a transport point of view the venue is very easy to get to. From Central Copenhagen as well as from the airport there are trains straight to Elsinore – via Copenhagen Central Station (approx 1-hour journey). Train journeys are very reasonably priced in Denmark and a one way ticket can be had at DKK 90 (around £8.50) at current pricing.

We're in the process of planning the conference in more detail and as always would be delighted to receive your input with regards to topics for presentations and training courses.

If you have a suggestion for a presentation you would like to give at the conference we'd also like to hear from you. Please email us.

For those of you who like to plan long-term already now we can announce that:

- Dyalog 2009 will take place in the USA in the autumn. Dates and venue to be announced nearer the time.
- Dyalog 2010 will take place in Berlin (Germany) – for the moment the timetable says July.

Dyalog Version 12

Support for Unicode character data. Version 12.0 allows applications to store and manipulate Unicode character data. Applications can now accept entry of data in any of the world's writing systems – from Amharic through APL, Chinese and Tolkien's Elvish script to Zulu. All components of Dyalog support Unicode: the development environment itself, the Dyalog language and file systems, built-in GUI objects, and interfaces to COM/OLE, .Net and ODBC.

The adoption of Unicode also means that data is no longer translated as it enters and leaves the Dyalog interpreter. APL source code can be stored in Unicode text files and edited using any editor which supports these files. A simple source-code management system and an interface to the popular Subversion system are included.

Conga for simplified communications. Conga is a mechanism for developing client and server components that communicate with other web or network components. Conga supports client/server programming, and includes sample code to implement web clients and servers. From version 12.0.2 Conga supports Secure communications using the Secure Socket Layer (SSL).

Built-in charting. We're happy to tell you Version 12 includes the Causeway product series: RainPro – including the .NET version, SharpPlot for high-quality business graphics, and NewLeaf for reporting and typesetting. You can produce a graphical representation of numerical data in seconds by clicking on a chart icon whilst running your session – or write your own APL scripts to produce high-quality reports with integrated business graphics.

Journalling component files. The built-in array file system is now a journalling file system, which means component files can no longer be damaged by network failures or system crashes which occur in the middle of a file update.

Language bar. All APL symbols are presented in a Language Bar in the session, with tips to remind you of the definition of each – and a link to comprehensive on-line help. The language bar can be completely configured to provide help for your own functions.

On-screen keyboard (Microsoft Windows only). Hold the Control, Alt or AltGr key down for a split second, and you will be presented with a pop-up keyboard which shows the location of APL symbols on your keyboard, and allows you to pick them off using the mouse. Dyalog version 12.0 comes with standard Windows APL keyboards for a dozen languages – and you can easily edit your own keyboards using the Microsoft Keyboard Layout Creator.

Unicode & Classic editions. When converting existing applications to use Unicode character data, some code conversion may be required. For many applications, the required changes may be trivial; some applications will run without any changes. However, large applications with many interfaces to external data may require significant work – if for no other reason than that the external databases and interfaces need to be upgraded in order to store Unicode data. Users who have developed Auxiliary Processors and Dynamic Link Libraries will need to upgrade these to handle Unicode data.

For this reason, version 12.0 and the next several versions of Dyalog will be available in two variants: a Unicode edition which uses Unicode for all external interfaces, and a Classic edition which uses the same interfaces and translate tables as 11.0 and earlier versions of Dyalog.

Workspaces and files can be shared between the Unicode and Classic editions. Both editions also provide extensive inter-operability with earlier versions. Component files can be shared with old versions subject to certain limitations (for example, the files must contain data which can be represented in the old systems).

New online libraries for help and documentation. We have created two new online libraries which are now publicly available via the website. The Help library is created as a structured topic tree which you can expand. There is also a search function where you can either select from a menu or type in your search criteria.

The Documentation Centre contains all the available documentation and will be updated on a regular basis as and when new manuals and training materials becomes available. In connection with the Platinum release of Version 12 all the manuals will further be available as PDF downloads as well as print-on-demand via LuLu.

New board member with extensive software development experience

It gives us great pleasure to introduce Michael Holmberg Andersen, who joined the Board of Directors in April 2008 from SimCorp. Michael has occupied the position as Senior Vice President for the IMS Development Department where he is responsible for SimCorp's software development activities.

It gives us great pleasure to welcome Michael to the Board where we are looking forward to taking advantage of his extensive experience to further grow Dyalog Ltd in the years to come.

Kx Systems

Personal Developer Edition of Kdb+

Since making the 32-bit version of Kdb+ freely available more than 1,500 copies have been downloaded. It is our goal to encourage programmers around the world to learn q and Kdb+ in order to meet the increasing demand for talented Kx experts driven by the challenges of growing data volumes in the global financial markets. For more information visit <http://kx.com/developers>

Code.kx.com

Kx programmers are continuing to make significant contributions to our global technical community via code.kx.com. This resource is now available to everyone and we would encourage you to take a look if you haven't already done so. Contributions include examples of code for connecting to other applications and languages.

Q for Mortals

Jeffrey Borror's long-awaited book on the Q language, *Q For Mortals – A Tutorial in Q Programming*, is now available from Amazon. The book provides entry-level explanations for Q and Kdb+ users.

New partner & OEM program

Rising data volumes are choking vendors and many applications are being redesigned and rewritten. Partly as a result of this Kx is receiving increased interest in partnerships from application vendors. We are always happy to work with new partners and to provide connectivity to Kx products or embedded versions of Kdb+. If you would like to find out more, please contact partners@kx.com.

Customer events

Thank you to all those who were able to join us for our London party. If you're in New York don't forget our 15th anniversary celebration on 11 June. The party will be held during the SIFMA event in New York, which we are co-sponsoring. If you are a customer or a partner and have not yet received your invitation please email us at kxevents@kx.com. We're very much looking forward to seeing you there.

Drowning in data

The April edition of *Market Data Insight* carries an article by Simon Garland, Chief Strategist at Kx, on the challenges presented by growing volumes of data, and how to reduce latency.

MicroAPL Ltd

The launch of APLX Version 4 has been very well received by new and existing users. It is available for all the platforms we support (Windows, MacOS, Linux, and AIX), in both Desktop and Server editions. The most important new feature is the extension of the APLX language specification to include object-oriented programming features (classes, objects and inheritance), using a syntax which we think will seem very natural for existing APL users. As well as user-defined classes written in APL, you can also directly use classes and libraries written in other architectures, including .Net, Java, and Ruby. Version 4 also includes many other important enhancements including regular-expression searching, a powerful new general array editor, direct import and export of data in a number of popular formats such as CSV, HTML and UTF-8, and a new 'QuickSym' input method.

We have also been working on enhancing our documentation and tutorials, and we will soon be making all our manuals and a new APL Tutorial available in HTML format on our website, in addition to the PDFs which are already freely downloadable.

The Vector Product Guide

compiled by Gill Smith

VECTOR's Product Guide aims to provide readers with useful information about sources of software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups:

- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

*Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.
Please help us keep it up to date!*

All contributions and updates to the Vector Product Guide should be sent to:
Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385,
Email: gill@apl385.com

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	DETAILS
ADVOCORP Oy	Consultancy	APL application conversions, APL Windows interfaces, APL to APL level interfacing to any system under Windows, TCP/IP network and database connectivity, APL based financial client/server applications, Cognos Planning and ReportNet consultancy.
APL Borealis Inc.	Support and Development	APL Software Support and Development. Specialists since 1979 in Sharp APL, APL*Plus, APL+Win, Dyalog APL
APL Solutions Inc	Consultancy	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1989.
Assured Systems	Consultancy	APL+Win and/or J and/or VB custom systems development. Conversion of APL+DOS to APL+Win a speciality.
AUSCAN Software	Consultancy and Training	APL software development, training
Ray Cannon	Consultancy	APL, C, C++, Assembler, Windows, Graphics on PC and IBM Mainframe. Experience in Insurance, Chemical, and Airline Industries
Causeway	Consultancy and Training	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-Implementation check of Causeway applications.
CoSy	Consultancy	CoSy.com, Coherent Systems, provides rapid development in the K language and associated data base products, with a particular interest in quantitative (financial) modelling.
David Crossley	Consultancy	Experienced in large APL system developments since 1989 for PC or mainframe.
Dinosoft Oy	Consultancy	Specialised in very large databases.
Dittrich & Partner	Consultancy	APL programming and analysis; APL workshops and training on the job
Dyalog	Consultancy	APL and Unix system design, consultancy, programming and training.
DynArray	Consultancy	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evestic AB	Consultancy	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivatives plc	Consultancy	Financial trading software in Q, K and kdb+
First Derivative Analytics Ltd.	Consultancy	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APL/W.
General Software	Consultancy	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
HMW	Consultancy	System design consultancy, programming. HMW specialize in banking and prototyping work. full members of DSDM consortium and Agile Alliance.
Hoekstra Systems	Consultancy	APL consultancy, programming, etc. Also UNIX system administration
INFOSTROY	Consultancy	Broad experience in various APL platforms. Special skills and knowledge in developing complex applications for investment, financial and construction markets. Implementation of hybrid solutions based on APL, Delphi, C#, VBA, SQL servers.
Insight Systems	Consultancy	We have experience with just about every APL system and platform in common use during the last 20 years, from SHARP APL under MVS or Linux to APL+Win and in particular Dyalog APL under Windows 9x, NT or 2000. If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways

KJK	Consultancy and software development	migrations, to the development and support of shrink wrapped solutions based on APL. Even if we don't have time to do the work ourselves, we will know where to find someone who is an expert in your version of APL and your application area, on your continent.
Lambert Technology	Consultancy	APL-based data management: conversions, ad hoc-analyzing tools, well-interfaced methods for defining, processing and browsing of multi-dimensional reports. Rapid custom software development based on proven modular toolset approach.
Phil Last	Consultancy	APL programming, consulting & training; web design and construction.
Lescasse Consulting	Consultancy	APL consultancy, modelling and programming.
Lingo Allegro	Consultancy	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APL/W. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lucas Solutions	Consultancy	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
Milinta Inc	Consultancy	Rates depend on task and location.
Ellis Morgan	Consultancy	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Windows programming.
Nussebaum gift	Consultancy	Business Forecasting & APL Systems.
Optima	Consultancy, Training & Development	IT Consultant with a strong focus on APL.
Orbien Consulting Inc.		We have been in business since 1990 and have a team of APL professionals with many years experience in pharmaceutical, industrial and financial systems on both PC and Mainframe platforms. In addition to pure APL we can also offer assistance with network integration, ACT1 customisation and issues concerning the internet and/or web design.
Graeme Robertson	Consultancy & Training	Consultancy
Skelton Consulting	Consultancy	Design and write custom software. Maintain and upgrade APL systems. Deliver customized APL training courses
Snake Island Research Inc	Consultancy	K/Q Consultancy
Strand Software	Consultancy	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
Rex Swain	Consultancy	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Sykes Systems Inc	Consultancy	Independent consultant, 25 years experience. Custom software development, PC and/or mainframe.
	MACfns	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
ubJL GmbH (APL Software Team)		MACfns has been released. A suite of over 100 high-accuracy, high-speed assembler code utility functions for APL+Win and APL+DOS. MACfns has been thoughtfully designed and implemented for APL programmers. It includes functions for segmented string manipulation, data restructuring, translation and datatype adjustment, and many numerical computations. Typical speedups range from 4-10 times, with reduced workspace usage, datatype conservation, identity detection, and greater numerical precision amongst the other benefits. Documentation of MACfns is extremely detailed and thorough.
		Analyses, design and programming.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification. APL unicode fonts designed and built for all flavours of APL.
APL Borealis Inc.	APL Training	poa	Hands-on courses in Introductory, Intermediate, Advanced and Windows APL. Courses are customized and flexible, and may be delivered on-site, with strong emphasis on methods for efficient and maintainable APL systems development.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email acam@blueyonder.co.uk.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Right Seat Software	Vox Proxy	\$199 (comm) \$69.95 (edu)	Vox Proxy is authorware for PowerPoint(r) 2000 or 2002 which allows the use of Microsoft Agent Technology (3D talking animated characters) within slide shows. VP appears on PowerPoint's main menu and provides editing side-by-side with slides. Automated script-writing provides control of PowerPoint, allowing the use of characters for live presentations or fully-automated tutorials, demos, or training programs. Optional CD Prep program allows the user to create auto-starting CD's that will play on any version of PowerPoint or without PowerPoint. Vox Proxy for educational use.

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Germany e.V.	Germany	APL Journal	Semi-annual meetings	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join Material fee 5,000yen a year
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	The APL Planet	Semi Annual meetings	\$13 (US), \$25 (non-US)
SovAPL	Moscow, Russia	-	Seminars and Annual Meeting	
Toronto SIG	Toronto, Canada		Occasional meetings, APL Skills Database, Toronto Toolkit	

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
Acadvent Ltd.	Dr A M Sykes	171 Gower Road, Swansea SA2 9JH, UK. Tel: 01792-201776 Email: Alan.Sykes@which.net
ACM SigAPL		ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADVOCORP Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@retti.fi
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788365 Email: adrian@apl385.com
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: curtis_jones@prodigy.net
APL Borealis Inc.	Richard Procter	381 Manor Road East, Toronto, Ontario M4S 1S7, Canada. Tel: (416) 457-7828. Fax: (416) 482-6582 Email: info@aplborealis.com
APL Consultants Japan	Kyosuke Saigusa	1507 shimomizo, sagami-hara, kanagawa, Japan 2290015 Tel: +81-042-778-2127 Fax: +81-042-778-2205
APL Germany e.V.	Dr. Reiner Nussbaum	Buchenerstrasse 78, D-68259 Mannheim, Germany. Email: reiner@nussbaum-gift.de
APL Italiana S.p.A.	Stefano Lanzavecchia	via Montefratto 1, 20144 Milano, Italy. Email: stf@apl.it
APL Software/Services	Dick Holt	1570 Heather Field Lane, Earlsville VA 22936 USA. Tel: +1 434-973-4255 email: dickholt@gmail.com
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4821 Fax: +1 (301) 589-4618 Email: aplsi@starpower.net
Assured Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: hillrj@melbpc.org.au
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@ca.inter.net
Beautiful Systems, Inc.	Jim Goff	PO Box 2235 Jenkintown, PA 19046, USA Tel: +1 (215) 635-0375; Fax: +1 (215) 635-9212 Email: info@beautifulsystems.com
Ray Cannon	Ray Cannon	21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: ray_cannon@compuserve.com
The Carlisle Group	Paul Mansour	544 Jefferson Avenue, Scranton PA, 18510 USA. Tel: +1 570-963-2036
Causeway Graphical Systems Ltd	Adrian Smith	Brook House, Gilling East, YORK, UK. Tel: 01439-788413 Email: adrian@causeway.co.uk
Cinerea AB	Rolf Komernark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email: rolf@cinerea.se
ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmed.com
CoSy.com	Bob Armstrong	42 Peck Slip #4B, New York, NY 10038-1725, USA. Tel: +1 212-285-1864 Fax: +1 212-285-1864. E-mail: bob@CoSy.com.
David Crossley	David Crossley	187 Le Tour du Pont, 84210 ST DIDIER, France. Tel: +33.4.90.66.08.87 Email: crossley@au-village.com
Danish User Group	Helene Boesen	c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark
Dinosoft Oy	Pertti Kalliojärvi	Yrjökatu 11 D 17, 00120 Helsinki, FINLAND. Tel: +358 9 674 001 Email: sales@dinosoft.fi
Dittrich & Partner Consulting GmbH	Axel Holzmueller	Kieler Strasse 17, D-42697 Solingen, Germany. Tel: +49 212-260 660 Fax: +49 212-260 6666; Email: info@dpc.de
Dyalog Ltd.	Morten Kromberg	Grove House, Lutyens Close, Chineham Court, Basingstoke, Hampshire RG24 8AG, UK. Tel: 01256-358461 Fax: 01256-316559 Email: sales@dyalog.com; support@dyalog.com
Dynamic Logistics Systems GmbH	Michael Baas	Wilhelm-Schöffer-Str. 29, 63571 Gelnhausen Germany. Tel: +49 6051 13067 Fax: +49 6051 16142. Email: info@dls-planning.com
DynArray Corporation	Dr James Brown	16360 Monterey Rd. Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408)-782-6648 Fax: +1 (408)-782-6627 Email: info@DynArray.com
Evestic AB	Olle Evero	Berteliusvagen 12A, S-146 38 Tullinge, Sweden. Tel & Fax: +46 778 4410 Email: olle.evero@mail.com
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi

First Derivatives plc	Michael O'Neill	First Derivatives House, Kilmorey Business Park, Kilmorey Street, Newry, Co. Down, N. Ireland BT34 2DH. Tel: 028 3025 2242 Fax 028 3025 2060 Email:enquiries@firstderivatives.com
First Derivative Analytics Ltd.	Ken Chakahwata	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-339620. Email: KenChakahwata@compuserve.com
General Software Ltd	M.E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 832463 E-mail: martin@gsoft.plus.com
HMW Computing	Chris Hogan	Hamilton House, 1 Temple Avenue, London EC4Y 0HA, UK. Tel: 0870-1010-469; Email:HMW@4extra.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salisbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028. Fax: 01483-837324 Email: Bob.Hoekstra@HoekstraSystems.ltd.uk
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. Email: acam@blueyonder.co.uk
IBM APL Products	Nancy Wheeler	APL Products, IBM Silicon Valley Lab, Dept H36/F40, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 [+1 (408) 463-2752] Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com
INFOSTROY	Alexey Miroshnikov	11 Bolshaya Monetnaya Str., suite 6, St. Petersburg 197101 Russia. Tel:+7 812 325-9797 Fax: +7 812 233-9685 Email:aim@infostroy.ru
Insight Systems ApS	Helene Boesen	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel:+45 70 26 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
Japan APL Assoc	Julchiro Takeuchi	Kelo Univ. Faculty of Sci.&Tech. 3-14-1 Hiyoshi, Kohoku-ku Yokohama Kanagawa, Japan 223-8522. Tel:+81(045)563-1411 Fax:+81(045)566-1617 E-mail:takeuchi@ae.kelo.ac.jp
JSoftware Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@jsoftware.com
KJK-tieto Oy	Kimmo Kekäläinen	Merikasarminkatu 10 B 56,00160 Helsinki, Finland. Tel: +358 50 55 27 207; Email: kimmo.kekalainen@kjk-tieto.fi
Kx Systems	Simon Garland [Europe]	555 Bryant St., No. 375, Palo Alto CA 94301 USA. Tel: +1 866 kdb fast email: info@kx.com [in Europe simon@kx.com]
Lambent Technology Ltd	Stephen Taylor	81 South Hill Park, London NW3 2SS, UK. Tel: +44(0)20 7813 3786. Email: stj@lambenttechnology.com
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 8JY, UK. Tel: 01992-633807 Fax: 0121-359 0375 Email: phil.last@net.nfl.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Walter G. Fil	203 N. LaSalle Street, Suite 2100, Chicago, Illinois 60601, USA. Tel:+1 (800) 546 4621 E-mail: lingo-allegro@visto.com
Lucas Solutions	Jim Lucas	Stubbedamsvej 9C, 3.tv., 3000 Helsingør, Denmark Tel: +45 49 26 52 42. Email: jel@danbbs.dk
Alastair Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0) 7802 430202 Email: alastair.kinloch@btinternet.com
MicroAPL Ltd.	Richard Nabavi	The Roller Mill, Mill Lane, Uckfield, E.Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Milinta Inc.	Dan Baronet	Contact Dan Baronet at dbaronet@milinta.com
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263843 Email: apl@ellismorgan.co.uk
Dr. Nussbaum gift mbH	Dr Reiner Nussbaum	Buchenerstrasse 78, D-68259 Mannheim, Germany Tel: +49 621 7152190, Email: reiner@nussbaum-gift.de
Optima Systems Ltd	Paul Grosvenor	Optima House, Mill Court, Spindle Way, Crawley, West Sussex, RH10 1TT, UK. Tel: 01293 562700 Fax: 01293 562699 Email:paul@optima-systems.co.uk
Orbien Consulting Inc.	Ros Fovargue	33 Cherryhill Ave., Toronto, Ontario, Canada, M1C 1W9. email: rfovarg@interlog.com or Ros.Fovargue@sympatico.ca
Qualedi Inc.	Nicole Schless Georges Bringham	121 West Main Street, Milford, CT 06460, USA. Tel: +1 (203) 874-4334 Fax: +1 (203) 876-9083. Email: sales@qualedi.com; info@qualedi.com
Renaissance Data Sys	Ed Shaw	P.O. Box 511, Botsford, CT 06404, USA. Tel: +1 (203) 270-9729 Email: aplbooks@earthlink.com
Right Seat Software, Inc.	Tom Atkins	1110 12th Street, Golden, CO 80401, USA. Tel:+1 303 278 2244. Fax: +1 303 278 6957. Email: info@voxproxy.com

Graeme Robertson	Dr. Graeme D. Robertson	15 Little Basing, Basingstoke, Hants RG24 8AX, UK. Tel: +44 (0) 1256-364071 Email: GraemeDR@nildram.co.uk
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00149-Roma Trullo, Italy Email: mario.sacco@tin.it
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com
Skelton Consulting GmbH	Charles Skelton	Weinlinge 9, 70329 Stuttgart, Germany. Email: c.skelton@skelton.de
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Email: bernecky@acm.org
Soliton Associates		Soliton Inc, 44 Victoria Street, Suite 2100, Toronto, ON M5C 1Y2 Tel: +1 (416) 364 9355 Email: sales@soliton.com
SovAPL Russian Chapter of SIGAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region 249020, Russia Tel: +7(08439)47109 Fax: +1 (530) 6885510 Email: askom@obninsk.com
Strand Software Inc		P.O. Box 330, Excelsior, MN 55331 USA Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@strandsoft.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rex@rexswain.com
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1-818-347-5779 Email: roysykes@earthlink.net
Toronto SIG	Richard Procter	P.O.Box 39, Mansfield, Ontario, L0N 1M0, Canada, email: info@torontoapl.ca
ubJL GmbH (APL Software Team)	Kai Jäger	Thunseldastr. 22, 90482 Nürnberg, Germany. Tel + 49 911 482512; Fax: +49 911 482518 Email: jaeger@ubjl.de
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP SITES

IBM APL2 [ftp.software.ibm.com/ps/products/apl2](ftp://ftp.software.ibm.com/ps/products/apl2)

WORLD WIDE WEB SITES

Acadvent Ltd.	www.acadvent.com/
ACM SigAPL	www.acm.org/sigapl/
ADVOCORP Oy	www.rett.fi/
AFAPL	www.afapl.asso.fr/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.apl385.com
APL Forum	www.aplforum.com
APL Germany e.V.	www.apl-germany.de
APL Italiana S.p.A.	www.apl.it/
APL Journal, Germany	www.rhombos.de/shop/a/show/article/?216
APL Borealis Inc.	www.aplborealis.com
APL Consultants Japan	http://aplcons.com
APL to ASCII	see Waterloo Archive
Assured Systems	www.assuredsystems.com.au/
AUSCAN	http://home.ca.inter.net/~rjp/auscan/
Eke van Batenburg	www.bio.LeidenUniv.nl/~Batenburg/index.html
Carlisle Group	http://carlislegroup.com/
Causeway	www.causeway.co.uk/
CODEWORK	www.codework-it.com/tangram/eng/
CoSy (Bob Armstrong)	CoSy.com/
Dinosoft Oy	www.dinosoft.fi/
Dittich & Partner	www.dpc.de ; www.apl-online.de

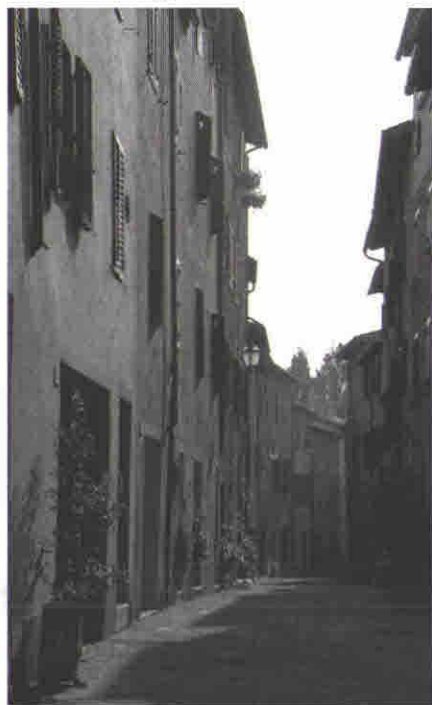
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/
Dyalog Ltd	www.dyalog.com/
DynArray	www.dynarray.com/
Dynamic Logistics Systems GmbH	www.dls-planung.de www.dls-planning.com
FinnAPL	www.pyr.fi/apl/
First Derivatives plc	www.firstderivatives.com
Hoekstra Systems	www.HoekstraSystems.ltd.uk/
IBM APL2	www.ibm.com/software/awdtools/apl/
Infostroy	www.infostroy.ru
Insight Systems ApS	www.insight.dk/
Japan APL Association	www.ae.keio.ac.jp/lab/soc/takeuchi/japla/
JSoftware Inc	www.jsoftware.com/
KJK-tieto Oy	www.kjk-tieto.com
kx systems	www.kx.com
Lambent Technology	www.lambenttechnology.com
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo-allegro.com/
MicroAPL Ltd	www.microapl.co.uk/apl
Milinta Inc	www.milinta.com
Nussbaum gift	www.nussbaum-gift.de
Optima Systems Ltd	www.optima-systems.co.uk
Qualedi, Inc.	www.qualedi.com
Renaissance Data	www.aplbooks.com/
Right Seat Software, Inc.	www.voxproxy.com
SigAPL	www.acm.org/sigapl/
Skelton Consulting GmbH	www.skelton.de
Snake Island Research Inc.	www.snakeisland.com
Soliton	www.soliton.com/
Strand Software Inc.	www.strandsoft.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.ca/
ubJL GmbH (APL Software Team)	www.aplteam.de
Waterloo Archive	www.math.uwaterloo.ca/apl_archives/Welcome.html
Jim Weigang	www.chilton.com/~jimw/

RECENT MEETINGS

The San Quirico Moot 4-10 June 2007

reported by Adrian Smith (adrian@apl385.com)

Acknowledgements



Just a big thank-you to Paul Mansour for setting this up and inviting us out to experience Siena and San Quirico d'Orcia. The atmosphere of the old Tuscan hill-top town was fantastic (we hit festival time), the culture was pretty impressive (Jet2 fly Leeds to Pisa, so guess how Richard & I travelled), and the food was really wonderful.

The great advantage of the 'moot' format is that everyone turns up with lots of interesting things to talk about, but there is *no programme* published ahead of time.

The first morning was spent (after clearing away the breakfast and moving all the tables around) in deciding what was most interesting, and haggling over the order of play. This had the huge benefit that the later talks could be adjusted to respond to earlier ideas – Richard spent most of the second evening adding a couple of John Scholes'

language ideas to Rowan, so we could actually play with them 2 days later.

The layout 'within the day' was also a little radical and worked brilliantly. We went for a fairly early breakfast, cleared the tables and worked straight through (with regular breathers) to a deadline of a 3:00pm lunch. The restaurant was down 2 back lanes and around enough corners it felt like you had walked quite a way, and the lunch was so good we were probably still there at 4:30. Then it was 'time

off' in the garden until the cool of the evening and our choice of evening meal. I think nearly all the time was well-spent, although my notes are vague to non-existent on the afternoon discussions.

Clearing the Air on Unicode – Morten Kromberg

This was a very useful working introduction to Unicode, and an opportunity for Morten to air some of the knotty issues that Dyalog were beginning to trip over. For example the implications of having `⎕DR` report something other than 82 for character vectors was kicked about until we all understood the reasoning. Silly questions like "Which way to write Hebrew?" were debated, and the general view that these are *character vectors* in APL was felt to be defensible, so the answer is (phew) from left-to-right. What about allowing a lot more characters in assigned names? The Unicode standard does suggest a reasonable subset here, so we could research these, but again the present collection of characters (Dyalog is quite permissive already – `Nestlé+123`) was thought to be good enough. Cyrillic and Greek are possible future extensions.

There were a couple of mapping issues at the ragged edge, for example the APL quote (I can't even show you one here) has no Unicode equivalent, but everyone in the room had long abandoned Ctrl+K in favour of the plain ASCII quote, so hopefully it is a non-issue. The underlined alphabet is a much knottier problem – there is no such thing in the Unicode standard! There is, however, a *circled* alphabet so the meeting wanted to explore the possibility of adding these glyphs into the APL385 font, and Adrian was sent off to the garden to play with FontLab. After some time, it became clear that trying to put an A into a circle and preserve the fixed-width of the character resulted in an unreadable and rather ugly blob. We agreed to leave this one aside until Minnowbrook (see later) where we knew Lee Dickey was coming, and Lee sat on the Unicode committees. When in doubt, consult the Oracle!

Project Jasper – Stefano Lanzavecchia

"Suddenly, dynamic languages are cool!" was the theme of this rattle through something called Jasper which is an attempt to wrap a database as a set of classes that are built for you on the fly. Essentially, it parses the entity-relationship diagram (implicit in your DataSet design) and creates classes on the fly to let you go digging around it, with all the stored procedures showing up as methods.

Hard to say how well the idea washed with the serious database techies present – there is a conflict at the core of the mapping process between the OO view of life (a hierarchy with inheritance) and the Relational view (a network with no fixed

links). However, it set me running with some ideas that rapidly crystallised into a way to wrap Dyalog classes around Selections, which went through some very rapid prototyping and is written-up in full a few pages further on.

Hyper-operators – John Scholes

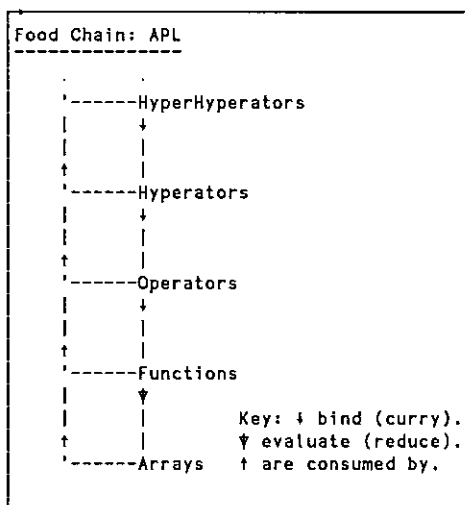
“Higher level operators in APL have been considered and rejected many times in the past ...” *Pure Functions in APL and J* – Ed Cherlin, APL 1991

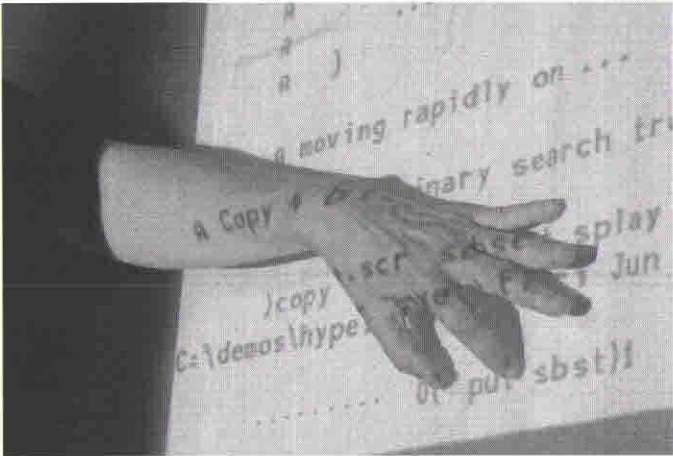
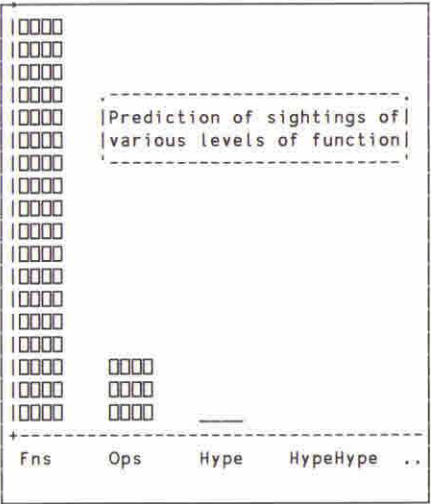
As you all know, the world is supported by four enormous elephants, which stand on the back of a great Turtle. As for what the Turtle stands on – what a dumb question – “it’s Turtles all the way down ...” of course.

John presented this material with greater than usual diffidence, clearly hoping that we would shoot it down in flames. He had even taken the trouble to paint some handy targets on it, and made sure to point them out!

John’s Script, verbatim apart from formatting up the HTML a bit

If we proceed with caution we have a little freedom: Paul’s Moot → Minnowbrook → Dyalog Conference → ??? Why would I need something higher than an operator? Next in progression: function → operator → ... Operators deal in functions, what deals in operators?





Illumination on hand

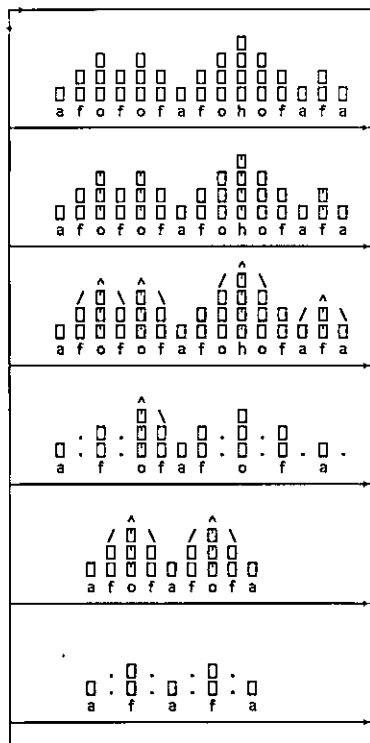
Hyperators should bind left, like operators.

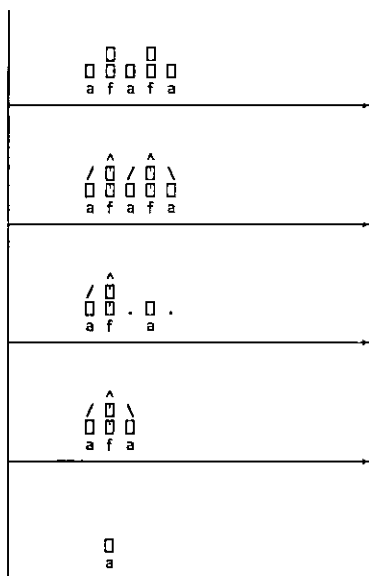
```

A let:
A
A 1 2 3 be arrays,
A + * ÷ be functions,
A ° . be operators,
A X Y be hyperators.
A then:
A
A 1 × 2 + 3 → 1 × (2 + 3) A fn-array binding
A + . × ° ÷ → (+ . ×) ° ÷ A op-fn binding
A ° X . Y ° → (° X .) Y ° A hype-op binding

```

An item of class ω (>2) binds with its neighbour(s) of class $< \omega$ to derive an item of class $\omega-1$.





```

R 'hello' ° , 'world'
R <--2--> 4 3 <--2-->
R '-----'
R ('hello' ° ,) 'world'
R <-----3-----> <--2-->
R '-----2-----'

```

The last step, the binding of a derived class-3 item with its neighbouring class-2 object(s) commissions an evaluation of the derived (class-3) function, to which all items in the binding hierarchy are exposed. For example, a dyadic function derived from a dyadic operator:

```

R ∇ z + l (f op g) r
R [1] ...
R has access to left and right arguments and left and right operands.
R "Header" syntax:
R [0] z + a d g A d is a dyadic function.
R [0] z + a (b d f) g A .. .. operator.
R [0] z + a (b (c d e) f) g A .. .. hyperator.
R Dfns:
R {
R ααα A enclosing braces bound a monadic hyperator
R ωωω A .. .. dyadic hyperator
R

```

```

R      α ω      R left and right arguments.
R      αα ωω     R left and right operands.
R      ααα ωωω   R left and right hyperands.
R      ∇∇∇      R hyperator self-ref.
R      ∇∇       R derived operator:      (ααα ∇∇∇ ωωω)
R      ∇        R derived function:  (αα (ααα ∇∇∇ ωωω) ωω)
R      }

```

Examples

```

      nwise
{      A N-wise reduction J.Brickman.
      d+ps+pw
      αα/(,ω)[((-φdα-1)↓sp1×/s)°.+(1α)-□io]
}
      1 + nwise 18
1 2 3 4 5 6 7 8
      2 + nwise 18
3 5 7 9 11 13 15
      3 + nwise 18
6 9 12 15 18 21

```

Let's change / to \ and see what happens ...

```

      □ed'nwise'
      1 + nwise 18
1
2
3
4
5
6
7
8

      2 + nwise 18
1 3
2 5
3 7
4 9
5 11
6 13
7 15

      3 + nwise 18
1 3 6
2 5 9
3 7 12
4 9 15
5 11 18
6 13 21

```



```
2003529930406846464979072351560255750447825475569751419265016973710894059556
31145308950613
```

```
0880933348101038234342907263181822949382118812668869506364761547029165041871
916351587
```

```
...
775300413847171245779650481758563950728953375397558220877750607233944558789
5905719156736
```

```
)copy dfns rats factors
C:\dfns\dfns saved Tue May 20 00:29:42 2008
r34 + 3 rats 4          A rational 3r4
r45 + 4 rats 5          A ..      4r5
fmt←r rats              A r-format
fmt r34 +rats r45       A 3r4 + 4r5
31r20
fmt r34 xrats r45       A 3r4 x 4r5
3r5
fmt r34 ÷rats r45       A 3r4 ÷ 4r5
15r16
```

nats and rats can cope with only *unsigned* arithmetic. WIBNI they could *share* a mechanism to do signed arithmetic:

```
A 123 - nats signed 456      A signed nats
A r34 - rats signed r45     A signed rats
A ( a aa aaa vvv w )
```

Hyperator "signed" might look like this:

```
A signed + {
A   signs+ '-' => * "α ω          A remember signs.
A   lft rgt+signs+ "α ω          A unsigned arguments.
A   fn + αα{fn+αα ◊ => cr'fn'}ω  A fn operand.
A   '+'=fn: ... lft αα aaa rgt   A unsigned arith.
A   '-'=fn: ...
A   ...
A }
```

Moving rapidly on ... copy 4 binary search tree (BST) operators ...

```
'fmt'sbst (0('put'sbst)1)('put'sbst) 2          A simple BST
insert
i=1.
'2=2
)copy dfns foldl
C:\dfns\dfns saved Tue May 20 00:29:42 2008
```

```

      'fmt'sbst 0 ('put'sbst)foldl 1 2
insert
1=1.
  '2=2
    'fmt'sbst 0 ('put'sbst )foldl 15
insert
1=1.
  '2=2.
    '3=3.
      '4=4.
        '5=5
          'fmt'avl 0 ('put'avl )foldl 15
insert
1=1
2=2|
  |
  |.3=3
  |>4=4|
    '5=5
      'fmt'redblack 0 ('put'redblack)foldl 15
insert
1=1
2=2|
  |
  |.3=3
  |>4=4|
    '5=5
      'fmt'splay 0 ('put'splay )foldl 15
insert
1=1.
  '2=2.
    '3=3.
      '4=4.
        '5=5

```

A simple BST

A simple BST

A avl tree insert

A redblack insert

A splay tree

WIBNI [1] we could pass the tree operator as a parameter:

```

      try + {'fmt'ααα 0 ('put'ααα)foldl ω}
insert.
  A      avl try 15
  A      redblack try 15
  A and so on ...

```

A ααα tree

That's all Folks.

From Content to Web Layout – Stephen Taylor

“This is worth a lot more, because there is a lot less of it”

Generating a good web-site from a collection of text files is harder than it looks. What Stephen showed us was a very minimal bunch of Dfns which used all sorts of simple conventions in the file-names and the ability to embed simple APL code anywhere in the source material (such as `Copyright 2006 - {tTTS}`) to make the construction process simple and easy to adapt. One day it may stabilise to the point where he can write it up for us!

Wrap Up



Yes, the Dyalog Duck has its origins in the 13th century!

Seriously, this was a good workout for everyone who came; we went away a good deal more bronzed and only a little more pickled in the local red wine. Morten made us all feel very unfit by cycling up most of the nearby mountains, while Richard & I earned several nutcase points by walking to the next town for the evening meal.

Several issues got explored with a lot more freedom than is possible in a formal conference setting, and I think the future of APL is just a little more secure as a result. Let's do it again some time!

Notes

- [1] WIBNI: Wouldn't it be nice if ...

Minnowbrook 2007

reported by Adrian Smith (adrian@apl385.com)

Acknowledgements

Minnowbrook was one of the great institutions of the APL world, which I never managed to get to. The idea was to bring together anyone who could lay claim to the term *implementor*, put them under a comfortable roof well away from the office, and get them to co-operate on new features and new ideas. Probably that is why almost everyone has `FTIE` and `FMT` and they basically behave in similar ways.



Blue Mountain Lake

The Minnowbrook Conference Center has long been an outpost of Syracuse University, and is set in a lovingly-rebuilt hunting lodge on Blue Mountain Lake in the Adirondacks. In the autumn, it is just a wonderful place to hang out, listen to the loons on the water and watch the maples turning gold before your very eyes. The WiFi network was accessible well out on the water, so it was possible to check your email while (if you were being sensible) someone else was paddling your canoe. Such places are always conducive to good thinking, and we have Roy Sykes to thank for reviving the Minnowbrook tradition with a speculative email early in the year, to which some twenty APL implementors replied in the affirmative.

Here are a few of them unwinding after a heavy morning session in the classroom:



Unwinding between sessions

What did we all learn?

It is hard to summarize the experience, as I am sure everyone came away with something different. For me, some of the highlights were:

- Joe Blaze (in the round table on marketing APL) admitting to some outstanding sales figures for their 'lightweight array engine' that forms the basis for APLNext. This is being sold at \$50 per year through software clubs and forums, mostly in India. It made me think that we APL 'insiders' are probably aiming far too high and that a mass-appeal product needs to be much lighter and easier to get your brain around than any current APL.
- The total lack of supporting material on the Internet for anyone trying to start in APL. Maybe this is why you can succeed with a simple array engine, but not with a full APL environment?
- Sven Bodo-Schultz on his Single-Assignment C (SAC) project. See the SAC-home on the Internet for more background! This is a high-performance C compiler which is aimed specifically at exploiting parallel (multi-core) processors. Bob Bernecky has been targetting it with his APL->C translation tool for some while now, and is getting very promising results. From an APLer's point of view, single-assignment dodges all the nasty problems with re-use of variables with different types. Every assignment gets a fresh name, and an immutable data-type. Problem solved!

- Dave Liebtage on Unicode. Dave tried *very hard* not to look superior as he pointed out that IBM has been fully Unicode for over 2 years now. They set the standard for the APL symbols in the first place, and they have also led the way in mapping APL functions to Unicode glyphs, where there is ambiguity. Fortunately, everyone else is now firmly in line astern. It was also agreed (thanks to Lee Dickey for being authoritative on this) that it was perfectly OK for APL to map out to the 'circled alphabet' with glyphs drawn as ABCDE...Z which makes a lot of pragmatic sense. Whether we should continue to use this alphabet in variable names is another matter, but it will take time to shake it out completely.
- Paul Mansour on FlipDB. This was a basic primer session on how to use his database, and a lot of the ideas made a great deal of common sense. If you want to know where he is coming from conceptually, I strongly recommend *Database in Depth* by C.J. Date (pub. O'Reilly) which I don't always agree with but, like APL, it takes a solid theoretical model and sticks to it. Being able to navigate foreign-key trails with simple expressions like `SP[SNO].QTY` is a huge notational improvement on the traditional 'join' syntax, and adds power as well as clarity. Just as APL makes many programming challenges 'trivial' so FlipDB takes several of the 'hard' problems that often result in pages of dense SQL, and expresses them in one simple APL-like expression.
- Objects in Rowan were sketched out in the sessions (and no doubt on the canoe) and Richard implemented enough over the next few days in the cabin to have a working interpreter before we left the area. I'll let him make sense of his notes in time for the next Vector.



The WiFi comfortably reached the shoreline, which made for some very cosy programming spots! Roy's two dogs (notice the conference badge with her name on it) did a great job of shepherding us back indoors when the sessions were due to restart.

Other highlights were harder to pin down – listening to John Scholes and Bob Bernecky arguing strategies for memory management late into the night in the bar was one I remember, and of course mastering the 'Minnowbrook Shuffle' was high up the list for all of the delegates. For the

uninitiated, *Shuffleboard* is a bit like table-curling. You skim a series of pucks down a sanded board, scoring for those which lie nearest the far end in a series of numbered beds. It got quite competitive after a few beers, with some very closely (and rather noisily) contested games.



You can see that the blue team has one just short of a high-scoring line, while the red team has been a little too conservative!

There was also a very good pool table, and lots of easy hill-walking straight out of the back door.

I fear we missed out on the float-plane ride in favour of walking up Castle Rock, but it did spot us and give us a good wave with its wings as it went over!



**Adrian, Gill & Richard Smith with Dan Baronet and Morten Kromberg
on Castle Rock, above Blue Mountain Lake.**

Wrap-up

This felt to me like the kind of thing we should try to do every few years. The initial objective of synchronising APL developments is still extremely valid, as we all begin to make steps towards Classes and Objects, as well as interfacing to other languages (Java from APL2 and .Net from Dyalog and APLX). If we do move towards shipping some more lightweight array technology, agreement between the various APL providers will be very helpful in capturing the attention of the world's programming community. The location was as good as it could be, so let's hope to see it again before too long.

DISCOVER

Alive and Well

by Morten Kromberg (mkrom@dyalog.com)

Editor's Note

I have reproduced this, originally posted at *comp.lang.apl*, in part to mark the end of a certain conversation. Many heads now gray or simply shiny believed when younger that one day all programming would be done in APL or something recognisably like it, and that software development would resemble what we do. Nothing like that happened.

After years 'in the wilderness' there are signs Iversonian languages are once again attracting wider interest. There is nothing in this to wonder at. Programmers are not markedly less susceptible to fashion's tide than other humans. There is a question as to how APL once lost respect, but it is not a very interesting conversation. It is a verbal wake, welcome neither in the pages of *Vector*, nor at meetings of the British APL Association.

But a closely-related question is interesting. In what current roles are the APLs likely to enjoy respect? How should we describe their virtues, and to whom? *Vector* welcomes this conversation. Dyalog Ltd is a Sustaining Member of *Vector* and one of the leading APL vendors today. Morten Kromberg thought long and hard about the future of the APLs before investing in the company and becoming its Chief Technical Officer. His post at *comp.lang.apl* kicks off a conversation *Vector* is happy to host.

Stephen Taylor

It *does* seem to be true that:

1. Some APL vendors never successfully made the transition from the mainframe to the PC.
2. Some APL programmers with grey or white hair only see others with the same hair colour, and do not know any of the many young APL programmers who joined the community recently.

3. Unlike in 1980, it is very hard to find a consulting job where all you need to know is how to write APL programs.
4. It is almost impossible to interest Computer Science departments in APL.
5. If you work in a traditional software department you are unlikely to be allowed to start a new project in APL. Software engineers still don't think APL looks like anything they learned at school and that it is 'broken'. The fact that they can't deliver software on time, on budget and to spec is only just starting to shake their faith that they are doing the right thing.

Over the past few weeks, we have seen a number of people post messages at `comp.lang.apl`, expressing the belief that APL is dying, due to various combinations of the above. Fortunately, it is simply not the case: APL and other array languages are well established in certain niches. The value of solutions based on these technologies is measured in billions of dollars per year, and their existence is *not* threatened.

It is a hard time to be an APL programmer. I think it is fair to say that most of us once chose APL because it offers a very high level of abstraction, allowing us to very quickly turn ideas involving data manipulation into executable code. But today, the platforms upon which we are expected to deliver software are changing rapidly, and we are expected to take interest in the very issues that we picked APL to avoid getting distracted by. In particular, user interfaces and databases now require extensive layers of toolkits, and these layers are shifting under our feet. XAML, Silverlight, Virtual Machines of one description or another, ODBC, JDBC, ADO, ADO.NET. We are expected to embed APL software in browsers which are not allowed to use local storage, even though this is an obviously bad idea for software which analyzes huge amounts of data. Remember CICS? It's the same story all over again. Traditional web development techniques are another example of mainstream technology trying to beat APL into a mould created for lightweight transactional systems. Keep your data in an SQL database, look at one record at a time...

Architectures get more and more complex. If you are a computer scientist or software engineer, rewriting the software every few years in order to increase the beauty of the code is what you love – why you chose the job. If you are an APLer, chances are you find all this depressing. Fortunately, while the higher layers of tools are still in a state of flux, the lowest levels of the new infrastructures are now stabilizing. We should soon see tools and interfaces from vendors that allow APLers to integrate solutions with popular platforms without all of us having to go out and learn the details of the latest advances.

The big APL vendors of old, staffed by people who loved APL and often had the same problem-oriented (rather than technology-oriented) attitudes, were slow to realize the impact of the technology shift. Consulting, timesharing and licensing revenues dropped sharply as the PC took over from the mainframe. Almost no marketing has been done since the late 1980s, and no new educational materials have been produced – with the notable exception of those written for J. The traditional vendors were caught in a negative feedback loop. Several did not pull out.

Accelerating the decline was the fact that much of the early revenue was a function of a huge but fundamentally temporary business in the 1970s and 1980s, where all large companies were building their own software for just about every purpose. APL was one of the best tools on the market for this pioneering work. However, the production of – and competition in the market for – the ‘shrink-wrapped’ software which replaced most of the APL systems written in this period is a different game entirely: key factors are a cheap and easily recruited/fired workforce, documentation, procedures, and marketing. I think that most APLers that I know would not want to work in these jobs, which the industry is trying to turn into outsourceable ‘unskilled’ jobs, even if APL were an acceptable technology to these businesses.

We can argue about whether this or that system which was converted would have been cheaper to maintain and more useful if the technology had remained APL. I have personally witnessed many extraordinarily bad business decisions to replace APL systems, made by management teams with absolutely no understanding of what they were doing (some of whom are now back in the fold as customers, I am happy to report). But in many situations, the business logic which caused managers to replace the use of APL by more ‘predictable’ workforces and technologies are ‘sound business decisions’. It is in the nature of APL to be used as an executable specification language: APL is often used as much to discover the specification as to implement the system. Having an executable spec makes ‘agile’ interaction with the user much easier than with many other technologies.

In many businesses (the financial industry being the prime example), the specifications never quite settle down. Fortunately, APL interpreters are highly efficient and capable platforms, which support the use of APL in high-volume production applications. Although APL is dynamic and interpreted, APL-coded solutions in this market often perform better than traditional solutions. I believe that this is because the layered approach of traditional software development tends to drive the development in the direction of re-use of overly general components, which need to be bent a bit too much and therefore perform badly.

Too many layers in an object hierarchy lead to inefficient code, even after compilation.

So yes: there *are* many large APL (based) businesses which have died – but the family of APL languages will not die unless they are one day replaced by something *more capable* in the areas where APL is strong. This is quite simply because it provides unique value in a number of situations. The market that these situations represent is rapidly growing, and very poorly served by other technologies, so there is significant room for growth for APL, J, K and other new dynamic and array-oriented software development methodologies. The mainstream software industry is only just starting to shift its focus from administrative systems to analytical software, and we have a 40-year head start. If you look at trends in modern programming languages, they are heading in *our* direction: dynamic languages, inference in languages like C#, growing use of ‘Reflection’ (inspecting types and doing different things depending on what was passed to you), all of these are signs that mainstream technology is waking up to the fact that if you want to write code quickly, and especially if it needs to do anything analytical, you need the kind of flexibility that APL already has.

This would be a *very bad time* to start moving the language towards where everyone else is now!

I admit that:

1. The golden age of cushy (mainframe) consulting jobs will not return.
2. Trying to sell APL in head-to-head competition with Java and C# for traditional ‘software construction’ projects is almost impossible.
3. APL is pretty useless in a Computer Science department. It is not the goal of these institutions to teach the art of extracting specifications or the construction of working software so APL has little to offer them (OK, I admit have a chip on my shoulder: I dropped out of CS pretty early on after discovering that only 15-20% of the marks were given for arriving at a working solution).

However, unless you specifically want to achieve one of the above three goals, there is no reason to despair. The market for turning complex ideas into working software is growing rapidly, and all the brilliant ideas that Ken Iverson and the early APL developers put into the language are more relevant than ever before.

Today, the successful (growing) users of APL seem to be entrepreneurs (or entrepreneurial businesses) with novel ideas that they want to bring to the market ahead of the competition, or before conditions change. If you want to be an ‘APL consultant’ (internal or external), you must learn how to wrap APL as components

to be embedded in – or communicate with – mainstream components. You must learn about SQL and XML and other emerging technologies, and how to help the domain experts who have the ideas and the skill to write APL for the application core.

In rare cases, one person can do all of the above (for a while), but generally we need to build teams combining domain- and technical experts. The vendors must do what they can to make the jobs of both these groups easier, and also explain the mechanics to managers who need to understand how to make this work.

At Dyalog, we firmly believe that we can safely ignore the decline of the APL market which followed the unnatural blip in the 70s and 80s, even though some old APL systems are still in their last throes. It is a false signal with respect to the fundamental value of the technology. The truth is that APL is actually still ahead of its time. It is our good fortune that we never had a mainframe business, as it means that for us, APL has been always been growing – and that growth now seems to be accelerating. We believe that we can further accelerate the growth through:

1. Working hard to improve integration with platforms (Microsoft.Net, Mono, perhaps Java). We've been working hard for the last several years, with the addition of the .Net interface, Object Orientation and finally Unicode support in V12.0, we feel we have made big steps in the right direction. Although it is one of the fundamental values of our product that people who are not software engineers have a complete package which allows them to build all the components of an application, we must also support large organizations who want to build multi-layered, and 'multi-tiered' applications using a combination of teams skilled in APL and other technologies.
2. Producing new books, training materials, 'code patterns' – and provide new tools for sharing code between APL users. Expect to see the first results of this work to appear during 2008.
3. Cultivating contacts in the dynamic/agile programming communities, where we believe we will soon see the first openings for marketing APL directly to development teams (as opposed to going via the users first).

We don't think it will be long before we'll be back in the business of cold-calling certain types of development teams to sell APL as a modern technology. We are already working to start new academic programmes using APL. It is too early to say when we will succeed with this, but we are optimistic.

Review: APLX version 4 – from the viewpoint of an experimental physicist

by Dr Beau Webber, Lab-Tools Ltd (beau@lab-tools.co.uk)

Which APL? Why APLX?

When I first read about APL in *Byte* magazine so many years ago (August 1980?), I realised it was *the* language for a working physicist, given its ability to apply complex manipulations to structures of data just by typing a few symbols. But in those days, having written my program in APL, I had to hand-translate it into Basic to run it on a PDP-11. Then came APL\11 on the VAX-750, as part of Unix – I upgraded our local copy, adding such features as Pipe[1]. The advent of APL68000, on the Atari ST, from MicroAPL, meant I could have a copy of APL running on the lab bench – as well as at home – and I applied it to everything from analysing Nuclear Magnetic Resonance (NMR) diffusion data to controlling the lights and sound for our village dramatics society.

I have used other versions of APL over the years, and have helped with the development of Tim Budd's APLc[2] APL-to-C compiler, but have found that as the APL offerings from MicroAPL have developed over the years, this has been my favoured APL interpreter – and following that early use for sound and lighting control, APLX on PC has become my main software tool for control of my physics experiments.

National Instruments' LabVIEW[3] is a very powerful tool for instrument design – what is it about MicroAPL's APLX, version 4 that now makes it, for me, the preferred design platform?

There are a number of answers to this, but above all, as I will demonstrate with particular examples from on-going projects, there is the ability to have a seamless join in a loop that goes from a screen-based front panel with buttons and drop-down menus, interacting with a number of asynchronously operating independent APL processes, some communicating over USB to firmware operating parallel hardware (at 100s of MHz) in gate-arrays that control the experiment and collect the data, which is then passed back over the USB to the APLX programs for analysis and adjustment of the instrumental parameters.

In addition, rather than

- translating my APL source code (following writing and testing) to LabVIEW VIs (visual/graphical programs), as I did in the 1990s; or
- translating to Basic and Fortran, as I did in the 1980s; or
- using APLc to translate to C

I can now apply them immediately to the experiment in hand, modifying them as needed to suit the data emerging from an experiment. Front-panels can be re-configured on the fly, to suit the needs of 'soft-instruments'.

New Features in APLX Version 4

Version 4 of APLX – currently 4.1.0 / 7.1 – is a major upgrade from the previous version 3.09. The document that comes with it has 108 pages summarising the changes, with examples, under the following main headings:

- Object-Oriented Programming in APLX v4
- Creating, editing, and modifying APL classes
- Using Internal (User-Defined) Classes
- Using External (.Net, Java, Ruby) Classes
- User-interface programming in .Net
- Using System Classes
- New System Functions for object-oriented programming
- System Method
- Other New System Functions
- Enhanced System Functions
- New System Commands
- Zilde
- New Structured-Control Error handling
- Enhancements to System Classes (Quad WT)
- Entering APL symbols using QuickSym
- User-interface enhancements

With so many upgrades in this new version, this must at best be a preliminary report, concentrating on those features that have been particularly relevant to the code I have been working on since this version was released just before last Christmas.

Object-Oriented Programming

As can be seen, the biggest single development in this upgrade is to address a long-standing complaint against APL, in that it is said not to fit well with modern Object-Oriented programming (OO) – at one stroke MicroAPL addresses this by adding OO syntax options to calls such as those for window creation and manipulation, as well as adding easy access to any OO programs installed on the same machine.

Using External (.Net, Java, Ruby) Classes

Accessing the features of, say, .Net or Java is now vastly simplified:

.Net: Create an instance of the .Net DateTime class, defined in the .Net class libraries, but not specifying an initial date to the constructor of that class:

```
NETDATE ← '.net' ⌵NEW 'System.Date.Time'
NETDATE.ToString
0001-01-01 00:00:00
```

Here we see the zero date and time returned in ISO format, which is the preferred format on my machine.

Java:

```
'java' ⌵CALL 'java.lang.Integer.toBinaryString' 37
100101
```

APLX has extensive built-in graphics ability, but with version 4 one can now also use Windows Forms directly from APLX:

```
▽DOT_Forms;X;F;EditResult;EditIn;ButtonDo
[1] ⍝ Simple example of using Windows Forms from APLX v4
[2] ⍝ Create main form
[3] F←'.net' ⌵NEW 'Form'
[4] F.Text←'Expression Evaluator'
[5] ⍝
[6] ⍝ Create edit box for input line
[7] EditIn←'.net' ⌵NEW 'TextBox'
[8] EditIn.Left←12 ⋄ EditIn.Top←21
[9] EditIn.Width←265 ⋄ EditIn.Height←20
[10] EditIn.Font←'.net' ⌵NEW 'Font'
[11] F.Controls.Add EditIn
```

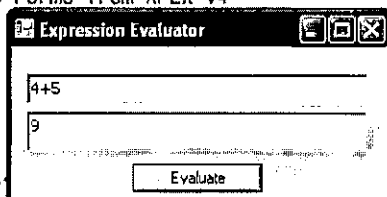


Figure 1. .Net graphics example, showing a section of the code in the update reference manual, and running (a slightly modified version of) the code in the example files.

New system functions for object-oriented programming

APLX Version 4 includes the following new system functions to support object-oriented programming:

□CALL	Call external static method
□CLASS	Class hierarchy for object or class
□CLASSES	References to user-defined and external classes
□EVA	Event arguments
□EVAL	Evaluate external expression
□EVN	Event name
□EVT	Event target
□GETCLASS	Get reference to named class
□IC	Insert into class
□INSTANCES	Instances of a class or descendants
□NEW	Create new instance of class
□NULL	Return reference to null object
□RECLASS	Change class of objects
□REARENT	Change parent of user-defined class
□RECLASS	Set up external environment
□THIS	Reference to current object

As yet I have much work to do before I can fully apply this rich new capability.

Other new system functions

- | | |
|---------------------------------|--|
| ☐ EXPORT | Export APL array to file in specified format |
| ☐ IMPORT | Import data from file in specified format |

APLX has had the ability to exchange data with other software such as Excel for quite a while, but this has been extended with a range of other formats, including HTML.

This exchange can in fact be carried out by a number of mechanisms; see the section on the clipboard, at the end of this report, on enhancements to the user interface.

User-interface enhancements

At this point I will take an item out of the above order, as it is a very significant one, that affects the way I create and manipulate data structures for my instrumentation:

General Array Editor

APLX Version 4 includes a powerful new General Array Editor. Subject to available memory, this allows you to edit almost any simple, mixed or nested APL array, the only limit being that the array cannot exceed five dimensions. You can edit any element and, if you wish, restructure it: for example, converting a numeric scalar element to a three-dimensional nested array. You can also restructure the whole array. The editor also supports editing arrays containing object references. (*APLX V4 Update Manual*)

What I have already found this so very useful for – in addition to just looking at the structures of data by just double-clicking on them – is to create complex data structures defining Nuclear Magnetic Resonance pulse sequences.

Given two nested lists, one defining pulse channel names, the other defining time-delays in terms of variables (that may later change on the fly), one creates an empty pulse-program skeleton, and then uses the General Array Editor to populate the significant transitions in the timing-structure:

```

PP + Channels pp_create PPnDelays;R;C
R
A create a pulse program skeleton
A given a list of Channel names and (Pulse Program Name +
  Delay time names)

```

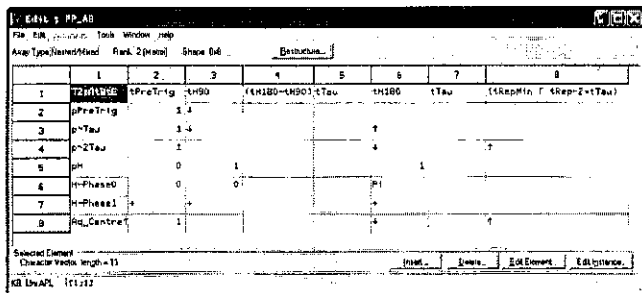


Figure 2. Creating an (90° – – 180° – – Echo) NMR Pulse Sequence with the General Array Editor.

This is then evaluated, the remaining time-slots being filled in using the specified states and transitions in the default values. A facility closely related to the General Array Editor, the Grid tool, is used to display the result for checking, before the final stage of conversion to a binary pattern. This is then up-loaded over the USB to a stack-based sequencer implemented in an Altera gate-array module.

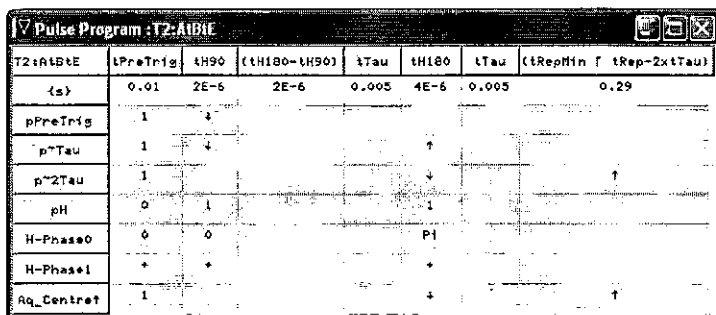


Figure 3. First stage of compiling an (90° – – 180° – – Echo) NMR Pulse Sequence.

The communication with the gate-array module is implemented by accessing the DLL routines that control the USB data transfer directly using APLX calls. This ability to talk directly to high-performance parallel-operating hardware over the USB is a major strength of APLX. This report ends with a further example of such instrumentation.

Enhanced System Functions – string search and replace using regular expressions

A feature that I had requested has, with Version 4, been added to APLX – a built-in ability to employ Regular Expressions. Text is just a character array to APL, and while the standard APL search and replace function is very powerful when embedded inside APL's array manipulation capabilities, the textual pattern-matching capabilities offered by (Regular Expressions) (RE) can vastly simplify the specification of some tasks.

Up to now to use Regular Expression I have had to make calls on the Unix `sed` (stream-editor) and `grep` (global-recognition expression parser), using the shell `tcsh` in Cygwin. Now that RE parsing is built-in, I do not have to assume that Cygwin is installed, and I have the option of releasing some of these programs for more general use.

I am an avid fan of music, particularly good live music, and an example of such a program that makes extensive use of RE parsing is associated with my music page[4] that tells people about the live music in Kent. A program that I use regularly (but is still under development) makes use of APLX's ability to access the web directly – I maintain a list of pointers to local musicians' web pages, and (with their permission) read their gig-lists directly from their personal sites. There is then some fairly straightforward pattern matching to identify years, months and days, and ensure the dates found are in some sort of logical order. (One site lists months in reverse order, another puts archival gig photos with dates on the gig-listing page!) There is then what would, without REs, be some fairly lengthy code, that has to recognise any reasonable format of time of day (range) and then convert it to the format I use on my gig-listings.

The gig-listing page for the local Kentish group Waterhorse[5], for instance, is shown in Figure 4. Note the pop-up box giving venue information for each event – this is correctly transferred by the gig reading program:

WATERHORSE

Home About Gigs Media Gallery Shop Links Contact

visit venue
xyps.net

East, Chilly, The Hitcher, Whitstable, Kent
CT5 7BP
01227 262001

Waterhorse gigs (yet) for more information about the venue		
25th May	10th Anniversary, Whitstable (21.00)	i
7th June	Whitstable Village (17.30 - 18.30)	i
14th June	Peninsula Swimming, Margate (15.30)	i
21st June	East, Chilly, Whitstable (16.00)	i
6th July	European Water-skiing Championships, Ramsgate (18.00)	i
11th July	The Hitcher, Whitstable (20.45)	i
12th September	Peninsula Swimming, Margate (20.00)	i
28th September	The Hitcher, Whitstable (20.00)	i

Figure 4. Waterhorse gig-listing web page 2008-05-20

Some bands bracket and hyphen one or both of dialling code and event times. As an example of RE pattern matching and replacement, the recognition and interpretation of bracketed event times is simplified if we first recognise and un-bracket these bracketed telephone numbers; as an example RE for this we can use:

```
Txt+'Some text (01227) 123456 and (yet) more text'
Rgx+'(^.*)((\d{3,7})|(\d{4,7}))(\d{4,7})' '\1 \3 \5 \6'
,2 16 055 Txt Rgx
Some text 01227 123456 and (yet) more text.
```

By combining the strengths of APL to manipulate arrays of text with that of Regular Expressions to define compact transformational expressions, we may use the web-aware facilities of APLX to access the gig-listing page directly from the

above musicians and drop the transformed contents into an editor for checking and manual editing if necessary. (None was in this case.)

Figure 5 shows this gig calendar in raw KentFolk EventTable format – note the user defined pop-up menu:

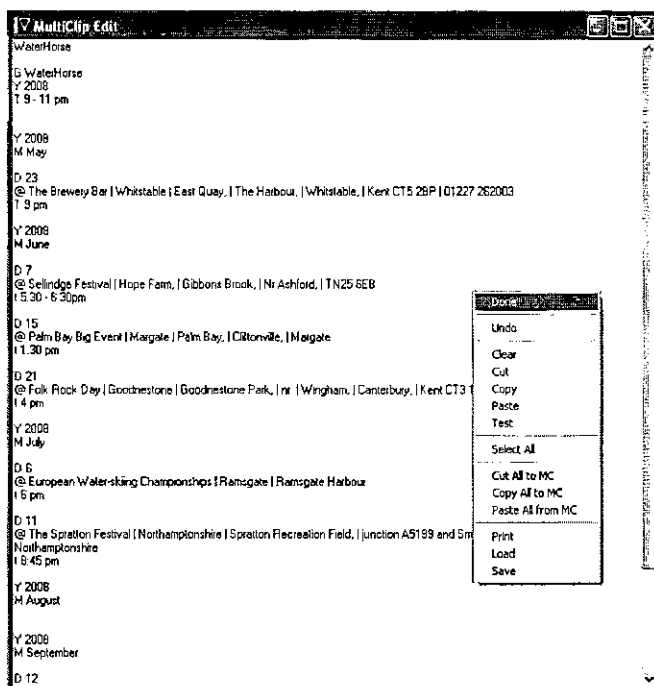


Figure 5. An editable gig-list in KentFolk raw EventTable format, automatically transformed from the above Waterhorse web page, as of 2008-05-20.

Currently I then use a hand-written HTML output stage to generate the KentFolk music calendar (by accessing various look-up tables) which is then uploaded to the server using further APLX commands. Now however APLX version 4 has added a native ability to transform text and tables into HTML, which should greatly simplify such a process. Currently the upload to the server is done via calls on FTP, but the new ability with APLX version 4 to access Microsoft .Net protocols directly should give both greater versatility and greater security for applications that need it.

Norton		Options	
?		14	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Telogy 70's 80's 90's Covers 9 - 11 pm Wychem Jazz 9 Ramsgate Road Broadstairs. CT10 1QQ 01843 88178		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
www.norfolk.com/wharfedale		The Rovers Kent's Gotmen and Acoustic Folk-Rock Blues 12.0 - 11 pm Wychem Jazz 9 Ramsgate Road Broadstairs. CT10 1QQ 01843 88178	
Waterlilies 3-10-95 Soling Festival Hope Farm, Gibbons Brook, Pt. Aschford, TN15 6GB		Harry Tails the music of old and new America Top quality 8 pm West Pecham Music Festival, The Green, West Pecham,	
14		21	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury		Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury	
Jazz Notes Jazz 9 - 11 pm Simple Samba Canterbury			

Figure 6. KentFolk web calendar for June 2008, showing the transformed gig information for Waterhorse

The regular-expression search engine used in APLX is based on version 7.1 of Perl Compatible Regular Expressions (PCRE)[6], an open-source product written by Philip Hazel, who did many good things in his time at the University of Cambridge.

I am still working my way through all the many enhancements, but one I have found useful in the preparation of this report is the ability to export to HTML. There is more than one way to do it, but what I have been doing is just using the Windows clipboard.

If one just copies a section of the contents of the APLX working window and, say, pastes it into a font-aware program, such as MS Word, one gets a copy with the APLX Upright font, with colours; however if one tries exporting this from MS Word as filtered HTML one sees (viewing the text in the Vi or Vim editor) one has characters that will be undefined if one does not have the APLX Upright font installed:

```
<p class=MsoNormal style='text-autospace:none'><span style='font-size:10.0pt; font-family:"APLX Upright";color:blue'> Txt „ 'Some text (01227) 123456 and (yet) more text.' </span></p>
<p class=MsoNormal style='text-autospace:none'><span style='font-size:10.0pt; font-family:"APLX Upright";color:blue'> ,Txt „ 2 16 æSS Txt '(&#91;*&#93;)(0&#100&#97;{3,7})([&#91; ]-&#93;*)(\d{4,7})( *.&#36;)' '\1 \3 \5 \6'</span></p>
<p class=MsoNormal style='text-autospace:none'><span style='font-size:10.0pt; font-family:"APLX Upright";color:black'>Some text 01227 123456 and (yet) more text.</span></p>
```

If however one copies the contents of the clipboard explicitly as follows, one gets a copy on the clipboard that has been converted to HTML:

```
'&#91;' &#91;wi 'contents' ('&#91;' &#91;wi 'text')
```

In particular one now gets a copy that uses the APL385 Unicode font, as is specified for these *Vector* articles:

```
<p>Txt &#8592; 'Some text (01227) 123456 and (yet) more text.' <br>
,Txt &#8592; 2 16 <span style='font-family:"APL385 Unicode">&#9109;</span>SS
Txt '(&#91;*)(&#93;)(0&#100&#97;{3,7})([&#91; ]-&#93;*)(\d{4,7})( *.&#36;)' '\1 \3 \5 \6' <br>
Some text 01227 123456 and (yet) more text.</p>
```

This works *both* ways – one can copy the HTML of the APLX example code here, and paste it back into APLX v4 and then run it.

Using APLX v4 for controlling instrumentation and collecting data

This is just an example of how one can apply APLX as the heart of an advanced set of instrumentation – hopefully a full discussion will come later – however the advantages of having a powerful array-manipulating language like APL intimately connected over the USB to gate arrays with parallel circuitry running at 100s of Megahertz can not be overestimated.

The easy configuration of on-screen menus, interactive text and other entry boxes, as well as buttons and 'LEDs', allow rapid creation of software-based front-panels – as well as on-the-fly reconfiguration of the front-panel at run time for 'soft-instruments'. Direct access to .Net graphical objects will aid porting/interfaces non-APL instrumentation designs to APLX.

Here is one particular piece of kit, a one-dimensional NMR imager using novel technology, created by Lab-Tools for the University of Surrey Physics Department. The Radio Frequency (RF) stages were implemented in off-the-shelf modules using ASICs, the Intermediate Frequency (IF) signal processing and data capturing/averaging stages implemented in the gate-array as firmware, and the

Audio Frequency (AF) and base-band signal processing and data fitting implemented in APL in the host processor (a lap-top, for field-portability):

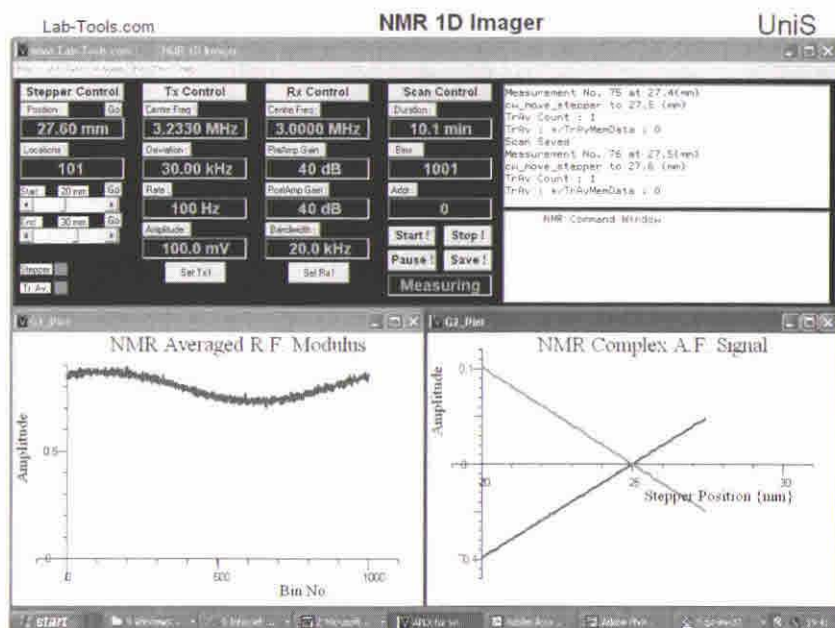


Figure 7. APL interface to gate-array controlled NMR 1D imager.



Figure 8. NMR hardware, for gate-array controlled NMR 1D imager.

And so what use have I made of APL, and APLX in particular?

All I can say is: go see my published papers[7]. I would not have been able to do a fraction of this research without APL to help me.

References

- [1] "A Pipe has two ends. Using APL in a multiprocess/multiprocessor environment. A proposal for a flexible but easy to use syntax." J.B. Webber *APL Quad Quote* 20(1), pp1-2, Sept 1989
 - [2] APLc: <http://home.earthlink.Net/~swsirlin/apl.html>
 - [3] LabVIEW: <http://www.ni.com/labview/>
 - [4] KentFolk: <http://www.kentfolk.com>
 - [5] Waterhorse: <http://www.waterhorse.co.uk/>
 - [6] Perl Compatible Regular Expressions (PCRE): <http://www.pcre.org/>
 - [7] www.kent.ac.uk/physical-sciences/publications/jbww.html
-

Tacit J and I

by Neville Holmes

*School of Computing and Information Systems, University of Tasmania
Launceston 7250, Australia
(Neville.Holmes@utas.edu.au)*

This essay gives the background to the series of articles expounding the joys of functional calculation that starts in this issue of *Vector*. It is largely autobiographical, finishing with a description of experience teaching tacit use of J to Honours students using the articles of the series.

APL in Boxes

It's so long ago that I can't remember just when I first heard of Ken Iverson's APL, but it was in the mid-60s and I remember two particular events.

A colleague at IBM Australia, Jacques Grenot, introduced me to Iverson's 1962 book *A Programming Language* [1]. Ken had taught a class at IBM's Manhattan Systems Research Institute using the book and its 'box notation' that Jacques had taken and been very impressed by.

At roughly the same time, 1964, the *IBM Systems Journal* came out with a formal definition of IBM's pending unified computer architecture written by Ken and two IBM colleagues [2]. This definition used box notation to compress into two wallcharts with accompanying explanation what internal gossip claimed had previously taken two six-foot shelves of ring-binders to contain. The same gossip claimed also that numerous errors had been uncovered through use of the notation.

Box notation had two related virtues, at least for people with a technical background: it used familiar mathematical symbols and it was extremely succinct. Also, the boxes and the sequencing lines placed outside the boxes made the structure very plain to see. For these reasons the notation found some subsequent use for specification in proposals for computing systems, though not by me.

APL\360

In 1970 I took up a staff position in IBM Australia's Systems Development Institute in Canberra. Because of my familiarity with box notation, and because of its development into an interactive system, I pressed to have APL\360 installed on

the Institute's 360 Model 67 running, after a false start with TSS/360, under CP/CMS which later came to be called VM/370.

APL\360 was delightfully immediate. You keyed the expression for your computation into a typewriter terminal and the result was typed on the next line, typically straight away. The 2741 terminal had a 'golf ball' printing element that allowed most of the box notation's mathematical symbols to be used. Because the boxes were impractical on a line-by-line terminal, a few extra symbols had to be added to allow sequencing within function definitions to be defined.

The APL notation was quite easy to learn for anyone in the right frame of mind, and so I had quite a few joint development projects using it in tertiary education in fields such as conventional and naval architecture.

Because computers were still quite novel to the public, the Institute often played host to groups of people from schools or from the community who wished to see a wonderful computer in action. I wrote several interactive packages for them in APL, for example to test them on roman numerals, so that after they had had the equipment in the machine room explained to them, they could be taken into the terminal room to actually *do* something on the computer themselves.

The enthusiasm of young people was remarkable, particularly primary school pupils who were often very reluctant to be moved away by their teachers. Adults were usually much more sedate. On one memorable occasion an elderly visitor took much persuasion to actually seat himself at a terminal but could not be persuaded to touch the keyboard.

The advent of the IBM 5100 enabled me to work with teachers, mainly primary school teachers, to put a computer temporarily in their classroom. The 5100, a 'luggable' computer, rather cunningly used microprogram to simulate a System/360 host for the IBM Basic and APL\360 interpreters. There was no graphic capability, but I wrote an APL package to allow primary school children to key in expressions like

```
A RECTANGLE 17 BY 5 OF 'MW '
```

to be shown interesting patterns. Gratifyingly, some children learnt how to add useful functions to my package. Sadly, I only had the one machine and so could not lend it for long periods.

Unfortunately, the Institute was not successful and was closed down in the mid 70s and I reverted to conventional field work. However, I was still able to get APL used, though usually indirectly through the excellent spreadsheet and

database packages that were available. When visual display terminals started being adopted, use of APL was greatly encumbered by IBM's prolonged delay in providing the APL character set for them.

Tacit J

In the 80s I shifted to Sydney and APL moved into my background. Then I retired from IBM and at the beginning of 1989 took up a position as lecturer in a new degree course in Applied Computing just starting at the Tasmanian State Institute of Technology in Launceston. Ironically, my first duty was to teach Fortran in a service course unit; my first programming language in 1960 had been Fortran for the IBM 650.

As an APL enthusiast I had planned to insinuate APL into my teaching, and I got a copy of IBM's APL2 for the PC for use in teaching eventually, and to keep my hand in in the meantime. Unhappily, the PCs available to me would not run APL2 properly, and after a long struggle I had to give up. Then I reluctantly tried J.

It worked first try. Two differences were uncomfortable. J's use of the ASCII character set was neat but ugly compared to APL's mathematical symbols. The replacement of the up-to-date workspace by the indirect script seemed to be a backwards stride. What more than compensated for these, though, was my discovery of the tacit style of coding in J.

The idea of inferring a functional expression's use of its arguments seemed to me then, and still seems so, to be the purest form of functional programming available. Since then I have only used explicit definitions when deficiencies in my knowledge or in the notation forced me to. At one stage I coded a simulation for von Neumann's Princeton IAS computer and an assembler for it entirely in tacit mode.

An early use I found for J was in a first-semester first-year course unit called "Professional Computing" [3]. As part of this unit I had students learn elementary use of four kinds of coding schemes: functional (tacit J), logical (Prolog), database (SQL), and object-oriented (SQL). Each was learnt entirely through interactive lessons, written in tacit J in the first case.

More seriously, for about a decade I taught an Honours unit called "Functional Programming" in which students were required to undertake a serious project using tacit J. For the first half of the semester students were given a sequence of lectures, actually more like demonstrations, using the handouts that are the basis of this series of articles. Each week the students were required to complete a test given in the handout to confirm their learning of the material.

The students seemed to enjoy the unit, though in the later years they complained of the difficulty of the work. Significantly, this was mostly from full-fee paying overseas students.

I enjoyed teaching them. My main difficulty was, time and time again, getting around a shortcoming of tacit J. One not quite tacit aspect of J is the use of brackets as primitive functions to bring an argument freely into an expression when it was inconvenient to imply its use: [for a left argument,] for a right argument.

The problem was that there was, and is, no similar way to bring an operand freely into a tacit expression. All too often I had to give up insisting on tacit coding and let the student code explicitly. Earlier versions of J had something of this kind, *lev* [. for a left operand and *dex*] . for a right operand, but tragically, because these were parsed as operations they were extremely difficult to use and were dropped out early on.

Occasionally I passed the more interesting results of student projects on to *Vector* for publication [4]. They appreciated getting them published, and getting a year's free subscription, but they didn't renew. Sometimes I asked why, and the answer was usually along the lines that, although they found tacit J fascinating, they didn't see it as something they would be able to use in their postgraduate work. Their coursework had told them that programming was synonymous with Java, and tacit J and I were seen as novelties.

Conclusion

Teaching tacit J coding was a most enjoyable experience. I hope that the publication of my course material for such teaching will bring enjoyment to future readers of *Vector*.

As my "Functional Programming" course unit was only one semester long, my hand-out material was almost entirely on tacit J proper. If a student wanted to link to system capabilities then they had to work this out with little help from me. That many did so perhaps indicates their interest in pure functional programming.

References

- [1] Iverson, K.E. (1962) *A Programming Language*, John Wiley & Sons, New York, 286pp.
 - [2] Falkoff, A.D., Iverson, K.E., & Sussenguth, E.H. (1964) "A formal description of SYSTEM/360", *IBM Systems Journal*, 3(3), 198-261 ().
 - [3] Holmes, W.N. (2000) "Teaching 'Professional Computing'", unpublished
 - [4] Benn, D. (1996) "Turtle Graphics in J", *Vector*, 12(3)
 - Towers, A. (1999) "An Approximation of the N-body Gravitation Problem in J", *Vector*, 15(4)
 - Schofield, B.A. (1999) "Windows BMP Files and J", *Vector*, 16(1)
 - Charman-Williams, A. (1999) "Bayesian Machine Learning in J", *Vector*, 16(2)
 - Chamley, C.A. (1999) "Functional Implementation of Moments Based Region Descriptors in J", *Vector*, 16(3)
 - Dazeley, R. (2001) "J for JPEG: Windows Bitmap to JPEG Baseline Compression", *Vector*, 18(1)
 - Horton, M. (2004) "Celtic Knotwork in J", *Vector*, 21(1)
-

LEARN

Functional Calculation 0 : Numerical Apéritif

by Neville Holmes, University of Tasmania (Neville.Holmes@utas.edu.au)

Abstract

This article is the first in a series expounding the joys of functional calculation. Functional calculation does with operations applied to functions and numbers what numerical calculation does with functions applied to numbers. The functional notation used as the vehicle in this series is provided by a freely available calculation tool called J.

This article gives a detailed illustration of a few numerical uses of the notation supported by the J interpreter. This explanation is preliminary to following articles, which will review some of that tool's support for numerical calculation. The illustration takes a simple function and shows how J, or any other tool for functional calculation, can be used to investigate special values of the function.

Calculation

Calculation is an honourable and satisfying occupation with a long history and a rich tradition [1].

Relatively recent developments in electronic computation have led to the adoption of three classes of aids to calculation, namely:

- electronic or programmed calculators based directly on traditional calculation,
- spreadsheet programs, and
- complex mathematical systems.

Electronic calculators are unreliable and very poorly designed [2]. Spreadsheet programs are very powerful and useful for the kind of calculation that suits them,

but clumsy and limited otherwise. The complex mathematical systems are extremely powerful, but in an elephantine way.

The kind of functional calculation described in following articles is like an enormous but sympathetic extension of traditional calculation. The extension is particularly effective in dealing with the kinds of tables to which spreadsheet programs are suited but allowing conveniently for tables of many dimensions. It is also useful for collections of tables of different kinds. The extension is of surprising expressive power so that problems which would otherwise require the might of a complex mathematical system can be tackled simply.

This article will go through some simple numerical calculations step by step, explaining them in detail. Readers will get most benefit out of the description if they get their own interpreter and themselves carry out the calculations (and any others suggested by the examples) as they are being described.

The notation used here is called simply J, and programs (for many of the more popular computing platforms) to evaluate expressions in the notation can be downloaded from the Internet at <ftp://archive.uwaterloo.ca/languages/j> and help is available from the Usenet group *news:comp.lang.apl*

Because the programs which evaluate J expressions as they are keyed in seem to interpret the notation, they are called interpreters. The J interpreters that can be got for free from the Internet are old versions, but information about up-to-date versions, and much, much more, can be found at <http://www.jsoftware.com/>

The primitive functions, those functions with symbols for names, used in this article are few, and very many more such are provided by the interpreter, and are listed and briefly explained by the interpreter's help facilities.

The purpose of this article is to show how simple functions can be used simply to get useful results, the kind of results that show the interpreter in the light of a utility calculator. To allow this to be better appreciated, examples which involve the use of lists of numbers have been chosen.

The calculations used here as an example are based on an exploration of a function named *f* in the description. Any reader carrying out the example calculations can define *f* by keying in

```
f =:-o.
```

but this definition will not be further explained here because the expression used to define *f* is a functional expression and, as such, will be explained in later articles of this series.

Finding Zeroes

Given the function *f* and a need to explore its behaviour, the first step is to name a useful list of test values, and then to apply the function to those values. Keying in the expression

```
nn =: i.11
```

uses the primitive function *i.* (pronounced *integers*) to produce the first eleven non-negative integers, to which the copula *=:* (pronounced *gets* or *becomes*) gives the name *nn*.

The copula is used only for conferring names, and it is not a function as such. No result is shown because the interpreter is just naming. However any named result can be looked at just by keying in its name, in this case

```
nn
0 1 2 3 4 5 6 7 8 9 10
```

shows *nn* to be a list of ten integers.

Some general remarks are appropriate at this point. The J interpreter takes in the expressions it evaluates indented, but the results are shown un-indented. For the first expression shown above, the spaces between the *nn* and the *=:*, and between the *=:* and the *i.*, are not required by the interpreter, but have been put there to make the expression more readable. More than one space could have been used with same result, and spaces could have been put between the *i.* and the *11* without affecting the result. However, putting a space within the *=:* or the *i.* would have affected the result just as putting a space within the *11* would have, because the notation uses suffixed dots and colons to extend the ASCII character set, so that in effect *=* and *=.* and *=:* are three different characters. The function *f* can now be evaluated for the argument *nn*, and its result displayed, by

```
f nn
1 _0.423311 1.10587 10.6608 42.0318 132.705 384.579 1074.64 2955.83 8074.81 21995
```

which shows a quite distinctive behaviour indeed.

However, the inconveniently small font forced on the printing above of the results of `f nn` makes it difficult here to appreciate fully the nature of that behaviour, so it is useful to define with scant comment

```
show =: ".&(0.1&":)
```

which names a function that can be used as in

```
show f nn
1 _0.4 1.1 10.7 42 132.7 384.6 1074.6 2955.8 8074.8 21995
```

to force each value to be displayed with at most one decimal place. This is perhaps not so useful here, but will be so later.

One of the values in the result list is negative. Notice that the notation disambiguates the hyphen by representing the property of negativity by the sign `_` so that the hyphen can be kept to stand for the negation and subtraction functions. To investigate how `f` behaves for negative arguments, the test values can have ten subtracted from them before applying `f` by keying in

```
show f nn-10
31.4 28.3 25.1 22 18.9 15.7 12.6 9.4 6.4 3.5 1
```

where the values displayed have been rounded to one decimal place by `show`. To make completely plain what the argument value is for each result value, the *laminat* function `,:` can be used in

```
a ,: show f a =: nn-10
_10 _9 _8 _7 _6 _5 _4 _3 _2 _1 0
31.4 28.3 25.1 22 18.9 15.7 12.6 9.5 6.4 3.5 1
```

where the test values are temporarily named `a` so that they can be stacked by `,:` on top of the results. Care must be taken in putting together expressions like the one keyed in above. For example, the copula `=:` gives the name to its left to its predicate, which is everything to its right, just as in English sentences the predicate of a verb is everything to its right. Providing there is a space between the `f` and the `a`, then there are two names, and the copula only uses the name next to it. If there's no space, the name is `fa` and there is no function `f` anymore. Similarly, the predicate of the *laminat* function is everything to its right, which is the result of applying `f` to the values of `a`.

The same technique can be used to stack the values of `nn` on the result of `f nn` by

```
nn ,: show f nn
0      1      2      3      4      5      6      7      8      9      10
1 _0.4 1.1 10.7 42 132.7 384.6 1074.6 2955.8 8074.8 21995
```

which shows quite different behaviour from that for negative argument values.

On some versions of J you might not get the table of the example looking as neat as it does here. This will be because the host system is putting your numbers out using an unsuitable type font, one that does not space the letters equally, thus causing the misalignment in tables. You should change the font, presumably to the one that works best for me on the screen, that is, ISIJ, though APL385 Unicode is the font used in this printed article for examples.

The most interesting feature here is that the function has two zeroes, one between arguments of 0 and 1, and one between 1 and 2. These zeroes can now be explored one at a time, though the exploration of the second zero is here left as an exercise for the reader.

The First Zero

The simplest zero to explore is the one between 0 and 1, and by

```
a ,: show f a=:nn%10
0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1
1 0.8 0.6 0.4 0.2 0.1 _0.1 _0.2 _0.3 _0.4 _0.4
```

the eleven values between and including 0 and 1 are displayed. Because the restrictions of the ASCII character set require the `%` symbol to be used for the *divide* function, rather than the more usual `÷` symbol, the expression `nn%10` asks for `nn` to be divided by ten. Thus, the expression `nn%10` gives a list of values between 0 and 1 incrementing by tenths, and from the result of applying `f` to `nn%10` it can easily be seen that the zero lies between argument values of 0.5 and 0.6, about halfway between.

The limitations of the function `show` are evident here, but it can easily be redefined as

```
show =: ".&(0.2&":)
```

which then returns a second decimal place to sight in

```
a ,: show f a=:nn%10
0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1
1 0.79 0.59 0.41 0.24 0.08 _0.06 _0.19 _0.29 _0.37 _0.42
```

so that the zero can be predicted to be slightly closer to 0.6 than to 0.5. Not only are fractional values like 0.5 and 0.6 always displayed with a leading zero, but they must also be keyed in with one. Hence the next step in finding the zero, going down to hundredths, is

```
a ,: show f a=: 0.5+nn%100
0.5 0.51 0.52 0.53 0.54 0.55 0.56 0.57 0.58 0.59 0.6
0.08 0.06 0.05 0.03 0.02 0.01 _0.01 _0.02 _0.04 _0.05 _0.06
```

where the leading zero of the 0.5 of the expression being keyed in is required, not optional. The new display of the results shows that the zero is between argument values 0.55 and 0.56.

The result values are becoming rather too close to zero, and will now need to be scaled up, so in

```
a ,: show 1000*f a=: 0.55+nn%1000
0.55 0.551 0.552 0.553 0.554 0.555 0.556 0.557 0.558 0.559 0.56
5.38 3.97 2.56 1.16 _0.24 _1.64 _3.04 _4.44 _5.83 _7.23 _8.62
```

the results are multiplied by a thousand before being displayed, the * symbol being used to stand for the multiplication function because the x symbol is not provided in the ASCII character set! In the absence of parentheses, the right argument of a function is the entire expression to the right of the function symbol, so that the * in 1000* takes the result of the application of f as its right argument. By now, the argument values are getting a bit too long to print out comfortably, and the way they correspond should be getting to be clear, so they won't be shown stacked any longer. The argument of f now being from 0.55 to 0.56, and the zero being located between 0.553 and 0.554, the next narrowing down can be done by

```
show 10000*f 0.553+nn%10000
11.6 10.2 8.79 7.39 5.99 4.58 3.18 1.78 0.38 _1.02 _2.42
```

which now places the zero between arguments 0.5538 and 0.5539. The J interpreter makes it easy enough to bring down the previous line and simply insert the three extra digits needed at each stage of the calculation. But by now the strings of zero digits are becoming a bit hard to keep visual track of, so, using e-

notation for the large scaling integer and the small interval size, the next step becomes

```
show 1e5*f 0.5538+nn%1e5
3.79 2.39 0.99 _0.42 _1.82 _3.22 _4.62 _6.02 _7.42 _8.83 _10.23
```

This process of narrowing down has now become somewhat repetitive. Of course, the whole process could be made automatic, but by doing it step by step the succession of numbers gives a better feel for how the function f is behaving. Nevertheless, the repeated part of the expression used just above could relatively easily be encapsulated (by an experienced J user) as in

```
g =: ]*f@(+nn&%)
```

so that the investigation can now proceed as follows.

```
show 0.55382 g 1e6
9.86 8.46 7.06 5.66 4.26 2.85 1.45 0.05 _1.35 _2.75 _4.15
show 0.553827 g 1e7
0.51 _0.89 _2.29 _3.69 _5.09 _6.49 _7.9 _9.3 _10.7 _12.1 _13.5
show 0.5538270 g 1e8
5.14 3.73 2.33 0.93 _0.47 _1.87 _3.27 _4.68 _6.08 _7.48 _8.88
show 0.55382703 g 1e9
9.31 7.91 6.51 5.11 3.71 2.31 0.9 _0.5 _1.9 _3.3 _4.7
```

For some time now the results have been showing a strange uniformity. It might be suspected either that the arithmetic is beginning to give out, or that the variation between successive values has become uniform. This can be checked by taking the result values in pairs and subtracting them. This is done (without explanation here) by

```
show 2-/\0.55382703 g 1e9
1.4 1.4 1.4 1.4 1.4 1.4 1.4 1.4 1.4 1.4
```

which immediately shows the postulated uniformity. Indeed, keying in the expression without the show gives ten values 1.40169 so it is very uniform. This could allow a fairly accurate interpolation by hand of just where the zero is, but, for the purposes of exploration, the estimation process is continued here as before, except that now

```
show =: ".&(0.1&":)
```

because only one decimal place is now needed to be shown.

```

show 0.553827036 g 1e10
9 7.6 6.2 4.8 3.4 2 0.6 _0.8 _2.2 _3.6 _5
show 0.5538270366 g 1e11
6.2 4.8 3.4 2 0.6 _0.8 _2.2 _3.6 _5 _6.4 _7.8
show 0.55382703664 g 1e12
6.3 4.9 3.5 2.1 0.7 _0.7 _2.1 _3.5 _4.9 _6.3 _7.7
show 0.553827036644 g 1e13
7.2 5.8 4.4 3 1.6 0.2 _1.2 _2.6 _4 _5.4 _6.8
show 0.5538270366445 g 1e14
1.9 0.5 _0.9 _2.3 _3.7 _5.1 _6.5 _7.9 _9.3 _10.7 _12.1
show 0.55382703664451 g 1e15
4.9 3.3 2 0.7 _0.7 _2 _3.3 _4.7 _6.2 _7.5 _8.9
show 0.553827036644513 g 1e16
6.7 4.4 6.7 2.2 0 0 0 _2.2 _6.7 _2.2 _6.7

```

The values have become erratic, so much so that doubt much attach to any attempt to add further digits to the value found for the first zero. However, checking with

```

f 0.55382703664451 0.553827036644513 0.5538270366445135
4.88498e_15 6.66134e_16 0

```

suggests that at least the first fifteen decimal places should be reliable, provided the calculation built into the interpreter is reliable to such precision. It is important to realise that the barrier that has been encountered here is one of precision, not range, since

```

4*4e32 4e_32
1.6e33 1.6e_31

```

shows that a much greater range can be satisfactorily used.

Finding the lowest Value

While the value of the second zero can be as easily found as the first, between the two zeroes all the values of f are negative, it might be presumed. This can be explored by

```

show 0.5 g 5
0.4 _0.9 _1.8 _2.3 _2.1 _1.2 0.7 3.6 7.8 13.7 21.6

```

where the interval between arguments is 0.2 and starting from 0.5. This suggests a minimum value for f of about -2.3% when the argument is about 1.1, probably just a bit larger than 1.1.

Before going further, it is convenient to redefine f as its negation, thus

```

f =: -&(-o.)

```

so that the values being looked at will be free of the negative sign, though now what was lowest value will have become the highest. Consequently,

```
show 1.1 g 100
45.2 45.3 45.4 45.4 45.5 45.5 45.4 45.4 45.3 45.1 45
```

suggests that, while the now-highest value of f is 0.455, the argument corresponding to this value is round about 1.14. But this is now estimation, and one decimal place for `show` is too few.

The next steps in calculation are therefore

```
show =: ".&(0.2&":)
show 1.14 g 1000
454.65 454.66 454.67 454.68 454.68 454.68 454.68 454.67 ...
```

which verifies the highest value to be about 0.45468 but it is mainly a guess that the argument for that value is about 1.144. Even when we issue

```
show=: ".&(0.3&":)
```

to get three decimal places, which then allows

```
show _45468+10*1.144 g 10000
0.151 0.172 0.191 0.206 0.218 0.226 0.232 0.234 0.234 0.23
0.223
```

where the `_45468+10*` reduces the size of numbers displayed and gets an extra decimal place at the same time. The procedure can be continued, thus

```
show _4546823+100*1.1447 g 100000
0.447 0.455 0.46 0.461 0.46 0.455 0.447 0.436 0.422 0.405 0.384
```

which adds an extra digit (a bit uncertainly – is it 2, or 3?) to the value of the argument, and two extra digits to the value of the function. The procedure goes on, here splitting the difference between the 2 and the 3.

```
show _454682346+1000*1.144725 g 1000000
0.102 0.116 0.126 0.134 0.138 0.139 0.137 0.132 0.124 0.113
0.098
```

which confirms 2, and its successor 9, and again adds a couple of digits, 13, maybe even 138, to the value of the function. Now

```
show _454682346138+100000*1.144729 g 1e7
0.132 0.395 0.626 0.825 0.994 1.131 1.236 1.311 1.353 1.364
1.344
```

shows that the last digit of the previous function value should have been 9 rather than 8, but still gives another two function-value digits for the single argument digit. In the next step

```
show _45468234613935+1e6*1.1447298 g 1e8
0.273 0.547 0.766 0.984 1.117 1.297 1.344 1.391 1.43 1.477
1.477
```

leads immediately to

```
show _4546823461393647+1e7*1.14472989 g 1e9
10 5 _4 5 1 1 _4 1 1 _4
```

where everything now blows up at a point where the function value is known to about 16 digits, but the argument value to only 9.

This mismatch between the accuracy of the result and that of its argument is a good illustration of the inherent unreliability of simple computer arithmetic. More extreme examples of mismatch could easily be concocted.

Conclusion

The procedures described in this article show the way, or the style, in which calculations can be carried out interactively using a tool like J. If the calculation tool is mastered, most calculations can be carried out in the same style. The barrier is attaining that mastery. Indeed the repeated searching for an answer by a kind of trial and error is not characteristic of functional calculation, since mastery will enable functions to be defined which can do the searching automatically. Such virtuosity would not be appropriately demonstrated in an *apéritif*.

The interactive style of calculation depends on two factors illustrated in this article. The first is simply the ability to key in an expression and have it evaluated 'on the fly'. The second is the ability to give a name to an expression then use that name from then on as though it were that expression.

This article is only an *apéritif* to the main meal of getting to grips with a ramified functional calculation tool.

References

1. Menninger, K. (1958) *Zahlwort und Ziffer*, Vandenhoeck & Ruprecht, Göttingen, 2nd edition (as *Number Words and Number Symbols* by MIT Press in 1969).
2. Thimbleby, H.W. (1995) "A New Calculator and Why it is Necessary", *The Computer Journal*, 38(6), 418–433.

Tutorial: User-Defined Classes in APLX Version 4

by Simon Marsden, MicroAPL Ltd (microapl@microapl.co.uk)

Introduction

APLX Version 4 adds object-oriented programming facilities to APLX's core APL2-compatible language. These facilities are broadly similar to those implemented in other object-oriented programming languages (such as C++, C#, Java, or Ruby), but with the difference that APL's array-programming approach applies to classes and objects in the same way as it applies to ordinary data. In addition, the APLX implementation is fully dynamic, that is to say classes and objects can be changed at run-time or when debugging.

Classes can be written in APL, or they can be written in other languages (such as C#), or they can be built-in to the APLX interpreter as System Classes (analogous to the familiar System Functions). In this tutorial, we'll focus on classes written in APL. (If you want to try out the tutorial yourself, you can download a demonstration version of APLX Version 4 from <http://www.microapl.co.uk/apl>)

Jargon

The fundamental building block for object-oriented programming is the *class*. For example, in a commercial invoicing application, a given class might represent the attributes and behaviour of an invoice, and another class might represent a credit note. In an application concerned with geometry, a class might represent a sphere, or a rectangle, or a polygon. A class contains definitions both for program logic (functions and operators, known collectively as the *methods* of the class), and for data (named variables associated with the class, known as *properties*). The term *members* is used to describe both the properties and methods of a class.

In most cases, when you come to use a class, you need to create an *instance* of that class, also known as an *object*. Whereas the class represents an abstraction of (say) an Invoice, or a Sphere, or a Rectangle, an object represents a particular invoice, sphere or rectangle. Typically, you may have many instances of a given class, each containing independent copies of data (*properties*), but all supporting the same program logic (*methods*).

Getting Started

To make this clearer, let's use an example of a class representing a circle. To keep things simple initially, we will give the `Circle` class a single property representing the radius of the circle.

There are a number of ways to create the `Circle` class, but for now we'll just start with a `CLEAR WS` and enter the following APL line, which creates a class and inserts a property called 'radius' into it:

```
'Circle' ⌵IC 'radius'
```

This has created a new class, which we can see by using the `)CLASSES` system command, which is analogous to `)FNS` and `)VARS`:

```
)CLASSES
Circle
```

To create an instance of the `Circle` class, you typically use the `⌵NEW` system function:

```
FirstCircle←⌵NEW Circle
```

This has created a new instance (object) and assigned a reference to it in the variable `FirstCircle`:

```
)VARS
FirstCircle
```

If you try to inspect the new object, APL will display it in the following format by default. Note that (unless you change the default display) the object is displayed by showing its class name in square brackets.

```
FirstCircle
[Circle]
```

Now let's assign a value to the new `Circle` object's radius:

```
FirstCircle.radius←10
FirstCircle.radius
10
```

Note the *dot-notation* used to specify the object and property being assigned. Except that it's a member of the object `FirstCircle`, the radius property behaves like any other APL variable:

```

      FirstCircle.radius
1 2 3 4 5 6 7 8 9 10
      FirstCircle.radius=20
0

```

We can also create a vector containing five different circles:

```

      CircleList←NEW "5pCircle
      CircleList
[Circle] [Circle] [Circle] [Circle] [Circle]

```

It is possible to set the radii of all of the circles in the same statement:

```

      CircleList.radius←10 50 20 30 20
      CircleList.radius
10 50 20 30 20
      CircleList.radius=20
0 0 1 0 1

```

Notice that APLX lists the radius values for all five circles in a single vector, which can be used in expressions.

We could also specify all the radii using a single scalar, in which case the scalar is assigned to the radius property in each object using scalar extension:

```

      CircleList.radius←20
      CircleList.radius
20 20 20 20 20

```

You cannot access a property if it does not exist in the class definition. For example:

```

      FirstCircle.Colour←'Red'
VALUE ERROR
      FirstCircle.Colour←'Red'
^

```

To add a new property it is necessary to modify the class definition, which we will cover in more detail later.

Now let's add a method called *Area* to our *Circle* class. Again, there are many ways of editing classes in APLX, but for now you can use the following:

```

)EDIT Circle.Area

```

Use the editor window to create the following method:

```

      ▽R←Area
[1] R←(01)×radius*2
      ▽

```

(You do not enter the ▽ characters or line number when using the editor window.)

We can now call our new method using the following notation. (Notice that the new method can be applied to existing objects – try doing that in Java!)

```

      FirstCircle.Area
314.1592654

```

Within the method, the object's radius can be referred to directly as just `radius`, without using dot notation. APL knows that the `Area` method has been called for the object `FirstCircle`, and hence uses the `radius` value contained in the object.

Again, if you have an array of objects you can apply the same method to each one.

```

      CircleList.radius←10 50 20 30 20
      CircleList.Area
314.1592654 7853.981634 1256.637061 2827.433388 1256.637061

```

Internally, APL will call the `Area` method on each of the objects in turn, in the order in which they occur in the object list.

It is also possible to have vectors containing references to objects of different classes. Suppose that we have a new class called `Square`, in addition to our `Circle` class, and that `Square` also has a class method called `Area`:

```

      FirstSquare←NEW 'Square'
      ShapeList ← FirstCircle,FirstSquare
      ShapeList
[Circle] [Square]
      ShapeList.Area
314.1592654 100

```

Note that there is not necessarily any relationship between the `Area` methods in the `Circle` and `Square` classes.

What would happen if we create a new `Circle` object and immediately try to display its `radius` property?

```

        AnotherCircle←NEW Circle
        AnotherCircle.radius
VALUE ERROR
        AnotherCircle.radius
^

```

Because the radius property of the new object has not been assigned, we get a VALUE ERROR; the new object is incomplete. Is there some way we can ensure that only completely-built objects are created by NEW?

One way in which this can be done is to add a new class method called a *Constructor*. This is an APL method which has the same name as the class. The APL interpreter will call the constructor method automatically when a new object is created.

The Constructor for the Circle class might look something like this:

```

    VCircle R
[1] radius←R
    V

```

Note: We haven't worried too much yet about how to edit APL classes. APLX has a powerful class editor, which is what you would use in practice. For now, if you want to experiment with adding this constructor to your class, try:

```
)EDIT Circle.Circle
```

The constructor method we added takes a right argument, which is the initial value of the new circle object's radius. To create the circle, the value is passed as an argument to NEW:

```

        AnotherCircle←NEW Circle 100
        AnotherCircle.radius
100

```

In some object-oriented languages, constructors are the only way of assigning initial values to properties. In APLX, it is also possible to specify the default value of any properties in the class definition. Instead of using a constructor, we could have changed our Circle class to specify that the radius property of all new Circle objects should have an initial value of 10. This is discussed further in the section *Types of Property* below.

System Methods

As well as user-defined methods like *Area* above, each object/class can make use of a number of *system methods*. There are around a dozen of these, but for now we will just note two. The system method `⌈CLASSNAME` returns the name of an object's class as a character vector:

```
FirstCircle.⌈CLASSNAME
Circle
```

The system method `⌈NL` can be used to discover the names of the methods and properties in a class (or an instance of that class):

```
Circle.⌈NL 3
Area
FirstCircle.⌈NL 2
radius
```

Inheritance

Suppose that we wanted to re-write our example to handle two types of shape, circles and squares.

One way to do this would be to use a general class called *Shape*. We could add a property which specified the type of shape (0 for *Circle*, 1 for *Square*), and we could add an *Area* method, something like this:

```
VR←Area
[1] :If type=0
[2]   A Circle
[3]   R←(o1)*radius*2
[4] :Else
[5]   A Square
[6]   R←sidelength*2
[7] :EndIf
V
```

However, in object-oriented APL there is a much more elegant way to do this, by using *Inheritance*.

When you define a class, you can specify that it *inherits* from another class. The new class is said to be the *child*, and the class it inherits from is the *parent* or *base* class. Inheritance means that (unless you explicitly change their definition), all of the properties and methods defined in the parent class are also available in the child class. This works for further levels of inheritance as well, so that methods

and properties can be inherited from the immediate parent, or from the parent's parent, and so on. The terms *derived classes* or *descendants* are sometimes used to denote the children of a class, and the children's children, and so on. Similarly, the term *ancestors* of a class is used to denote the parent, parent's parent, and so on.

So you might have a class *Shape*, representing an abstract geometric shape. This might have properties called *X* and *Y* giving the centre point of the shape, and methods called *Move* and *Area*.

A *Circle* class might inherit from *Shape*, introducing further properties such as *radius*. Equally, a class *Polygon* might also inherit from *Shape*, and further classes *Triangle* and *Square* inherit from *Polygon*. All of the classes *Circle*, *Polygon*, *Triangle* and *Square* are *derived* from *Shape*. Because of the way inheritance works, they would all include the properties *X* and *Y*, and the methods *Move* and *Area*.

When a class inherits from another, you can specify that the definition of a given method of the parent (or the initial value of a property) is different in the child class. In our example, you would need to supply a different definition of the *Area* method for a *Circle* and a *Square*. This is known as *overriding* the method.

Definition of the *Area* method in the *Circle* class:

```
VR←Area
[1] R←(01)×radius*2
V
```

Definition of the *Area* method in the *Square* class:

```
VR←Area
[1] R←sidelength*2
V
```

For classes defined in APLX, all methods can be overridden, and all methods are *virtual*, that is to say if method *A* in a base class calls another method *B*, and the second method *B* is overridden in a child class, then running method *A* in the child class will cause the overridden version of *B* to be called, not the version of *B* defined in the parent. For example, if you are running a method defined in the base class *Shape*, and that method calls *Area*, the version of *Area* which gets called will be *Circle.Area* or *Square.Area* as appropriate.

APLX uses an inheritance model known as *single inheritance*. This means that a child class can be derived from only one parent (which may itself derive from another class, and so on).

Object References and Class References

When you create an object, i.e. an instance of a class (using the system function `NEW`), the explicit result that is returned is not the object itself, but a *reference* to the object. This reference is held internally as just an index into a table of objects which APLX maintains in the workspace. If you assign the reference to another variable, the object itself is not copied; instead, you have two references to the same object. This is discussed in more detail below.

Of course, because APL is an array language, you can have arrays of object references, and you can embed object references in nested arrays along with other data. For example, you might have an array containing references to hundreds of `Rectangle` objects.

You can also have a reference to a class. This makes it possible for general functions to act on classes without knowing in advance which class applies.

The Null Object

As its name implies, the *Null object* is a special case of an object, which has no properties and no methods of its own (although System methods may apply to it). A reference to the Null object displays in the special form:

```
[NULL OBJECT]
```

A reference to the Null object can arise for a number of different reasons:

If you have an array of object references, the prototype of the array is a reference to the Null object. For example:

```
VEC←NEW "Rectangle Sphere Triangle
VEC
[Rectangle] [Sphere] [Triangle]
1+3+VEC
[NULL OBJECT]
```

An external call or System method may return a Null object, for example if you are looping through a linked list of objects and reach the last one. And APLX may be forced to set an object reference to Null, because it is no longer valid. For example, this will happen if you `SAVE` a workspace which contains a reference to an external object (e.g. a Java or .NET object). On re-loading the workspace at a later date, the object reference is no longer valid since the external object no longer exists.

Types of Property

When you define a class, you specify the names of the properties of that class, which can be used to hold data associated with the class. You can optionally specify a *default value* for the property, that is the value which the property will have in a newly-created instance of the class. You can also specify that the property is *read-only*, which means it is not possible to assign a new value to it.

Most properties are *instance properties*, which means that each instance of the class has a separate copy of the property (for example, the X- and Y-position of a Shape). Occasionally, however, it is useful to define a *class-wide property* (known in some other languages as a *static* or *shared* property). This is a property where there is a single copy of the data, shared between all instances. This is useful for cases such as keeping a unique incrementing serial number (the next invoice number, for example), or to define a constant (such as a text string to appear on all invoices) for all members of the class.

Implementation note: APLX uses a 'create-on-write' approach when you assign to an instance property. This means that, if you have never changed the value of a property for a particular instance since the instance was first created, the value which is returned when you read the property is the default value stored in the class definition. Thus, if you change the class definition so that the property has a different default value, the change will immediately be reflected in all instances of the class, unless the property has been modified for that instance.

Name Scope, and Public versus Private Members

The members of a class (i.e. its properties and methods) can be either *public* or *private*. Public members can be accessed from outside the class, whereas private members can only be accessed from within methods defined in the class (or from desk calculator mode, if a method has been interrupted because of an error or interrupt and the method is on the)SI stack). Private members can also be accessed by methods defined in a child (derived) class. If you are familiar with other object-oriented languages such as C++ or Visual Basic, this means that private methods in APLX correspond to 'protected' methods in those languages.

Methods and properties are, by default, public, just as in conventional APL variables referenced in a function are global. (You can make them private by using the class editor, or localising them in the class header using the del editor.)

If you want to access a public member of an object from outside the class (i.e. not within a method of the class), then you use *dot notation* to refer to it. This takes the form `ObjectReference.MemberName`. For example, suppose you have a variable

`myrect` which is a reference to an object of class `Rectangle`. You could call the `Move` method and access the `X` and `Y` properties for that object as follows:

```

myrect.X←45
myrect.Y←78
myrect.Move 17 6
myrect.X
62
myrect.Y
84

```

Within the methods of the class itself, you don't normally need to use dot notation. This is because the search order for symbols encountered when executing a method is as follows:

1. First, APLX looks to see if the symbol refers to a member defined in the class of the object.
2. If not, it looks to see if the member is defined in the parent class (if any), iterating through each of the ancestors in turn.
3. If it is not found in any of the ancestors, it then looks in the local variables of the method.
4. Finally, it looks in the global symbol table.

Thus, a simple implementation of the `Move` method above (defined in the `Shape` class from which `Rectangle` derives) might be something like this:

```

▽ Move B
[1]  A Move shape by amount B specified as change to X, Y
[2]  (X Y)←(X,Y)+B
▽

```

Canonical Representation of a Class

The canonical (text) representation of a class is returned by `⌈CR`, in exactly the same way as applies to ordinary APL functions and operators. The first line is the class header. This comprises the name of the class, followed (if the class inherits from another class) by a colon and the name of the parent class. Any private members of the class (i.e. names which are local to the class) are then listed, separated by semi-colons. The header line ends with a left curly brace `{` character.

The properties of the class are then listed, one per line. The name of the property is listed first. If it has a default value, an assignment arrow follows, and then the transfer form of the expression which initializes the property. If the property is

read-only, two assignment arrows are used. If the property is class-wide (i.e. there is only a single copy shared between all instances in the workspace), then the whole line is enclosed in curly braces.

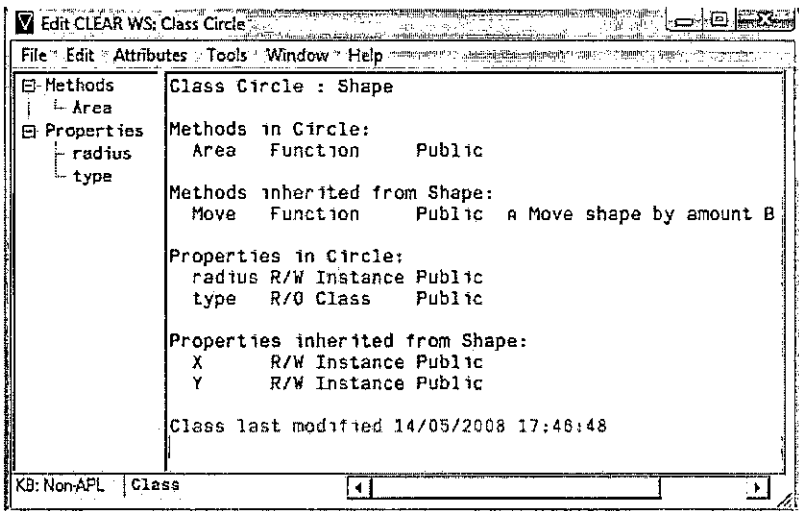
Any methods then follow, delimited by del characters, and a closing right curly brace ends the definition.

For example:

```

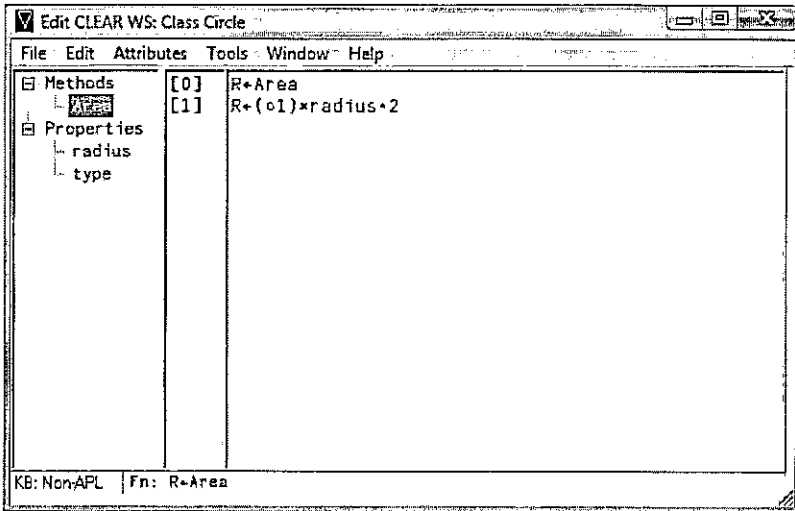
      CR 'Circle'
Circle : Shape {
  radius
  {type++'circle'}
  VR←Area
  R←(01)×radius*2
  V
}
```

As you would expect, `CR` can be used to turn the text form into a class definition in the workspace. In practice, though, you'll normally tend to use the APLX Class Editor, which shows the class as a whole:



APLX Class Editor

... or allows you to edit a specific method or property:



Method Editor

You can also use the del editor, or you can edit individual methods or properties directly.

Constructors

As we saw earlier, a *constructor* is a special type of method, which is run automatically when an instance of a class is created using `NEW`. It can be used to initialize the object, optionally using parameters passed to `NEW`. For example, you might use this mechanism to specify the initial position of a `Rectangle` object.

For a user-defined class, a constructor is defined as a method which takes a right argument, and which has the same name as the class itself. Any arguments to the constructor can be provided as extra elements on the right argument of `NEW`. When the constructor is run, these extra elements are passed as the right argument to the constructor. If there are no extra elements, an empty vector is passed as the right argument to the constructor.

For example, suppose the class `Invoice` looks like this:

```

Invoice {
    TimeStamp
    Account
    InvNumber
    {Serial+0}
▽Invoice B
    A Constructor for class Invoice. B is the account number
    Account+B
    TimeStamp+□TS
    Serial+Serial+1
    InvNumber+Serial
    ▽
}

```

This is a class which has a constructor and four properties. One of the properties (`Serial`) is a class-wide property, which means it has only a single value shared between all instances of the class.

When a new instance of this class is created, the constructor will be run. It will store the account number (passed as an argument to `□NEW`) in the property `Account`, and store the current time stamp in the property `TimeStamp`. It will then increment the class-wide property `Serial` (common to all instances of this class), and store the result in the property `InvNumber`. To see the properties, we can use the system method `□DS` which summarizes the property values:

```

S+□NEW Invoice 23533
S.□DS

```

```

Account=23533, TimeStamp=2007 10 11 15 47 34 848, InvNumber=1

```

```

T+□NEW Invoice 67544
T.□DS

```

```

Account=67544, TimeStamp=2007 10 11 15 48 11 773, InvNumber=2

```

Where a class inherits from another class, the constructor which gets run automatically is that of the class itself (if it has a constructor), or of the first ancestor class which has a constructor. Normally, in a constructor, you will want to do some initialization specific to the class itself, and also call the constructor of the parent class (using `□PARENT`) to do any initialization which it and its ancestors require. You can do this at any point in the constructor; there is no restriction on where you make this call to the parent's constructor; indeed, you don't have to call it at all if it is not appropriate.

In APLX, a constructor is also a perfectly ordinary method; it can be called in the normal way by one of the other methods in the class, or from outside (if it declared as Public). This can be useful for re-initializing an object.

Some object-oriented languages also include a special method called a *destructor*, which is called just before the object is deleted. APLX user-defined classes do not have destructors. This means that, if you need to release system resources (for example, close a file or a database connection), you need to call a method to do that explicitly before erasing the last reference to the internal object. However, APLX will automatically take care of deleting all the properties of the object, and releasing the memory back to the workspace.

Creating Objects (instances of classes)

As we have seen, the system function `ⓘNEW` is the principal means by which you create an object, i.e. an instance of a class. The class can be either written in APL (an *internal* or *user-defined* class), or a built-in System class, or a class written in an external environment such as .Net, Java or Ruby (an *external* class). `ⓘNEW` creates a new instance of the class, runs any constructor defined for the class, and returns a reference to the new object as its explicit result.

The class is specified as the right argument (or first element of the right argument). It can be specified either as a class reference, or as a class name (i.e. a character vector). Any parameters to be passed to the *constructor* of the class (the method which is run automatically when a class is created) follow the class name or reference.

If you specify the class by name, you also need to identify in the left argument the environment where the class exists, unless it is internal.

Creating instances of internal (user-defined) classes

Normally, you create an instance of a user-defined class by passing the class reference directly as the right argument (or first element of the right argument). For example, if you have a class called `Invoice`, you can create an instance of it by entering:

```
I←ⓘNEW Invoice
```

What is really happening here is that the symbol `Invoice` refers to the class definition, and when it is used in this way, it returns a *reference to the class*.

Note that you can also pass the class name rather than a class reference. The following are alternative ways of creating an instance of a user-defined class:

```
I←□NEW 'Invoice'
I←'apl' □NEW 'Invoice'
```

Why do we need this alternative syntax? The reason is that you can specify an external class name (i.e. a class written in a different language, and existing outside the workspace). For this case, the left argument specifies the external environment, and the right argument the class name. For example:

```
JAVADATE←'java' □NEW 'java.util.Date'
```

We won't discuss external classes any further here – that is a whole subject in itself!

Object References and Object Lifetimes

When you use □NEW to create a new object, that object persists until there are no more references to it in the workspace. It is then deleted immediately, if it is an internal or system object. If it is an external object, such as an instance of a .Net class, the fact that there are no more references to it in the APL workspace means that it is available for deletion by the external environment (unless the external environment itself has further references to the same object). However, in typical external environments such as .Net, Java and Ruby, the actual deletion of the object may not occur until later.

Consider this sequence, where we create an instance of an APLX class called *Philosopher* which has a property *Name*:

```
A←□NEW Philosopher
A.Name←'Aristotle'
```

At this point, we have created a new instance of the class, and we have a single reference to it, in the variable *A*. We now copy the *reference* (not the object itself) to a variable *B*:

```
B←A
B.Name
Aristotle
```

We now have two references to the same object. So if we change a property of the object, the change is visible through either reference – they refer to the same thing:

```

        B.Name←'Socrates'
        A.Name
Socrates

```

Now we erase one of the references:

```
)ERASE A
```

We still have a second reference to the object. The object will persist until we delete the last reference to it:

```

        B.Name
Socrates
)ERASE B

```

At this point, there are no more references to the object left in the workspace, and the object itself is deleted.

It follows from this that, if you use `NEW` to create an object, and do not assign the result to a variable, it will immediately be deleted again.

Using Classes without Instances

So far in this tutorial, we have concentrated on using objects as instances of classes. However, classes can also be very useful in their own right, without the need to make instances of them. There are two major reasons why you might want to define a class which can be used directly: defining constants, and keeping a namespace tidy.

Defining a set of constants

If you define a class with a set of read-only properties, those properties can be used as a set of constant values or 'enumerations'. For example, you might have a class called `Messages`, which holds all the messages which your application displays to the user:

```

Messages {
  OutOfMemory←'There is not enough memory to continue'
  AskModelName←'Enter the name of the model'
  OpComplete←'Operation Complete'
  AskReset←'Do you want to reset the model?'
  ...etc
}

```

You can then use this class in your application (without having to make an instance of it) to encapsulate all the messages and refer to them by name:

```
VR←CheckWS
[1] :If R←⊞WA<MIN_FREE_WS
[2] :ShowError Messages.OutOfMemory
[3] :EndIf
V
```

This keeps all the messages together in one place, allows you to refer to them by a name which is easy to remember and is self-documenting, but does not pollute the global symbol space with hundreds of APL variables.

Keeping namespaces tidy

In traditional APL systems, it often used to be the case that the number of global functions was very large. By placing related functions in a class, the workspace can be kept tidy, without having to resort to local or dynamic functions.

For example, in a statistical application, you might have a class *Average* which contained methods for calculating many different types of average (mean, median, mode, etc.). As long as these methods do not write to any property of the class, there is no need to make an instance of the class to run them; you can just run them using dot notation as *Average.Mean*, *Average.Median* etc.

Note that, in APLX classes, there is no pre-determined difference between a method which can only be run inside an instance (sometimes known as an *instance method*), and a method which can be run as a class member without an instance being created (sometimes known as a *static method*). The only difference is that, at run time, if a method writes to a property, an error will be generated if there is no instance to write to.

Finding out More

There is a lot more to APLX classes beyond this simple introduction. To find out more, take a look at the MicroAPL website and particularly the document "New Features in Version 4.0", which includes a longer version of this tutorial.

J-ottings 49:

Are you thinking what I'm thinking?

by Norman Thomson

"APL as a Tool of Thought" was the title of a long-running series of seminars held in New York, which, by a tiny change could be made into the claim "APL is a tool of thought". Generalise this into the statement "Some language is necessary as a tool of thought" and the resulting statement is what is known to linguists as the Sapir-Whorf Hypothesis (SWH). In short, you cannot think unless you have a language.

Whether this is true or not is open to question. Is it possible for an individual to think thoughts which are totally beyond his or her powers of articulation? Could there have been a caveman possessing all the scientific and cosmological vision of an Einstein or a Feynman, but who, having no language, was never able to communicate the fact? Proponents of SWH say no.

"APL is a tool of thought" can be paraphrased "APL is a labour-saving device for thinkers" in the same way that a Dyson is a labour-saving device for cleaners. Given that serious thinking is a hard, brain-taxing activity, "APL is a labour-saving device for thinkers" is probably not in dispute, and indeed the same could be said of any computer language. So what about the proposition "J is a better labour-saving device for thinkers", or more controversially still "APL (and J) enable people to think thoughts which were previously unattainable to them".

Computer languages in general differ from natural languages in the privacy of dialogue. Spoken human language competence is tested by a continual stream of reactions from receivers; if the latter respond in a puzzled or incomprehensible fashion, the speaker can often repeatedly retry, thereby learning by failure and feedback, one of the richest means of increasing language competence. By contrast, the response to failure with computer languages is normally an error message, impassive, and emotionally neutral.

One of the ways in which J is sharply differentiated from other computer languages, even APL, is in the relative infrequency of show-stopping error messages. Whereas erroneous input in other languages is likely to cause suspension of execution, as often as not J continues, but delivers an output which is often quite different from that expected. Put in another way, whereas a faulty expression in other languages leads to non-understanding, in J it is just as likely to

lead to misunderstanding. A possible response to an unexpected J output might be to shift a parenthesis a place or two, or to switch the odd conjunction, perhaps with the dialogue and re-submissions becoming more rapid as wishful optimism begins to overtake well-considered thought.

Any faulty input which did not deliver an error message is nevertheless the expression of a valid thought, failure to explore which is to forego a valuable opportunity for learning. Thus as well as working towards a correct expression, there is at least as much learning value in ascertaining what thought or intention would have resulted in the faulty input, which is after all the thought which would have been communicated to a more linguistically competent J receiver.

To be specific, consider a J dialogue in which I attempted to find all the combinations of x integers from $i..y$. My strategy, at first vaguely conceived in natural language, was first to obtain bit string patterns of all the natural numbers from 0 to $2y-1$, and then select those which contain exactly x bits, with the final step being to convert these bit strings into indices.

The first step is provided by a single J symbol. For the sake of definiteness, think in terms of combinations of 2 items out of 3.

```
]t=.#:#:i.8      NB. 8 is 2^3
0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1
```

or more generally

```
b=.#:#:i.@(2&^)  NB. 0,.._1+2^y as binary numbers
t=.b 3           NB. same t as before
```

There is no problem in calculating the number of bits in each of the $2y$ lists

```
(+/"1)t          NB. sum all the lists in t
0 1 1 2 1 2 2 3
```

My next thought was to select those lists whose bit-sum equals 2:

```
((2&=)@+/"1)t    NB. mark lists whose sum is 2
0 0 0 0 0 0 0 1
```

Something wrong! – what I should have thought was

```
(2&=@(+/"1))t NB. right this time?
0 0 0 1 0 1 1 0
```

Following a well-established route for selection using #~, the above thinking leads to

```
(#~2&=@(+/"1))t NB. (#~f)y means f(y) # y
0 1 1
1 0 1
1 1 0
```

From there the route to the summit is clear, but before proceeding, what about the wrong thought? Following the advice given earlier, I should, as a conscientious J learner, ask what it was which I conveyed to the hypothetical perfect J speaker/reader when I said ((2&=)@+/"1)t? In the first place such a reader would have mentally inserted a right parenthesis following the first verb to the left of @ so that what was inserted in the J sense was the verb $v = .(2&=)@+$. What does this convey? Any verb to the left of @ must be monadic, in this case *compare-with-2*. The fact that the insert adverb (/) follows v means that the + in v must be dyadic, and so v is *compare-with-2 addition*. Next since the inserted verb is defined as operating on lists ("1), the insertions are between individual items. Given that each list is a bit-list, the only possible results of adding neighbours are 0, 1 and 2. The last occurs only when both inputs are 1, or equivalently compare with 2 is true, and so in the present context v is equivalent to * with bits, that is *. (logical and). Here is confirmation:

```
v/"1 t
0 0 0 0 0 0 0 1
```

As an aside (and asides can be very valuable in language learning) suppose now that a second thought was to change $v = .2&=@+$ to $w = .2\&=(+)$. Only monadic verbs may appear to the left of @, and so = must now be self-classify, which returns the value 1 for all scalars. The next question is what does 2& mean? The definition of *bond* with a noun is

$$x \text{ m\&v } y \leftarrow \text{m\&v}^{\wedge}:x \ y$$

so $x \ 2\&=(+) y$ means $2\&=(+)^{\wedge}:x (y)$ in the present case. In words, the left argument is a repeat factor, so in the case of bits the verb =@+ is repeated either 0 or 1 times. If it is 0 the result is simply the right argument, if it is 1, the result is 1 by virtue of self-classify. The 2 in the bond is irrelevant, and could have been any number, real or complex.

Put all this together and dyadic w is equivalent to or :

```
0 w 1
1
```

Or is it? There is a snag, namely that the rank of the result of self-classify is a table comparing an object with its nub, and so has rank at least 2, so unlike $v/"1\ t$, $w/"1\ t$ is not a list of simple binary scalars:

```
(0 w 0),(0 w 1),(1 w 0),(1 w 1)
0
1
1
1
1
$w/"1 t
8 1 1 1 1
```

The 8 above is easy to explain, but why four 1s when each item of t is a 3-list? The answer is that each insertion after the first adds 1 to the rank following the self-classify rule. This is confirmed by

```
$w/"1 b 4
16 1 1 1 1 1 1
```

To return to the correct route, or call it perhaps the clear-thinking path, the experience of the exploration of wrong avenues suggests separating out the list summation process

```
s=. +/"1      NB. is x equal to the list sums of y
3 s i.2 3      NB. sum two lists, return 1 if '=3'
1 0
```

The right argument in the context of combinations is $b\ y$ and so the indicator list which identifies the relevant binary lists is the hook $(s\ b)y$.

Hence

```
d=.(s b) # b@]      NB. items of y which sum to x
```

delivers the bit strings which corresponds to the required combinations:

```
2 d 3
0 1 1
1 0 1
1 1 0
```

and a final tally using bits selects the relevant lists of indices

```

c=.d # i.@]    NB. convert binary to indices
2 c 3
1 2
0 2
0 1

```

Here is the final (and correct!) stream of thought which led to this approach to the problem :

```

b=.#:@i.@(2&^)  NB. i.2^y as binary numbers
s=.= +/"1       NB. x is equal to list sums of y
d=.(s b) # b@]  NB. items of y which sum to x
c=.d # i.@]     NB. convert binary to indices

```

But to return to the beginning, the primary object of this article was not the final algorithm itself, but rather reflection on the journey to get there, and on the value of following up wrong paths taken on the way. If you have read this article, you will almost assent to the propositions that APL and J are labour-saving devices for thinkers. But more subtly:

1. are there people for whom both APL and J allow the expression of thoughts which they were previously able to think but not express?; and
2. are there people for whom both APL and J allow the thinking of thoughts which they were previously unable to think?

Using the .Net DataSet Class with Dyalog 12

by Adrian Smith (adrian@apl385.com)

Acknowledgements

This material was prepared initially for Paul Mansour's wonderful moot in San Quirico, and was improved during the presentation by several of those present, notably Stefano. Thanks to everyone who mucked in to help.

Motivation

One of the wonders of .Net is just how much stuff you get for nothing. One of the little gifts it gives you is a perfectly usable relational database, which they would love to have you backend onto SQL Server, but which is quite adequate running completely stand-alone. I rather like playing with databases, so I spent half a day fooling with this, partly as a way to explore the Dyalog .Net interface. I quickly discovered that wrapping one of Dyalog's new classes around it could add quite a bit of fun to the experience, but it also became obvious that handling anything more than a few thousand rows would be amazingly slow unless all the data types were directly mapped into simple APL arrays. As soon as you hit *nulls* or (worse) *dates* APL converts each item into a namespace reference and things slow exponentially.

So this takes on from my note on new tricks for old dogs [1], and shows how you might actually make use of this stuff in a real working system, at a very reasonable speed. Nervous readers should note that there is some C# code involved, but it is very low on intellectual content, and consists mostly of all the loops that we don't want to see in APL.

Getting Started with the DataSet Class

I am taking it as read that anyone who has read this far knows roughly what a database is, so I will run quickly through how .Net handles the idea of tables and relationships. We will build the little EmpDept example (with random variations) that Oracle shipped back in the 70s.

The very first thing you need is a `DataSet` which is a wrapper for your `DataTables` and the `Relationships` between them:

```
ⓂUSING←' ' System.Data,System.Data.OleDb ' Causeway,D:\tools\apic\dbs.dll'
pr←ⓂNEW DataSet(←'Payroll')
```

There is very little a `DataSet` can do on its own, but if you have a quick look at its methods, you will see something rather useful:

```
prps←{ω.ⓂNL -2}
meths←{{ω/Ⓜ~'-' ε''ω}ω.ⓂNL -3}

meths pr
... GetXml ... WriteXml ...
```

When you are done, you can save your entire database to disk in a (remarkably) clean XML format which you can then rebuild later on. For small, self-contained projects, this seems a very good solution, as well as being completely free. By small, I mean anything up to a few hundred thousand rows.

Of course we need some tables, and the tables need some columns. The .Net SDK help file is really well written, and walks you through the process at a good pace, if you want to explore this more deeply. For now I just want to get something I can play with in APL, so here we go:

```
A The Emp table will need some columns, with type information
A Start by collecting the datatypes we will need
int32←System.Type.GetType(←'System.Int32')
double←System.Type.GetType(←'System.Double')
datetime←System.Type.GetType(←'System.DateTime')
timespan←System.Type.GetType(←'System.TimeSpan')

empid←et.Columns.Add(←'Id')
empdept←et.Columns.Add(←'Dept')
empname←et.Columns.Add(←'Name')
sal←et.Columns.Add('Salary' int32)
sink←et.Columns.Add('Annual Bonus' double'Salary / 10')
join←et.Columns.Add('Joining Date' datetime)
service←et.Columns.Add('Length of Service' timespan)
```

A couple of things to note in passing – getting a `Type` object in APL is strangely messy. There may be a better way of doing this, but I haven't found it yet. Also note how bonus is set to `salary/10`, in the definition of the table. I don't really see the use of this in a real system, but it is quite surprising that you can do simple arithmetic directly in the table definitions. Finally, under *coding habits* I suggest you file away the idea of always using a jot against methods where you have to

enclose the 'string' argument. This is redundant here, but makes it visually obvious that you are not really passing an enclosed array – just flagging it for enclosure by the called function. Also it will allow you to use *each* on an argument array without swearing. Another good habit is to set up *true* and *false* as standard globals to make your APL code look a little more like the examples in the help files.

Now we hit a small brick wall and get the first taste of some C# assistance:

```
A We need this as APL cannot pass an array of DataColumnns here?
Db.SetPrimaryKey(et,1)      A Static call on the library
A This has one primary key which cannot have duplicates
empid.(AllowDBNull Unique)+false true
A Just to show we can store custom properties (string only)
et.ExtendedProperties.Add'Owner' 'Adrian'
A We must have a deptid
empdept.AllowDBNull+false
```

In order to set the primary key, I need to pass an array of DataColumn objects. I don't think there is any way for Dyalog to do this (please prove me wrong, someone) so I dived over to TextPad and wrote a trivial bit of C# to do the job, given the column index or indices:

```
// Much more comfortable just to pass column numbers when creating tables!
public static void SetPrimaryKey(DataTable table, int col)
{ SetPrimaryKey(table, new int[] {col}); }
public static void SetPrimaryKey(DataTable table, int[] cols)
{ DataColumn[] keys = new DataColumn[cols.Length];
  for(int ct=0; ct<cols.Length; ct++)
    keys[ct] = table.Columns[cols[ct]-1];
  table.PrimaryKey = keys;
}
```

This code is included in the source for dbs.dll, along with a lot of other stuff I was expecting to have to write. One day it should become redundant.

Of course the department table will just be more of the same:

```
deptid+dp.Columns.Add+c'DeptId'
sink+dp.Columns.Add+c'Department'
deptid.(AllowDBNull Unique)+false true
Db.SetPrimaryKey(dp,1)
```

Finally, we draw a mental line between the tables (a real line if you always sketch these before you start typing) to enforce the link between them:

```
A Finally link them up to enforce the foreign-key relationship
  aff+ds.Relations.Add 'WorksIn' deptid empdept  A Parent->child
  aff.ChildKeyConstraint.DeleteRule+Rule.Cascade A Default
```

This means that anything you type in the *Dept* column of the *Emp* table *must* match an existing entry in the *DeptId* column of the *Dept* table. (And if you delete a department, then all the employees conveniently evaporate. This might not be the best option in production code ...)

As the final step, let's just add a few random departments and employees:

```
□ USING->' 'System'
A Depts must be loaded first
dept+ds.Tables[1]
nr+dept.NewRow
nr[0]+<'OR' ◊ nr[1]+<'Operational Research'
dept.Rows.Add nr
nr+dept.NewRow
nr[0]+<'GD' ◊ nr[1]+<'General Dogsbody'
dept.Rows.Add nr
A Now the employees
emp+ds.Tables[0]
nr+emp.NewRow
nr[0]+<'JM'
nr[1]+<'OR'
nr[2]+<'Jonathan'
nr[3]+24800
nr[5]+DateTime.Parse<<'23/3/1998'
nr[6]+TimeSpan.FromDays 678
emp.Rows.Add nr
...
A Now lets try the bulk insertion method
emp.BeginLoadData
sink+emp.LoadDataRow('MWG' 'OR' 'Marc' 18500 0)true
sink+emp.LoadDataRow('GMS' 'GD' 'Gill' 13503 0)true
sink+emp.LoadDataRow('RCS' 'GD' 'Richard')true  A Default sal
sink+emp.LoadDataRow('WHO' 'GD')true  A Default name to DBNull
emp.EndLoadData
```

Note that I bottled out on using the bulk method with *DateTime* and just allowed our last four employees to have null values here. Bulk insert is much better done with a bit more C# help, so we can walk on by and play with the limited data we have.

Playing with Selections

We can do a fair amount of processing in plain APL now, starting from the `pr` object (the *DataSet* instance we made at the beginning) and digging down layer by layer:

```
pr
System.Data.DataSet
  pr.DataSetName
Payroll
  pr.Tables
System.Data.DataTableCollection
  []pr.Tables
Emp Dept
```

Note the excellent monadic *squad* `[]`, which converts a collection into an array. We APLers like arrays, just like we Pirates always speaks in the present tense (sorry). On we go ...

```
emp←f[]pr.Tables           A Just the employee list
ss←emp.Select<'salary>20000' A What have we here?
ss
System.Data.DataRow  System.Data.DataRow
  ↳ss.ItemArray
JM   OR Jonathan  24800 2480  23/03/1998 00:00:00
678.00:00:00
ACDS OR Adrian   36800 3680  30/09/1974 00:00:00
142.21:21:18.9000000
  +/4↳"ss.ItemArray
61600
```

The good news is that you can get your data back out again, selected and ordered. The bad news is that you get it back in exactly the way you didn't want – as an array of *DataRow* objects. APL will run through these for you grabbing each row as a nested array, but note that each item in the *DateTime* column is actually a *namespace reference* so if we have a few thousand employees, this innocent expression is going to run up the fan on your laptop for quite a while. Several years in fact, for a not-unreasonable sample.

A more pressing problem is that none of the things an APLer wants to do (like find the median salary) are easy to achieve with this model. Time to step back and have a little think.

What do we *really* want here?

Clearly, we want arrays, and we want 'em *now*. It would be really great if the selection could be an instance of a Dyalog class, with all the columns as properties. Something like:

```
sel←Select emp 'salary>20000'    A Get a selection
      sel.Salary
24800 36800
      sel.Salary[2]
36800
      sel.Salary[2]*+1.05
      sel.Salary
24800 38640
```

This way a column in a selection behaves much like an external variable – we can index it and use any reasonable selective specification to update it. The other things we need are a good strategy for replacing [null] with a chosen numeric value, and a good way to represent DateTime objects and TimeSpan objects:

```
      sel.ColumnNames
Id Dept Name Salary Annual Bonus Joining Date
      Length of Service
      sel.JoiningDate
35877 27302
      sel.LengthofService
58579200 12345678.9
```

I chose to form the property names by compressing out blanks from the column names, and to represent the dates as the traditional ordinal values. TimeSpans are trickier, as these are 64-bit integers in .Net and my APL cannot handle these just yet. I chose decimal seconds as the least bad option.

If we reselect the whole table and just grab everything, you will see my default null substitutions:

```
sel←Select emp ♦ sel.GetMatrix    A Grab the whole table
ACDS OR Adrian      38640 3864 27302 12345678.9
GMS  OR Gill        13503 1350.3 -1 0
JM   OR Jonathan    24800 2480 35877 58579200
MWG  OR Marc        18500 1850 -1 0
RCS  FD Richard     -32768 -32768 -1 0
WHO  OR (null)      -32768 -32768 -1 0
```

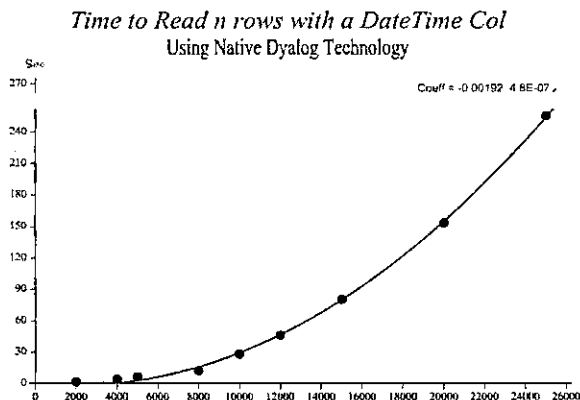
... but we can change this with another handy property:

```
sel.NullFillers[4 5]←0      A Change how nulls appear
sel.GetMatrix              A NB different nulls
ACDS OR Adrian 38640 3864 27302 12345678.9
GMS OR Gill 13503 1350.3 -1 0
JM OR Jonathan 24800 2480 35877 58579200
MWG OR Marc 18500 1850 -1 0
RCS FD Richard 0 0 -1 0
WHO OR (null) 0 0 -1 0
```

Of course the [null] stuff is fully reversible – setting an element to the current filler for that column results in a [null] in the table. The only remaining issue to illustrate is the speedup we can get by farming some of the grunt-work out to C# – I made a simple clone of a typical 'ticker' datafeed to give the engine enough data to chew on:

```
LoadTicker 1
sel←Select tk A To play with it
sel.ColumnNames
What When Value
sel.Rows
40000
+/sel.Value
200499.74
```

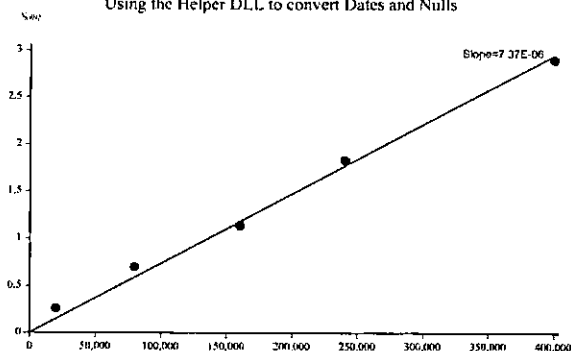
This is pretty well instantaneous after the brief delay while the table is populated for the first time. For a rough guide to the alternative, I offer the following chart:



If you compute the regression equation for 40,000 rows you will get $4.3e^{-7} \times 40000 \times 2$ giving 688s or a little over 11 minutes just to create the array of rows. Probably you will get a Syserror and your interpreter will exit stage left (pursued by a bear) long before the expression completes. For comparison:

Time to Read n rows with a DateTime Col

Using the Helper DLL to convert Dates and Nulls



This one has the added benefit of being linear rather than quadratic – note the x-axis had to be extended by a factor of about 20 to get meaningful times. That will do for showing you around the examples – what we need to do now is to show how the trick is pulled.

Farming out the Loops

Whatever you say about conventional languages, they are good at doing simple things quickly. Here is the code that sits at the back of the column reference:

```
public Array GetCol(int cinx)
// Grab whole Column (ensure APL gets the right datatype)
{ int col = cinx-1; // Fix for 1-origin
  Type coltype = APLTypes[col];
  Array coldata = Array.CreateInstance(coltype, Selection.Length);
  for(int row=0; row<coldata.Length; row++)
    coldata.SetValue(GetItem(row, col), row);
  return coldata;
}
```

The full C# source code can be downloaded [2] if you want to read all the preamble. The DLL maintains an array of data-types which APL has for the various columns, and starts by creating an array of the right type and size. It then

simply hammers round the elements in the column populating the array, which is returned to APL. This calls the lower-level function:

```
private object GetItem(int row, int col)
// Return one cell, checking for type and null
{ object cellvalue = Selection[row][col];
  if(System.DBNull.Value == cellvalue) // APL really hates these
    cellvalue = nullFillers[col]; // Plug in current filler instead
  else if(isDate[col])
    cellvalue = ((DateTime)cellvalue).ToOADate();
  else if(isSpan[col])
    cellvalue = {(TimeSpan)cellvalue}.TotalSeconds;
  return cellvalue;
}
```

After checking whether the data item is `null`, it special-cases the two complex types that can upset APL and converts the cell content to something that can be returned as a simple numeric value. In the common case where APL can cope, this is just a fall-through which returns the right element from the selection. The code to update the selection when you assign something to the APL property is (if anything) even simpler:

```
public void PutCol(int cinx, object[] newValues)
// Update data colwise (always an entire column)
{ int col = cinx-1;
  for(int row = 0; row<Selection.Length; row++)
    PutItem(row,col,newValues[row]);
}
```

This assumes that the APL side returns an array of the right length, which is always the case if you update a property with indexing or selective specification. It just does the reverse conversions:

```
private void PutItem(int row, int col, object newValue) // Fix one cell
{ Selection[row][col] = Dyalog2Data(col,newValue); }
private object Dyalog2Data(int col, object dataitem)
// APL types --> Database types
{ Type coltype = APLTypes[col];
  object newValue = Convert.ChangeType(dataitem,coltype);
  if (NullFillers[col].Equals(newValue))
    newValue = System.DBNull.Value;
  else if(isDate[col])
    newValue = DateTime.FromOADate((double)newValue);
  else if (isSpan[col])
    newValue = TimeSpan.FromSeconds((double)newValue);
  return newValue;
}
```

I think that shows you enough of the back-end processing to convince you that there really is no rocket science or magic going on. Now we need to hook up the Dyalog class mechanism to the other end of the process to make it all really developer-friendly.

Making a Nameless Class on the Fly

It is a real benefit of working in a dynamic language that you can lay the floor as you walk across it. Old hands are used to the trick of generating functions on the fly (often locally), fixing them and running them. Now we can do the same with classes – by fixing an unnamed class from a localised script, we can return an instance of something which never existed in the workspace at all. This is what happens when you type:

```
sel+Select emp 'Dept='OR'' 'Salary'
sel
Selection from Emp where Dept='OR' (5 rows selected)
sel[3]      A Indexer gives us the 3rd row
ACDS OR Adrian 1000 100 27302 12345678.9
```

The *display form* of the class has been set to return a nice description of the selection, just to make it all look like the SQLPLUS command prompt I remember from a previous life. The `Select` function takes a reference to a table and optionally a selection expression (in SQL) and one or more column names to use as sort keys. It calls the C# code in the constructor for the db helper, passing across the table and the selections. Note that we can localise DB here:

```
□USING+'' 'Causeway,D:\tools\aplc\db\dll' A Adjust to taste!
DB+□NEW Db table
colnames+DB.ColumnNames~'' ' ♦ writeable+DB.Writeable
scr+':Class' ':Access Public' ''
scr,+ ' Create arg' ':Access Public' ':Implements Constructor'
scr,+ ' DB+arg' ' □DF DB.ToString' 'v '
```

If I temporarily de-localise the script, we can see part of what it wrote for us (indexers are slightly special – we need a *shape* function so Dyalog knows when to call INDEX ERROR on us):

```
:Property Numbered Default DataRows
:Access Public
v value+shape
value+DB.Rows
v
v value+get arg
value+<DB.GetRow arg.Indexers
v
v set arg
DB.PutRow arg.Indexers (targ.NewValue)
v
:EndProperty
```

```

▽ value←GetMatrix
:Access Public
  value←DB.GetMatrix
▽

```

By setting an indexer as the default property, we can treat the entire selection as a simple vector, so fetching/updating it by row is trivial. There are several other pass-through calls like `GetMatrix`, and then we come to the only really dynamic part. Because the `Select` function has the list of (de-spaced) column names, it can easily make a property definition for every column:

```

:For ct :In itpcolnames
  A Expose all columns read-write unless marked readonly
  pname←ct>colnames
  scr,+'(' :Property ',pname)':Access Public'
  :If ct>writeable
    scr,+'▽ set arg'(' DB.PutCol ',(⌘ct),' (arg.NewValue)')'▽ '
  :End
  scr,+'▽ value←get'(' value←DB.GetCol ',⌘ct)'▽ '
  scr,+' :EndProperty'
:End

```

A typical (generated) entry looks like:

```

:Property Salary
:Access Public
▽ set arg
  DB.PutCol 4 (arg.NewValue)
▽
▽ value←get
  value←DB.GetCol 4
▽
:EndProperty

```

You may want to add some `:Trap` blocks, as you may get a crash in the C# on updating the data, and not want to get hung up in Dyalog's generated setter function, for example:

```

      sel.Salary+1 2
EXCEPTION
set_Salary[1] DB.PutCol 4(arg.NewValue)
      ^
    )si
[#.[[Unnamed]]] #.[Namespace].set_Salary[1]*

```

Here is a very minimal approach which improves things quite a bit:

```
▽ set arg
:Trap 90
  DB.PutCol 3 (arg.NewValue)
:else
  72 □signal~□exception.Message
:EndTrap
▽
```

This gets us back out to the session with a (generally quite helpful) message from the Microsoft people:

```
      sel.Salary←3 4
Index was outside the bounds of the array.
      sel.Salary←3 4
      ^
      (2>sel.Dept)←'ZZ'
ForeignKeyConstraint WorksIn requires the child key values (ZZ)
to exist in the parent table.
      (2>sel.Dept)←'ZZ'
      ^
```

A little more validation on the APL side of the fence would do no harm here – we have just trashed the salaries of the first two employees!

Having made the entire script, the final act is to make a transient class, and return an instance of it:

```
scr←''':EndClass'
sel←□NEW(□FIX scr)DB
```

As I said before, both `scr` and `DB` can be locals, so this is remarkably tidy in its use of the workspace. As a final bit of fun, let's whistle up a `.Net DataGrid` and point it at our table. If you download the workspace and try this out, you will see that as you type expressions in the APL session, the grid follows the changes automatically.

```
□USING←'System.Collections' 'System'
□USING←'System.Windows.Forms,System.Windows.Forms.dll'
f←□NEW Form
f.Text←'Hello, world'
f.Visible←1
emp←pr.Tables[0]
ev←emp.DefaultView
ev.AllowNew←false
```

```

dg+=NEW DataGrid
dg.(Top Left)+4 4
dg.(Width Height)+f.(Width Height)-4 4
dg.Anchor+=AnchorStyles.(Bottom+Top+Left+Right)
dg.SetDataBinding ev'' A Single table view
A dg.SetDataBinding pr'Dept' A or the drill-down variety

f.Controls.Add dg

```

	Id	Dept	Name	Salary	Annual Bonus	Joining Date	Length of Ser
▶	JM	OR	Jonathan	24900	2480	23/03/1998	678.00.00.00
	ACDS	OR	Adrian	36800	3680	30/09/1974	142.21.21.18
	MWG	OR	Marc	18500	1850	(null)	(null)
	GMS	OR	Gill	13503	1350.3	(null)	(null)
	RCS	FD	Richard	(null)	(null)	(null)	(null)
	WHO	OR	(null)	(null)	(null)	(null)	(null)

Display and editing in a .Net DataGrid

Or, of course, you can just use it as a handy editor for the data!

Wrap up

In summary, there is a very competent relational database sitting on your computer, and it is totally free. To save your work on the nearest hard-drive, all you need to do is:

```

pr.WriteXml 'C:\temp\payroll.xml' false A Save (with Schema)
emp.WriteXml<'c:\temp\emp.xml' A Save table (data only)

```

The XML you get is designed to be readable, and indeed editable, for example:

```

<Payroll>
  <Emp>
    <Id>JM</Id>
    <Dept>OR</Dept>
    <Name>Jonathan</Name>
    <Salary>24800</Salary>
    <Annual_x0020_Bonus>2480</Annual_x0020_Bonus>
    <Joining_x0020_Date>1998-03-23T00:00:00+00:00</Joining_x0020_Date>
    <Length_x0020_of_x0020_Service>P678D</Length_x0020_of_x0020_Service>
  </Emp>
... and so on

```

If I make a trivial change (adding Marc's middle initial) in Notepad, then start from cold:

```

    USING+'' 'System.Data,System.Data.DLL'
    pr=NEW DataSet(<'Payroll')
    pr
System.Data.DataSet
    pr.ReadXml(<'C:\temp\payroll.xml')
ReadSchema
    pr.Tables
    Emp Dept
    emp+pr.Tables A Recover the employee list
    sel+Select emp A Get a selection
    sel.GetMatrix
ACDS OR Adrian 36800 3680 27302 12345678.9
GMS GD Gill 13503 1350.3 -1 0
JM OR Jonathan 24800 2480 35877 58579200
MWG OR Marc W 18500 1850 -1 0
RCS GD Richard -32768 -32768 -1 0
WHO GD (null) -32768 -32768 -1 0

```

With Dyalog classes, you can put a wrapper around it that makes columns in selections behave almost like variables in the workspace. With a little help from a C# DLL, it can be made to run quite fast enough to support a realistic application. You can download the Dyalog 12 (Unicode) workspace and supporting C# source code (compilation instructions included)[2] and use it as you wish. Please let me know of any improvements you make to either side.

References

1. "New Tricks for Old Dogs: Making Sense of Classes and Namespaces", Smith, A., *Vector* 23(1&2) p113-117
<http://www.vector.org.uk/archive/v231/smith.htm>
2. Source code for this article (Dyalog 12 Unicode workspace and C#):
<http://www.vector.org.uk/archive/v233/smithdat/dbs.zip>

SALT: a Simple APL Library Toolkit

by Dan Baronet (dan@dyalog.com)

The ability to write readable APL code out to text files and read it back has been pretty much out of reach for mere mortals until recently. Reasons for wanting to do such a thing abound:

- Keeping code out of the workspace to conserve memory
- Sharing or sending copies of code to people who don't have an interpreter
- Storing readable code outside APL
- wanting a copy of the code independently of having an interpreter or being bound to a version of it

With the advent of Unicode it is now easier to achieve this. APL characters are represented in Unicode, and editors and viewers of Unicode text files are now commonplace.

Using Script Files in Dyalog APL

Since Version 11 it has been possible to get the source of some namespaces and the (then) new classes/interfaces. If desired, this source could then be written to file in Unicode format using the `UnicodeFile` class provided in `⎕SE`. This allowed one to create text files that could then be read or edited with programs such as Notepad, and keep the source out of the workspace:

```
F←⎕NEW UnicodeFile
F.Write 'MyFile.txt' text
```

With Version 12 the `UnicodeFile` class is no longer needed as the new `⎕UCS` system function can now create UTF-8 strings directly from the Unicode points (APL text/code). This stream can then be written out to file just like any other APL array. Reading back is also easy: read the stream, use `⎕UCS` to reconstruct the string.

```
(⎕UCS 'UTF-8' ⎕UCS text) ⎕NAPPEND tie      A write to file
⎕UCS ⎕NREAD tie 80,⎕NSIZE tie              A read from file
```

Anybody can write two functions to write and read, put them in `□SE` and have the ability to store and retrieve APL code from native Unicode text files anytime they want to.

Of course, once you start doing something like this you realise it might be a good idea to save multiple versions of a file or script, to be able to retrieve any of those versions, list them, compare them, remove unused versions, and so on.

And this is exactly what Dyalog did when we came up with SALT: we wrote two functions, `Load` and `Save`, and added a few more functions to look up directories and files. We added the ability to store multiple copies of the same file and a function to be able to compare them, even by using your favourite comparison program.

We went a step further, and added the ability to start APL without a workspace, simply by loading scripts and executing a statement to start the process.

We modified the editor to generate an event upon fixing an object thus opening the door to intercepting the fix and saving the modified file automatically.

We wrote a few scripts, besides SALT (itself a script) and made them available.

And that was pretty much all we did.

SALT comes bundled with Dyalog and is enabled by default. You can disable it through the configuration menu or through the accompanying SALT workspace. Simply `)LOAD SALT` and use `enableSALT` or `disableSALT`.

Usage

SALT functions use a consistent syntax: they all take a string argument which describes the arguments to the action to perform. They make use of switches à la Unix (e.g. `-version`). All script files have the extension `.dya log`. If a relative path is used they are stored under a default directory, which may be changed.

Here is a list of the main SALT functions: (brackets denote optional switches).

```
Save 'object \path\to\file [-version=x]'
```

Saves object in the file specified, with extension `.dya log`. If `-version` is specified the filename will include the version number. Once version numbers are in use there is no further need to specify the version; SALT will attribute a new version number each time an object is re-saved.

If a relative path is supplied SALT will store files under the default SALT path. This default is `\dyalog\installation\path\SALT` but it may be changed using the `Settings` function:

```
Settings 'workdir'    \new\path'  
Load    '\path\to\file' [-version=x]'
```

Loads the latest version or the version specified.

```
List    '\folder\or\file' [-recursive]'
```

Lists the scripts in a folder, recursively if `-recursive` is given.

```
Compare 'script1' [script2]'
```

Compares two scripts or the last two versions of a script if only one is supplied as argument.

There's More

There are other features not listed above: you can:

- skip the confirmation prompts in `Save`
- specify where to define the objects
- disperse their contents instead keeping them in their original namespace
- get the source instead of defining the object
- list the entire tree of a path
- clean the versions
- use external programs to compare objects, etc.

And it works as well under Unix or Linux.

If this is interesting, SALT comes with complete documentation which can also be found on Dyalog's website. Have a look!

In Session “Under the Table”

by Stephen Taylor (stj@dyalog.com)

APL is famously good at describing things using tables, and tables are an obvious way to represent the contents of a spreadsheet or the results of an SQL query. But tables don't have to grow very wide before we face a writing problem. We commonly want to select rows based on expressions that refer to columns. How to refer to columns clearly?

'Cannon's Canon' [1] – no numeric constants in the code – steers us away from referring to them by number like this:

```
tbl←0,~'Orders'DB.FindM('Item' 'Price' 'Quantity')('')
tbl[;4]←×/tbl[;2 3]
html,←'TABLE'tag'{TR'tag'TD'∘tag∘*''ω}/tbl[;1 4]
```

(DB.FindM queries a database table *Orders* and tag marks up character vectors with HTML tags). We can improve on this a little with local names for column indexes:

```
ITEM PRIC QUANT AMNT←i2∘ptbl
tbl[;AMNT]←×/tbl[;PRIC QUANT]
html,←'TABLE'tag'{TR'tag'TD'∘tag∘*''ω}/tbl[;ITEM AMNT]
```

We can write something closer to the simplicity of our first textbook by representing the table as a namespace in which named vectors represent the columns:

```
tbl←'Item Price Quantity'DB.Tbl'Orders'
tbl.(Amount←Price×Quantity)
html,←'TABLE'tag'{TR'tag'TD'∘tag∘*''ω}/⊆tbl.(Item Amount)
```

Dot notation gives us even better opportunities when we come to tables with matching column names. Consider a table *CON* of contact information from a spreadsheet. We'll use some of the records to update contacts already in our database. The others we'll add as new contacts. So the first question is: which records in *CON* represent existing contacts? Our first step is to match email addresses, record the corresponding *ContactID* and write a bitmask to show which records were not matched.

```
C←'ContactID Email CompanyID Name'#.DB.Tbl'Contacts'
no←0=CON.ContactID+(C.ContactID,0)[>1/(C.CON).Email]
```

Note

- use of `t /` with dot notation to look up a column from one table in its homonym in the other;
- the clarity of setting and testing for the 'null' value 0 in the same line.

But people use multiple email addresses, so we'll test the unmatched records to see if we can recognise any others. Here we'll look for matches on *both* CompanyID and Name.

```
(no/CON.ContactID)+ (C.ContactID,0)
... [ >t /1 no/" + @ + "(" C CON).(CompanyID Name) ]
```

Note

- use of dot notation to specify the CompanyID and Name columns from *both* tables;
- use of `+ @ + "("` (*zip each*) to convert each pair of vectors into a vector of pairs;
- use of `1 no/"` to mask only the CON pairs;
- use of `(no/CON.ContactID)+` to insert just the matched contact IDs, if any.

This is clearly not beginner's writing, but the single line is easier to analyse and maintain than the same operations dispersed across several lines and temporary variables.

References

- [1] "Three Principles of Coding Clarity" Taylor, S. *Vector* 18(4) pp113-124
<http://www.vector.org.uk/archive/v184/clar184.htm>

PROFIT

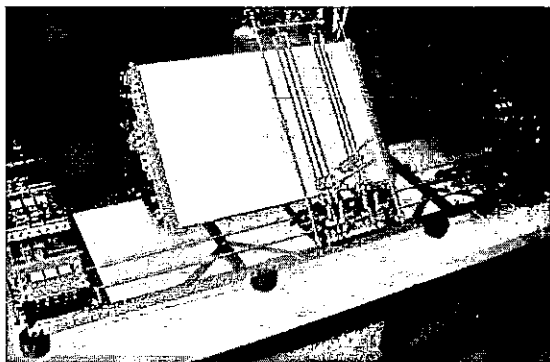
A J Simulator for a Meccano Differential Analyzer

by Keith Smillie (smillie@cs.ualberta.ca)

Introduction

The first differential analyzer to be constructed mostly of Meccano parts was built in 1933 at the University of Manchester by Professor Douglas Hartree assisted by his research student Arthur Porter about two years after the first differential analyzer was built by Vannevar Bush at the Massachusetts Institute of Technology. With the successful completion of the Meccano model funds were obtained for the construction of a full-scale machine which was built commercially. During World War II this machine and several similar ones were used for military purposes. At present there is considerable interest in the construction of Meccano models and their use in education.

The pioneering Manchester work is described in Porter (2003). Two papers, Robinson (2005a) and Robinson (2005b) give, respectively, a history of the Meccano models and a description of the contemporary construction of one such machine.



Tim Robinson's Differential Analyzer

This paper describes a five-integrator simulator implemented in J and gives two examples of its use. The script, which will run on Versions 406 and 504 of J, may be downloaded from www.vector.org.uk/archive/v233/smillie/da406.ijs.

Integrators

Integration with a differential analyzer is performed by one or more integrators, identical in design and differing only in their interconnections with other components of the machine. As shown in the figure below an integrator consists of a small vertical wheel which turns on a horizontal axis as it rolls on the surface of a horizontal disk which rotates on a vertical axis. The displacement of the wheel from the centre of the disk varies as the disk rotates and is proportional to the value of the dependent variable. The rotation of the disk gives the change in the independent variable.

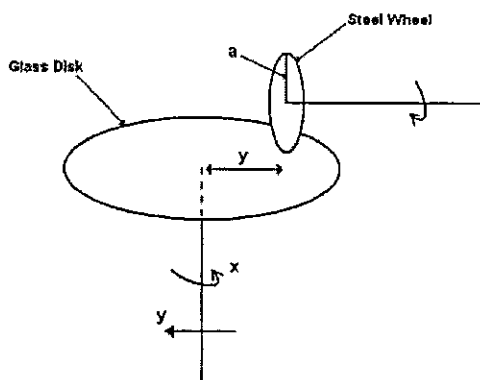


Figure 1: Integrator

Suppose that the disk rotates through a small angle Δx and the wheel is at a radial distance y from the centre of the disk. Therefore the wheel turns an amount

$$\Delta z = (y\Delta x)/a$$

where a is the radius of the wheel. Thus after a given period of operation the total rotation of the wheel is proportional to $\int y dx$. Since the wheel rests lightly on the disk so that it may slide freely along a radius as the value of the dependent variable changes, its torque must be amplified before its motion may be transmitted to another part of the analyzer. As this simulation models only the logic of the differential analyzer, the design and construction of torque amplifiers may be ignored although they are an important feature in the Meccano models.

Let us assume now that the disk rotates in discrete steps of size Δx and that the rotation of the disk begins when x has the value x_0 . Also assume that the initial radial displacement of the wheel on the disk is y_0 and that subsequent displacements are $y_1, y_2, \dots$. Then the successive rotations of the wheel are $(\Delta x)y_1/a, (\Delta x)y_2/a, \dots$. Therefore, the total rotation of the wheel after n steps is proportional to the sum

$$(\Delta x)y_0 + (\Delta x)y_1 + (\Delta x)y_2 + \dots + (\Delta x)y_n$$

As Δx becomes smaller, this sum will correspond more closely to the rotations of the physical disk and wheel, and thus become a better approximation to the value of the desired integral.

Simulator

The simulator has five integrators and eleven buses. The first bus corresponds to the independent variable in the differential equation, the next five buses to the five integrators, and the remaining five to auxiliary (non-integrating) buses. The buses are numbered 0, 1, 2 ... 10, and the values representing their cumulative rotations by the variables $z_0, z_1, z_2, \dots, z_{10}$. (The variable z_0 has the synonyms x and t .) The connections of the integrators with the buses are defined by dyadic functions, one for each integrator, and named I, II, III, IV and V. The left argument of a function gives the number of the bus associated with the rotation of the horizontal disk for the corresponding integrator, the right argument gives the displacement of the vertical wheel for that integrator, and the result the total rotation of the wheel which gives an approximation to the desired integral up to that stage of the integration. Successive pairs of values of the independent and dependent variables are stored as the rows of a two-column table associated with the integrator, and named TABLE1, TABLE2, TABLE3, TABLE4 and TABLE5.

Simple Harmonic Motion

In this section we shall give an example of the use of the simulator to solve the second-order ordinary differential equation for simple harmonic motion

$$d^2y/dx^2 = -y$$

with the initial conditions $y(0) = 0$ and $dy/dx(0) = 1$ which has the solution

$$y = \sin x$$

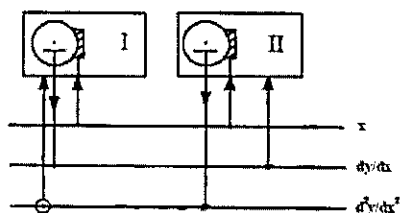


Figure 2: Simple harmonic motion

The connections to the buses of the two integrators required to solve this equation are shown in the schematic to the right. In the first integrator the right vertical line indicates that the first bus corresponds to the independent variable which controls the rotation of the horizontal disk, and the left vertical line that the third bus corresponds to the dependent variable given by the displacement of the vertical wheel on the disk. The middle vertical line indicates that the output of the integrator, i.e., the rotation of the wheel, controls the rotation of the second bus. (The small circle at the lower end of the connection with the third bus indicates that the value passed to the integrator is negated.) Similar statements may be made regarding the connections of the second integrator to the buses.

From this schematic we may derive the definitions

$$z1 =: 0 \text{ I } -z2$$

and

$$z2 =: 0 \text{ II } z1$$

for the integrators.

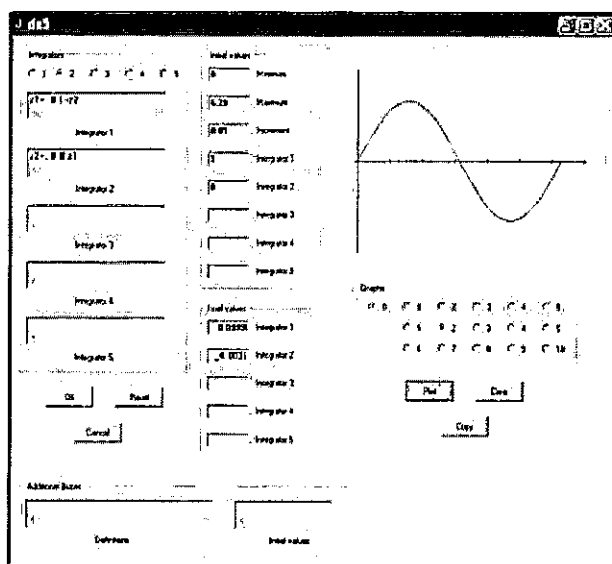


Figure 3: Simulator solution of the equation

Figure 3 shows the simulator solution of this equation. Note that we have plotted the solution against the independent variable x by selecting radio button 0 in the first row and radio button 2 in the second row in the box below the graph. This may be done either initially and then clicking the OK button to run the simulation, or after the simulation has been done and then clicking the Plot button. (The default selection of the radio buttons is 0 in the first row and 1 in the second which in this example will give the graph of the cosine function.) Also by selecting either radio buttons 1 and 2 in the first and second rows, or radio buttons 2 and 1, we obtain a circular plot. The Copy button below the graph box may be used to give a graph independent of the Windows form of the currently displayed graph which may be cut or copied into the clipboard and then pasted into any desired document.

Exponential Distribution

The exponential distribution arises in the statistical study of the strengths of certain materials and in the analysis of waiting times between random events. The probability density function is

$$p(x) = \lambda e^{-\lambda x}$$

where x is in the range $0 \leq x$ 0 is a parameter, and the cumulative density function is

$$P(x) = 1 - e^{-\lambda x}$$

We shall use the simulator to find the cumulative distribution function for the arbitrary value of 0.5 for λ .

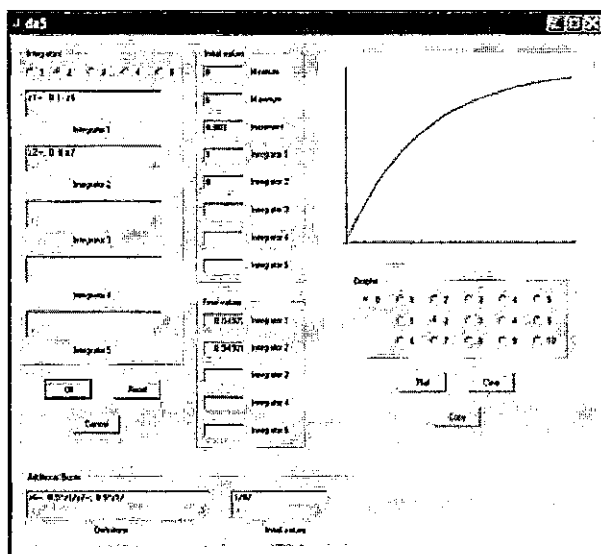


Figure 4: Exponential distribution

Since the differential equation $dy/dx = -y$ has the solution $y = e^{-x}$, the first integrator in the simulator may be defined by the expression

$$z1 = 0 \text{ I } -z1$$

Now we could write the expression

$$z1 = 0 \text{ I } -0.5 * z1$$

for the solution of the equation $dy/dx = -0.5 * y$. However it is more realistic to have the multiplication performed by one of the auxiliary buses as it would be in an actual differential analyzer. Therefore the first integrator could be defined by

$$z1 = 0 \text{ I } -z6$$

where $z6 = 0.5 * z1$. The second integrator is defined by

$$z2 = 0 \text{ II } z7$$

where $z7 = 0.5 * z1$ which may be used to give the cumulative distribution function. We should note that the definitions of the auxiliary buses and their initial values are entered in the Windows form in the Auxiliary buses boxes as the lists

$$z6 = 0.5 * z1 / z7 = 0.5 * z1 /$$

and

$$1/0/$$

respectively, with each definition or initial value terminated by a forward slash /. The graph displayed in the Windows form is that of the cumulative density function of the exponential distribution.

Acknowledgements

I wish to thank Arthur Porter for stimulating my interest in Meccano differential analyzers and for his continuing interest, and Tim Robinson for his many helpful comments throughout the course of this work.

References

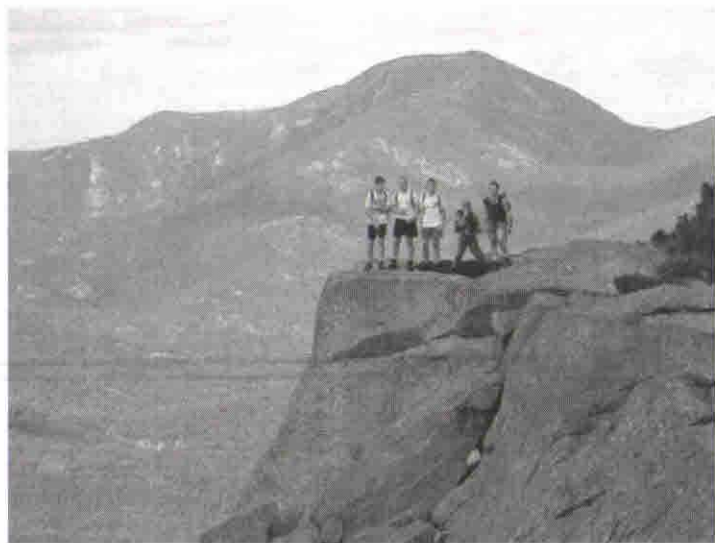
- [1] Lawrence, Bonita, 2007. Personal communication.
- [2] Porter, Arthur, 2003. "Building the Manchester differential analyzers: A personal reflection." *Annals of the History of Computing*, Vol. 25, N°2, pp. 86-92.
- [3] Robinson, Tim, 2005a. "The Meccano set computers." *IEEE Control Systems Magazine*, Vol. 25, N°3, pp. 74-83.
- [4] Robinson, Tim, 2005b. Tim Robinson's Differential Analyzer.
http://www.meccano.us/differential_analyzers/robinson_da/

GPS and J Propel an Adirondack Hiking Guide

by Cliff Reiter (reiterc@lafayette.edu)

Prologue

Last year I bought a runner's GPS based watch to get accurate feedback on the time and distance of my runs. Soon thereafter, an article appeared in the college paper [3] encouraging running. It showed several routes on local maps. That inspired me to try to create maps that showed my runs. GIS (Geographic Information Systems) are becoming common and there are web sites devoted to making maps of running routes. However, my attempts to utilize them were frustrating. I found public sources for scanned USGS topographic maps and I began overlaying them with the GPS data using J. Soon I was routinely displaying the routes for my local runs.



Swiss Family Reiter

My next goal was to use this method to show routes on hiking maps. In a paint program, I had hand marked hiking routes for many years for my Adirondack

hiking gallery maps [6]. The prospect of automating the process using higher quality topographic maps with accurate position data intrigued me. I successfully created some hiking route maps that way and began to ponder further possibilities such as automatically enhancing the maps with mile markers and other information.

I had years of experience hiking in the Adirondacks. I had tens of thousands of photographs from those hikes. I had recently found the process of publishing the third edition of my visualization book [7] at *Lulu.com* efficient. Now, I also had a fairly routine way to create hiking route maps. I decided to write an Adirondack hiking guide [8]. Writing the hiking guide was joy. "Off gathering data for my book" became a code phrase for "gone hiking", and when I was back, there was the fun of analysing the data in J.

However, unforeseen challenges were ahead. As I enhanced the book to include time and distance data along the route, I realized the distance accumulator on the watch sometimes ran backwards. Apparently it measures progress in the direction of motion. Thus, if you are running, it is perfectly accurate (so long as you are not running in a small circle). However, if you are grinding your way up a steep trail making tight switchbacks, the distance meter was often running backwards. Fortunately the latitude-longitude-altitude data was also saved so recovering the distance data was a matter of some interesting modelling directly from latitude-longitude-altitude data. I eventually realized that the altitude data from the hike could be used to compute "cumulative ascent" which seems useful for gauging the difficulty of a route, and is not given in the standard trail guide [2]. However, the raw altitude data is very noisy and some more modelling is needed. The next section describes the general way in which J was used to create the base maps but this note focuses on the subsequent process of enhancing the maps with GPS data.

Maps

While the first USGS maps I worked with were from Pennsylvania [5], I will describe the general process of preparing the New York maps from [4] to create the base maps that were used for my hiking guide. The 7.5 minute maps from the Adirondack region are mostly doubles. They are typically around 9475×6100 pixels. These were scanned from hard-copy topographic maps and stored as huge TIF files. I wanted to break these into pieces that can be easily enhanced in J. Then, since my routes will not respect map boundaries, relevant pieces can be re-assembled as necessary. The scanned maps typically were misaligned about half a degree from horizontal; that could mean an error of over a hundred pixels. They were too large to rotate using the `image3` add-on in a 32-bit version of J. Careful rotation of the maps in GIMP [1], followed by trimming (the maps had significant

margins) tended to leave images that had edge artefacts that were usually less than ten pixels across. Reading the trimmed images (`raw_read_image`) into J and using the `image3` add-on allowed for systematic splitting of the doubles and re-scaling images so that each 7.5 quadrant was 5460×3934 pixels. The re-scaling was done by a variant of `resize_image` from `image3` that does not preserve aspect ratio, as follows.

```
rescale_image=: 4 : 0
szi=.2{.$y
szo=.|.x
ind=.(<"0 szi%szo) <.*&.> <@i."0 szo
(<ind){y
)
```

There was a bit more to accomplish before the base maps were ready. In a few spots landmark names needed to be added. Trickier was that many trails had changed.

If a portion of a trail had been re-routed, the GPS route would overdraw only a portion of the dash-dash-dash trail from the original map, leaving the dashes where the trail had been re-routed. If the re-routed portion was small enough, those dashes are a sort of quaint historical piece of information. However, if a trail was on private land and is now closed, or had been completely decommissioned, then it was important to remove all the dashes marking the old trail to avoid confusing hikers.

I marked the trails for removal in a broad unused colour (magenta) on a copy of the map and then used a filter to guess whether to replace black pixels in the original that corresponded to magenta in the marked image with a suitable colour. Selecting the suitable colour is complicated: there is a desire to preserve contour markings when possible and use an appropriate background colour for forest, marsh or open rock in general. Our algorithm is probably not optimal, but we simply ordered possible colours based upon the likely importance; for example, the colour of contours was highest; for the filter we used the highest ranked colour that appeared in a small neighbourhood of the pixels that needed to be replaced. The details are beyond the discussion here.

We will now presume we have obtained a suitable base map, we know the latitude and longitude of its corners, and we want to add GPS route information to it. However, we first discuss the GPS data.

GPS Data

Exporting the GPS data from the watch led to XML files containing quite a bit of information. Since each exported data set approached 100Mb and had overlapping data, and we wanted to archive our running/hiking information every couple of weeks, it became inefficient to reparse every lap from every GPS history file every time we wanted some tracking data. Thus, we actually have a J utility that checks for new GPS data files, parses them, and creates appropriate J arrays giving summary and trackpoint information to be archived. The beginning of the XML version of hike 175 in those archives is given below. On average a trackpoint is saved every few seconds.

```
<Lap StartTime="2007-08-21T11:07:34Z">
  <TotalTimeSeconds>33836.120000</TotalTimeSeconds>
  <DistanceMeters>19909.160156</DistanceMeters>
  <MaximumSpeed>2.483250</MaximumSpeed>
  <Calories>7284</Calories>
  <Intensity>Active</Intensity>
  <Cadence>255</Cadence>
  <TriggerMethod>Manual</TriggerMethod>
  <Track>
    <Trackpoint>
      <Time>2007-08-21T11:07:34Z</Time>
      <Position>
        <LatitudeDegrees>44.020833</LatitudeDegrees>
        <LongitudeDegrees>-73.827731</LongitudeDegrees>
      </Position>
      <AltitudeMeters>645.181885</AltitudeMeters>
      <DistanceMeters>0.000000</DistanceMeters>
      <SensorState>Absent</SensorState>
    </Trackpoint>
    <Trackpoint>
      <Time>2007-08-21T11:07:35Z</Time>
      <Position>
        <LatitudeDegrees>44.020825</LatitudeDegrees>
        <LongitudeDegrees>-73.827749</LongitudeDegrees>
      </Position>
      <AltitudeMeters>645.181885</AltitudeMeters>
      <DistanceMeters>1.611095</DistanceMeters>
      <SensorState>Absent</SensorState>
    </Trackpoint>
  </Track>
</Lap>
```

Since the format for exported data presumably varies with the GPS device, we do not attempt to offer a general purpose script for extraction. However, a few utilities give the core that we required.

First is a function `headnum` that finds the number that is the leading portion of a string.

```
headnum=:[: ".@, (0 i.~ ] e. '0123456789.-'_"_){.]
```

Next is a function to convert to timestamps (with thanks to Raul Miller).

```
toTS=: 0 ". e.&(~." : i.10) ` (' ' &, :)
```

The `dyad cuts` cuts on the text given as a left argument, discarding the text.

```
cuts=: 4 : ' (x E. y) <@((#x)&.) ;.1 y'
```

Once the data has been cut into laps, the trackpoint information can be extracted with `tpx_in_lap`, given below. The result is a five column matrix. The columns correspond to 'time from start (hours)', 'latitude', 'longitude', 'altitude (metres)', and 'distance from start (miles)'.

```
tpx_in_lap=: 3 : 0
y=.('<Trackpoint>' E. y)<:.1 y
y=. '<Time>'&cuts& y
z0=.,/(i.&'{.}') &> y
y=. '<LatitudeDegrees>'&cuts& y
z=. ,headnum &> y
y=. '<LongitudeDegrees>'&cuts& y
z=. z,.,headnum &> y
y=. '<AltitudeMeters>'&cuts& y
z=. z,.,headnum &> y
y=. '<DistanceMeters>'&cuts& y
(3600000%~(-{.)tsrep toTS z0),. z,.,0.0006214*headnum &> y
)
```

For example, if we load the example data provided on the link at [9] in the path, `pathi`, we see the first few sets of GPS trackpoint information.

```
4{.tpx_in_lap fread pathi, 'dw02_lap.txt'
0 44.0208 _73.8277 645.182 0
0.000277778 44.0208 _73.8277 645.182 0.00100113
0.001666667 44.0209 _73.8278 643.74 0.00573542
0.003055556 44.0209 _73.8278 641.337 0.00912691
```

Plotting Routes

The plotting of routes on the maps was done by opening a drawing window using the adverb `show_raw_map` that is based upon the drawing functions in the `dwin2.ijs` script from the `fvj3` add-on. For these maps, an entire map far exceeded the screen coordinates, but the drawing window, despite only being partially visible, allowed all the pixels to be manipulated. The adverb `show_raw_map` is used to open a drawing window and then copies a raw image into the window. A raw image (in the sense of the `image3` add-on) is a `h` by `w` by `3`

array of ascii characters representing RGB values. The function `gis_to_win` modifies the rescaling function `SC` (which does window coordinates to pixels conversion) so that drawing can be done using longitude-latitude data; its left argument is the longitude-latitude bounds for the map shown.

```

    show_raw_map=:1 : 0
:
WIN_WH=:|.2{.$y
x dwin m
glpixels 0 0,WIN_WH,,(256 256 256&#.@(a.&i.))"_1 y
)

gis_to_win=:4 : '(1 _1*x)+"1 SC |."1 y'
```

We will describe many details of the route plotting, but all the details may be found in scripts and sample data on a link at [9]. First we read the hike trackpoint data from a raw data file. Then the time, latitude, longitude, altitude, and distance data are separated.

```

tpx0=:ja_read pathi,'hike175.ja'
't0 {a0 lo0 a0 d0'=:|:tpx0
```

The given times are fine, but the distance data needs to be modelled by the function `trek_d` that we will discuss in detail in the next section.

```

ts=:t0
ds=:trek_d tpx0
```

We next obtain the positions in the data for mile markers and a final marker. The latitude-longitude track points are split into three pieces so that the trail-less portion can be shown in a different colour. The text to annotate is computed.

<code>i=:ds I. (i.@>.,,)]}{:ds</code>	NB. position of mile markers
<code>tps=:({a0),.lo0</code>	NB. track points
<code>tps0=:({n0=:440}){.tps</code>	NB. first segment on trail
<code>tps1=:({n1=:2040}){.n0}.tps</code>	NB. trail-less segment
<code>tps2=:({n0+n1}){.tps</code>	NB. last segment on trail
<code>TXpt=:i{tps</code>	NB. mile marker lat-long
<code>TX0=:({4j1 ":",.i{ds},"1 'mi'</code>	NB. mile marker text
<code>TX1=:({4j1 ":",.i{ts},"1 'hr'</code>	NB. mile marker times

We set an offset of a few pixels. This can account for mis-registered maps or the offset from a pen's coordinates and its centre.

```
dxdy=:_10 _5
```

```
NB. map registration offset
```

We read a full topographic base map and then select a suitable portion using `n_adj_b`. That function selects a portion of an image according to proportional 0-to-1 coordinates as its left argument (lower left and upper right corners are specified).

```
a=:raw_read_image pathi,'h48.png'
```

```
view_image b:=(nc=:0.3 0.1 0.9 0.7) n_adj_b a
```

Next the map portion is shown, and the three portions of the hike are drawn in appropriate colours. Then mile markers and labelling text are drawn. Some details are suppressed, but all the details appear in [9].

```
(nc n_adj_co h48_co) 'lc' show_raw_map b
```

```
NB. draw three segments using appropriate colours
```

```
Ctext=:50 0 150
```

```
NB. colour for text
```

```
Ctrail=:255 255 0
```

```
NB. colour for trails
```

```
Ctless=:140 50 0
```

```
NB. colour for trail-less routes
```

```
dxdy(Ctrail;5 0) draw_trek tps0
```

```
dxdy(Ctless;5 0) draw_trek tps1
```

```
dxdy(Ctrail;5 0) draw_trek tps2
```

```
NB. draw mile points
```

```
dxdy (Ctext;15 0) draw_trek"2 ,:~"1 TXpt
```

```
NB. add some of text to map
```

```
gltextcolour '' [glrgb Ctext
```

```
15 20 '"Lucida Console" 16 bold'
```

```
draw_text (nn{.TX0) (nn=:8){.TXpt
```

Lastly, since the text may be difficult to read against some backgrounds, we apply a filter, `textedge`, to create a 2-pixel wide white shadow around the text. A side effect of the drawing commands was that a global variable, `trek_z`, giving the image was created.

```
view_image c=:2 textedge trek_z
```

The result of constructing and enhancing this map as above is shown in Figure 1.

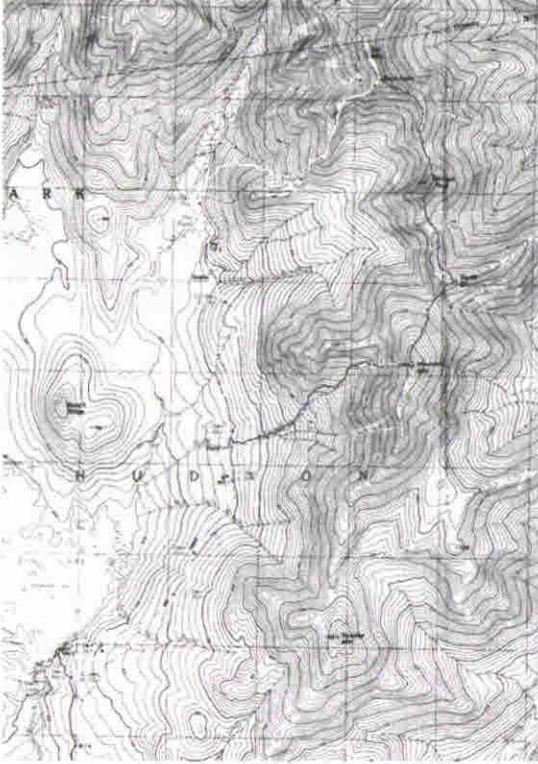


Figure 1. A Dix Wilderness Route

Distance Modelling

As we have noted, the distance data obtained directly from the watch is fine for runs, but is not accurate for hiking, most notably in situations where many small switchbacks are being used. We used the function `trek_d` to obtain the desired distance data. The core of that modelling is using a formula for converting pairs of latitude-longitude data into distance data based upon viewing the earth as an ellipsoid. Some global data and the function `lat_o_to_d` to accomplish that are given below. It is based upon an implementation by Chris Veness [10] of the Vincenty formula [11].

The arguments are two latitude-longitude pairs in degrees and the result is the miles between the points.

```

lalo_a=:6378137
lalo_b=:6356752.3142
20j15":%lalo_f%=(lalo_a-lalo_b)%lalo_a

lalo_to_d=: 4 : 0"1
'la1 lo1'=.x
'la2 lo2'=.y
L=.(2p1%360)*lo2-lo1
U1=._3 o.(1-lalo_f)*tan (2p1%360)*la1
U2=._3 o.(1-lalo_f)*tan (2p1%360)*la2
lam=.L
lamp=.2p1
while. 1e_12<|lam-lamp do.
  sins=.%:(*(cos U2)*sin lam)+*((cos U1)*sin U2)-(sin
U1)*(cos U2)*cos lam
  coss=.(sin U1)*sin U2+(cos U1)*(cos U2)*cos lam
  s=._3 o. sins%co
  sina=.(cos U1)*(cos U2)*(sin lam)%sins
  cos2a=. 1-:sina
  cos2sm=.coss-2*(sin U1)*(sin U2)%cos2a
  C=.(lalo_f%16)*cos2a*4+lalo_f*4-3*cos2a
  lamp=.lam
  lam=.L+(1-C)*lalo_f*sina*s+C*sins*cos2sm+C*coss*_1+2**cos2sm
end.
u2=.cos2a*((lalo_a^2)-lalo_b^2)%lalo_b^2
A=.1+(u2%16384)*4096+u2*_768+u2*320-175*u2
B=.(u2%1024)*256+u2*_128+u2*74-47*u2
ds=.B*sins*cos2sm+(B%4)*(coss*(*_1+2**cos2sm))-(B%6)*cos2sm* ...

(_3+4**sins)*_3+4**cos2sm
0.0006214*lalo_b*A*s-ds
)
```

Here are a few other utilities used for the distance determination. Others are loaded as part of `filter1.ijs` which is loaded when the scripts at [9] are run. The main function, `trek_d`, gently smooths the latitude and longitude data. The horizontal distance is computed using the `lalo_to_d` function on the smoothed data. Then the altitude data is smoothed (twice) by the fairly intense 15-point wide Spencer averaging. Then, using the Pythagorean theorem, the horizontal and vertical distances are blended. While one might argue for many variant schemes, we selected several test data sets and selected this scheme for our perception of its accuracy and robustness. The test data included runner's runs, where we expected to duplicate built-in distances, up-down hikes, where we expected the ascent to be slightly larger than the descent (due to mini switchbacks), and a couple of standard hikes, where we compared to distances in [2] (prioritised in that order).

Replacing the Spencer averaging with a wide Gaussian filter would also have been effective.

```

    Filtid=: 1 : 0
    (# m)"_ +/:(m&*)\ ]
  )

    len2=:+&.*:"1

    diff=:}. - }:

    ]wts=: (|.,.)74 67 46 21 3 _5 _6 _3%320
    locspen=: (+/ . *)&wts
    spencer=: 15 locspen\ (7 # {.) , ] , 7 # {

    trek_d=:3 : 0
    't0 la0 lo0 a0 d0'=. |:y
    sla=.(1 gauss 3)Filtid conext^:1 la0
    slo=.(1 gauss 3)Filtid conext^:1 lo0
    stp=.sla,.slo
    sa=.spencer a0*0.0006214
    td=.(spencer diff sa) len2 (}.stp) lalo_to_d }:stp
    +/\0,td
  )

```

In the context of the example given in the previous section and [9] we can compare the watch distance, d_0 , and the modelled distance, ds , as follows. Notice that at each step the differences are small, but the total effect is significant.

```

    5{.d0,.ds
           0           0
0.00100113 0.00163509
0.00573542 0.00425683
0.00912691 0.00704023
0.00964432 0.010248

```

```

    _5{.d0,.ds
12.3476 14.2734
 12.354 14.2789
12.3599 14.2854
12.3677 14.2914
12.3716 14.2953

```

Cumulative Ascent Modelling

The altitude data obtained from the GPS watch is notably noisy. That was indirectly noted in the previous section where the altitude data received heavy smoothing. Based upon visual inspection of familiar routes, it appears that the

altitude data shows unreal bumps, often near buildings, underpasses, cliffs, cols and summits. In this section we are interested in obtaining a single 'intensity of hike' statistic, namely the cumulative ascent over the entire hike.

We handle the noise in the altitude data by applying Gaussian smoothing. We use fairly heavy-handed smoothing, using a 21-point wide Gaussian filter. Since track points are typically a few seconds apart, this means that the smoothed data typically results from a couple of minutes' worth of hiking data. Figure 2 shows some of the altitude data from our hike (over South Dix and Hough and then partway up Dix) along with the smoothed version of the altitude data. Again, many variants could be chosen, but we selected several hikes where we had a good sense of what the answer should be, and selected this particular model due to our perception of its accuracy.

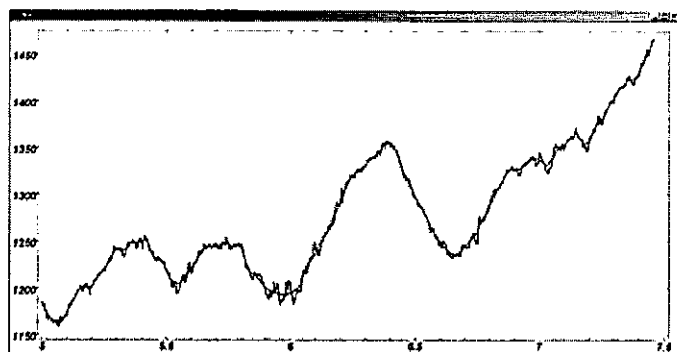


Figure 2. Altitude and Smoothed Altitude

Computing the cumulative ascent is accomplished by smoothing the data and summing the positive differences via the function `cum_alt` below. The result is given in feet.

```
cum_alt=:3 : 0
3.2808399 **/(#~0&<)diff(7 gauss 21)filt1d y
)

cum_alt a0
5375.66
```

This is an intense hike indeed. It is over 14 miles long, much of it is trail-less, and it ascends more than a mile along the way.

Conclusion

We found J very useful for the routine addition of hiking routes to base maps where the GPS data was from a runner's watch. J also allowed for some standardisation and enhancement of the base maps. J really shone when it became clear that in order to obtain accurate hiking distance and altitude data we needed to model them from the raw GPS data.

References

- [1] Spencer Kimball, Peter Mattis, Michael Natterer, Sven Neumann, et al., *GIMP* (GNU Image Manipulation Program).
- [2] Tony Goodwin, ed., *Adirondack Trails, High Peaks Region, 13th edition*, The Adirondack Mountain Club, Inc., 2004.
- [3] Brian Mason, *Run, Lafayette, Run!*, *The Lafayette*, 6 April 2007.
- [4] New York State GIS Clearinghouse, 1:24,000 Digital Raster Quadrangles, <http://www.nysgis.state.ny.us/gisdata/quads/drg24/index.htm>.
- [5] PA Spatial Data Access, 7.5 minute Digital Raster Graphics for Pennsylvania, <http://www.pasda.psu.edu/data/drg24k/>.
- [6] Cliff Reiter, *Take a Hike with Cliff, Adirondack Hikes*, <http://www.lafayette.edu/~reiterc/hikes/adir/index.html>.
- [7] Cliff Reiter, *Fractals, Visualization and J*, 3rd Edition, <http://www.lafayette.edu/~reiterc/j/fvj3/index.html>, Lulu.com, 2007.
- [8] Cliff Reiter, *Witness the Forever Wild, A Guide to Favorite Hikes around the Adirondack High Peaks*, <http://www.lafayette.edu/~reiterc/wfw/index.html>, Lulu.com, 2008.
- [9] Cliff Reiter, GPS and J Propel an Adirondack Hiking Guide Example, <http://www.lafayette.edu/~reiterc/j/oevector/index.html>.
- [10] Chris Veness, Vincenty formula for distance between two Latitude/Longitude points, *Movable Type Scripts*, <http://www.movable-type.co.uk/scripts/latlong-vincenty.html>.
- [11] T. Vincenty, Direct and Inverse Solutions of Geodesics on the Ellipsoid with Application of Nested Equations, *Survey Review*, XXII, 176, 88-93, 1975.

Index to Advertisers

Vector Back Numbers

5

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles to the Vector Working Group via the Editor:

The Editor, Vector
81 South Hill Park
London
NW3 2SS

Tel: +44 (0) 1439-788385
Email: editor@vector.org.uk

Please supply as much material as possible in machine-readable form. Our preferred format is HTML, using simple markup – we apply Vector's style sheet. Source code can be tagged with `<pre>` or `<code>` tags as usual. Authors wishing to use MS Word should install a copy of the APL385 Unicode TrueType font and use Unicode APL symbols if their interpreter permits 'copy as Unicode' from the APL session. Otherwise we can convert from APL+Win mappings if necessary.

Camera-ready artwork (e.g. advertisements) and 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

Subscribing to Vector

Your Vector subscription includes membership of the British APL Association, which is open to anyone interested in APL or related languages. The membership year runs from 1st May to 30th April.

Name: _____
 Address: _____

 Postcode / Country: _____
 Telephone Number: _____
 Email Address: _____

UK private membership	£20	☐
Overseas private membership	£22	☐
Airmail supplement (not needed for Europe)	£4	☐
UK Corporate membership	£100	☐
Corporate membership overseas	£110	☐
Sustaining membership	£500	☐
Non-voting UK member (student/OAP/unemployed)	£10	☐

PAYMENT – in Sterling or by Visa/Mastercard only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard or Visa number.

I authorise you to debit my Visa/Mastercard account

Number: _____ Expiry date: ____ / ____

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

Signature: _____

Data Protection Act:

The information supplied may be stored on computer and processed in accordance with the registration of the British Computer Society.

Send the completed form to:

BAA, c/o Rowena Small, 12 Cambridge Road, Waterbeach, CAMBRIDGE CB5 9NJ, UK

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

Committee

Chairman	Paul Grosvenor 01293-562700 paul@optima-systems.co.uk	Optima House, Mill Court Spindle Way, CRAWLEY West Sussex, RH10 1TT
Secretary	Anthony Camacho 0117-9730036 acam@blueyonder.co.uk	11 Auburn Road Redland BRISTOL, BS6 6LS
Treasurer	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB25 9NJ
Journal Editor	Stephen Taylor editor@vector.org.uk	81 South Hill Park LONDON NW3 2SS
Education	Dr Alan Mayer 01792-295779 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Activities	Ray Cannon 01252-874697 ray_cannon@compuserve.com	21 Woodbridge Rd, Blackwater Camberley, Surrey GU17 0BS
Projects	Ian Clark earthspotty@googlemail.com	
Membership enquiries	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB25 9NJ

Journal Working Group

Editor:	Stephen Taylor	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Anthony & Sylvia Camacho (0117-9730036), Ray Cannon (01252-874697), Stephen Taylor (077-1340 0852), Ian Clark	

Typeset by APL-385 with MS Word 2000

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for 'A Programming Language' - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL Italiana
via Moniferalto 1
14141 Milano, Italy
Email: slr@apl.it

Dyalog Ltd
South Barn, Minchens Court
Minchens Lane
Bramley, Hampshire RG26 5BZ
Tel: 01256 860080
Email: sales@dyalog.com
Web: www.dyalog.com

Insight Systems ApS
Nordre Strandvej 189C
DK-3450 Hellebæk, Denmark
Tel: +45 70 26 13 26
Email: info@insight.dk
Web: www.insight.dk

MicroAPL Ltd
The Rother Mill, Mill Lane
Uckfield, E. Sussex TN22 6ZA
Tel: 01825 768050
Email: sales@microapl.demon.co.uk
Web: www.microapl.co.uk/apl

Kx Systems
555 Bryant St. No. 375
Palo Alto CA 94301 USA
Tel: +1 866 kxbias
Email: general.enquiries@info@kx.com
In Europe: Simon Carland simon@kx.com

Optima Systems Ltd
Optima House, Millcourt, Spide Way, Croydon,
West Sussex, RH10 1HT
Tel: 01293 562700
Email: cam@optima-systems.co.uk

Compass Ltd
Compass House
60 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01483 514500

APL2000 Inc
One Research Court
Suite 140
Rockville MD 20850 USA
Tel: +1 (301) 208 7150
Email: sales@apl2000.com
Web: www.apl2000.com

HMW Computing
Hamilton House
1 Temple Avenue
LONDON EC4Y 0JA
Tel: 0207 4010 469
Email: HMW@btinternet.com

Sollton Inc
24 Victoria Street, Suite 2100
Toronto, Ontario M5C 1Y2, Canada
Tel: +1 (416) 364 9855
Email: sales@sollton.com
Web: www.sollton.com

First Derivatives plc
First Derivatives House
Kilmorey Business Park, Kilmorey Street,
Newry, Co. Down, NI, Ireland BT34 2DZ
Tel: 028 3025 2242
Email: enquiries@firstderivatives.com

The Carlisle Group
524 Jefferson Avenue, Scranton PA 18510
USA
Tel: +1 (570) 936 2036
Web: www.carlislegroup.com